CHAPTER 15

Query Processing

Query processing refers to the range of activities involved in extracting data from a database. The activities include translation of queries in high-level database languages into expressions that can be used at the physical level of the file system, a variety of query-optimizing transformations, and actual evaluation of queries.

15.1 Overview

The steps involved in processing a query appear in Figure 15.1. The basic steps are:

1. Parsing and translation.
2. Optimization.
3. Evaluation.

Before query processing can begin, the system must translate the query into a usable form. A language such as SQL is suitable for human use, but it is ill suited to be the system's internal representation of a query. A more useful internal representation is one based on the extended relational algebra.

Thus, the first action the system must take in query processing is to translate a given query into its internal form. This translation process is similar to the work performed by the parser of a compiler. In generating the internal form of the query, the parser checks the syntax of the user's query, verifies that the relation names appearing in the query are names of the relations in the database, and so on. The system constructs a parse-tree representation of the query, which it then translates into a relational-algebra expression. If the query was expressed in terms of a view, the translation phase also replaces all uses of the view by the relational-algebra expression that defines the view.[1] Most compiler texts cover parsing in detail.

Query → parse-tree → relational algebra

[1] For materialized views, the expression defining the view has already been evaluated and stored. Therefore, the stored relation can be used, instead of uses of the view being replaced by the expression defining the view. Recursive views are handled differently, via a fixed-point procedure, as discussed in Section 5.4 and Section 27.4.7.

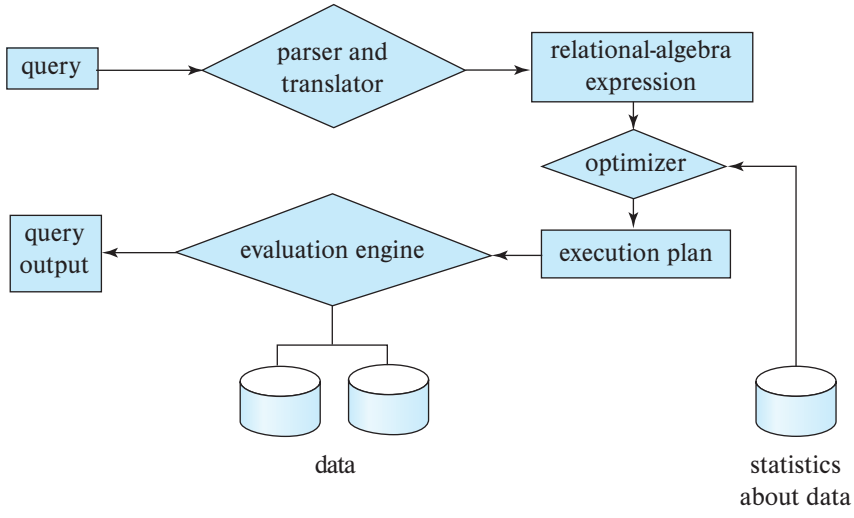


Figure 15.1 Steps in query processing.

Given a query, there are generally a variety of methods for computing the answer. For example, we have seen that, in SQL, a query could be expressed in several different ways. Each SQL query can itself be translated into a relational-algebra expression in one of several ways. Furthermore, the relational-algebra representation of a query specifies only partially how to evaluate a query; there are usually several ways to evaluate relational-algebra expressions. As an illustration, consider the query:

select *salary*
from *instructor*
where *salary* < 75000;

This query can be translated into either of the following relational-algebra expressions:

- $\sigma_{salary<75000}\ (\Pi_{salary}\ (instructor))$
- $\Pi_{salary}\ (\sigma_{salary<75000}\ (instructor))$

Further, we can execute each relational-algebra operation by one of several different algorithms. For example, to implement the preceding selection, we can search every tuple in *instructor* to find tuples with salary less than 75000. If a B^+-tree index is available on the attribute *salary*, we can use the index instead to locate the tuples.

To specify fully how to evaluate a query, we need not only to provide the relational-algebra expression, but also to annotate it with instructions specifying how to evaluate each operation. Annotations may state the algorithm to be used for a specific operation or the particular index or indices to use. A relational-algebra operation annotated

with instructions on how to evaluate it is called an **evaluation primitive**. A sequence of primitive operations that can be used to evaluate a query is a **query-execution plan** or **query-evaluation plan**. Figure 15.2 illustrates an evaluation plan for our example query, in which a particular index (denoted in the figure as "index 1") is specified for the selection operation. The **query-execution engine** takes a query-evaluation plan, executes that plan, and returns the answers to the query.

The different evaluation plans for a given query can have different costs. We do not expect users to write their queries in a way that suggests the most efficient evaluation plan. Rather, it is the responsibility of the system to construct a query-evaluation plan that minimizes the cost of query evaluation; this task is called *query optimization*. Chapter 16 describes query optimization in detail.

Once the query plan is chosen, the query is evaluated with that plan, and the result of the query is output.

The sequence of steps already described for processing a query is representative; not all databases exactly follow those steps. For instance, instead of using the relational-algebra representation, several databases use an annotated parse-tree representation based on the structure of the given SQL query. However, the concepts that we describe here form the basis of query processing in databases.

In order to optimize a query, a query optimizer must know the cost of each operation. Although the exact cost is hard to compute, since it depends on many parameters such as actual memory available to the operation, it is possible to get a rough estimate of execution cost for each operation.

In this chapter, we study how to evaluate individual operations in a query plan and how to estimate their cost; we return to query optimization in Chapter 16. Section 15.2 outlines how we measure the cost of a query. Section 15.3 through Section 15.6 cover the evaluation of individual relational-algebra operations. Several operations may be grouped together into a **pipeline**, in which each of the operations starts working on its input tuples even as they are being generated by another operation. In Section 15.7, we examine how to coordinate the execution of multiple operations in a query evaluation

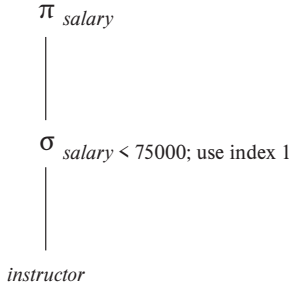


Figure 15.2 A query-evaluation plan.

plan, in particular, how to use pipelined operations to avoid writing intermediate results to disk.

15.2 Measures of Query Cost

There are multiple possible evaluation plans for a query, and it is important to be able to compare the alternatives in terms of their (estimated) cost, and choose the best plan. To do so, we must estimate the cost of individual operations and combine them to get the cost of a query evaluation plan. Thus, as we study evaluation algorithms for each operation later in this chapter, we also outline how to estimate the cost of the operation.

The cost of query evaluation can be measured in terms of a number of different resources, including disk accesses, CPU time to execute a query, and, in parallel and distributed database systems, the cost of communication. (We discuss parallel and distributed database systems in Chapter 21 through Chapter 23.)

For large databases resident on magnetic disk, the I/O cost to access data from disk usually dominates the other costs; thus, early cost models focused on the I/O cost when estimating the cost of query operations. However, with flash storage becoming larger and less expensive, most organizational data today can be stored on solid-state drives (SSDs) in a cost effective manner. In addition, main memory sizes have increased significantly, and the cost of main memory has decreased enough in recent years that for many organizations, organizational data can be stored cost-effectively in main memory for querying, although it must of course be stored on flash or magnetic storage to ensure persistence.

With data resident in-memory or on SSDs, I/O cost does not dominate the overall cost, and we must include CPU costs when computing the cost of query evaluation. We do not include CPU costs in our model to simplify our presentation, but note that they can be approximated by simple estimators. For example, the cost model used by PostgreSQL (as of 2018) includes (i) a CPU cost per tuple, (ii) a CPU cost for processing each index entry (in addition to the I/O cost), and (iii) a CPU cost per operator or function (such as arithmetic operators, comparison operators, and related functions). The database has default values for each of these costs, which are multiplied by the number of tuples processed, the number of index entries processed, and the number of operators and functions executed, respectively. The defaults can be changed as a configuration parameter.

We use the *number of blocks transferred* from storage and the *number of random I/O accesses*, each of which will require a disk seek on magnetic storage, as two important factors in estimating the cost of a query-evaluation plan. If the disk subsystem takes an average of t_T seconds to transfer a block of data and has an average block-access time (disk seek time plus rotational latency) of t_S seconds, then an operation that transfers b blocks and performs S random I/O accesses would take $b * t_T + S * t_S$ seconds.

The values of t_T and t_S must be calibrated for the disk system used. We summarize performance data here; see Chapter 12 for full details on storage systems. Typical values for high-end magnetic disks in the year 2018 would be $t_S = 4$ milliseconds and $t_T = 0.1$

milliseconds, assuming a 4-kilobyte block size and a transfer rate of 40 megabytes per second.[2]

Although SSDs do not perform a physical seek operation, they have an overhead for initiating an I/O operation; we refer to the latency from the time an I/O request is made to the time when the first byte of data is returned as t_S. For mid-range SSDs in 2018 using the SATA interface, t_S is around 90 microseconds, while the transfer time t_T is about 10 microseconds for a 4-kilobyte block. Thus, SSDs can support about 10,000 random 4-kilobyte reads per second, and they support 400 megabytes/second throughput on sequential reads using the standard SATA interface. SSDs using the PCIe 3.0x4 interface have smaller t_S, of 20 to 60 microseconds, and much higher transfer rates of around 2 gigabytes/second, corresponding to t_T of 2 microseconds, allowing around 50,000 to 15,000 random 4-kilobyte block reads per second, depending on the model.[3]

For data that are already present in main memory, reads happen at the unit of cache lines, instead of disk blocks. But assuming entire blocks of data are read, the time to transfer t_T for a 4-kilobyte block is less than 1 microsecond for data in memory. The latency to fetch data from memory, t_S, is less than 100 nanoseconds.

Given the wide diversity of speeds of different storage devices, database systems must ideally perform test seeks and block transfers to estimate t_S and t_T for specific systems/storage devices, as part of the software installation process. Databases that do not automatically infer these numbers often allow users to specify the numbers as part of configuration files.

We can refine our cost estimates further by distinguishing block reads from block writes. Block writes are typically about twice as expensive as reads on magnetic disks, since disk systems read sectors back after they are written to verify that the write was successful. On PCIe flash, write throughput may be about 50 percent less than read throughput, but the difference is almost completely masked by the limited speed of SATA interfaces, leading to write throughput matching read throughput. However, the throughput numbers do not reflect the cost of erases that are required if blocks are overwritten. For simplicity, we ignore this detail.

The cost estimates we give do not include the cost of writing the final result of an operation back to disk. These are taken into account separately where required.

---

[2]Storage device specifications often mention the transfer rate, and the number of random I/O operations that can be carried out in 1 second. The values t_T can be computed as block size divided by transfer rate, while t_S can be computed as $(1/N) - t_T$, where N is the number of random I/O operations per second that the device supports, since a random I/O operation performs a random I/O access, followed by data transfer of 1 block.

[3]The I/O operations per second number used here are for the case of sequential I/O requests, usually denoted as QD-1 in the SSD specifications. SSDs can support multiple random requests in parallel, with 32 to 64 parallel requests being commonly supported; an SSD with SATA interface supports nearly 100,000 random 4-kilobyte block reads in a second if multiple requests are sent in parallel, while PCIe disks can support over 350,000 random 4-kilobyte block reads per second; these numbers are referred to as the QD-32 or QD-64 numbers depending on how many requests are sent in parallel. We do not explore parallel requests in our cost model, since we only consider sequential query processing algorithms in this chapter. Shared-memory parallel query processing techniques, discussed in Section 22.6, can be used to exploit the parallel request capabilities of SSDs.

The costs of all the algorithms that we consider depend on the size of the buffer in main memory. In the best case, if data fits in the buffer, the data can be read into the buffers, and the disk does not need to be accessed again. In the worst case, we may assume that the buffer can hold only a few blocks of data—approximately one block per relation. However, with large main memories available today, such worst-case assumptions are overly pessimistic. In fact, a good deal of main memory is typically available for processing a query, and our cost estimates use the amount of memory available to an operator, M, as a parameter. In PostgreSQL the total memory available to a query, called the effective cache size, is assumed by default to be 4 gigabytes, for the purpose of cost estimation; if a query has several operators that run concurrently, the available memory has to be divided amongst the operators.

In addition, although we assume that data must be read from disk initially, it is possible that a block that is accessed is already present in the in-memory buffer. Again, for simplicity, we ignore this effect; as a result, the actual disk-access cost during the execution of a plan may be less than the estimated cost. To account (at least partially) for buffer residence, PostgreSQL uses the following "hack": the cost of a random page read is assumed to be $1/10^{th}$ of the actual random page read cost, to model the situation that 90% of reads are found to be resident in cache. Further, to model the situation that internal nodes of B^+-tree indices are traversed often, most database systems assume that all internal nodes are present in the in-memory buffer, and assume that a traversal of an index only incurs a single random I/O cost for the leaf node.

The **response time** for a query-evaluation plan (that is, the wall-clock time required to execute the plan), assuming no other activity is going on in the computer, would account for all these costs, and could be used as a measure of the cost of the plan. Unfortunately, the response time of a plan is very hard to estimate without actually executing the plan, for the following two reasons:

1. The response time depends on the contents of the buffer when the query begins execution; this information is not available when the query is optimized and is hard to account for even if it were available.
2. In a system with multiple disks, the response time depends on how accesses are distributed among disks, which is hard to estimate without detailed knowledge of data layout on disk.

Interestingly, a plan may get a better response time at the cost of extra resource consumption. For example, if a system has multiple disks, a plan A that requires extra disk reads, but performs the reads in parallel across multiple disks may, finish faster than another plan B that has fewer disk reads, but performs reads from only one disk at a time. However, if many instances of a query using plan A run concurrently, the overall response time may actually be more than if the same instances are executed using plan B, since plan A generates more load on the disks.

As a result, instead of trying to minimize the response time, optimizers generally try to minimize the total **resource consumption** of a query plan. Our model of estimating

the total disk access time (including seek and data transfer) is an example of such a resource consumption–based model of query cost.

15.3 Selection Operation

In query processing, the file scan is the lowest-level operator to access data. File scans are search algorithms that locate and retrieve records that fulfill a selection condition. In relational systems, a file scan allows an entire relation to be read in those cases where the relation is stored in a single, dedicated file.

15.3.1 Selections Using File Scans and Indices

Consider a selection operation on a relation whose tuples are stored together in one file. The most straightforward way of performing a selection is as follows:

- **A1** (linear search). In a linear search, the system scans each file block and tests all records to see whether they satisfy the selection condition. An initial seek is required to access the first block of the file. In case blocks of the file are not stored contiguously, extra seeks may be required, but we ignore this effect for simplicity.

  Although it may be slower than other algorithms for implementing selection, the linear-search algorithm can be applied to any file, regardless of the ordering of the file, or the availability of indices, or the nature of the selection operation. The other algorithms that we shall study are not applicable in all cases, but when applicable they are generally faster than linear search.

Cost estimates for linear scan, as well as for other selection algorithms, are shown in Figure 15.3. In the figure, we use h_i to represent the height of the B^+-tree, and assume a random I/O operation is required for each node in the path from the root to a leaf. Most real-life optimizers assume that the internal nodes of the tree are present in the in-memory buffer since they are frequently accessed, and usually less than 1 percent of the nodes of a B^+-tree are nonleaf nodes. The cost formulae can be correspondingly simplified, charging only one random I/O cost for a traversal from the root to a leaf, by setting $h_i = 1$.

Index structures are referred to as access paths, since they provide a path through which data can be located and accessed. In Chapter 14, we pointed out that it is efficient to read the records of a file in an order corresponding closely to physical order. Recall that a *clustering index* (also referred to as a *primary index*) is an index that allows the records of a file to be read in an order that corresponds to the physical order in the file. An index that is not a clustering index is called a *secondary index* or a *nonclustering index*.

Search algorithms that use an index are referred to as index scans. We use the selection predicate to guide us in the choice of the index to use in processing the query. Search algorithms that use an index are:

	Algorithm	Cost	Reason
A1	Linear Search	$t_S + b_r * t_T$	One initial seek plus b_r block transfers, where b_r denotes the number of blocks in the file.
A1	Linear Search, Equality on Key	Average case $t_S + (b_r/2) * t_T$	Since at most one record satisfies the condition, scan can be terminated as soon as the required record is found. In the worst case, b_r block transfers are still required.
A2	Clustering B^+-tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	(Where h_i denotes the height of the index.) Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer.
A3	Clustering B^+-tree Index, Equality on Non-key	$h_i * (t_T + t_S) + t_S + b * t_T$	One seek for each level of the tree, one seek for the first block. Here b is the number of blocks containing records with the specified search key, all of which are read. These blocks are leaf blocks assumed to be stored sequentially (since it is a clustering index) and don't require additional seeks.
A4	Secondary B^+-tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	This case is similar to clustering index.
A4	Secondary B^+-tree Index, Equality on Non-key	$(h_i + n) * (t_T + t_S)$	(Where n is the number of records fetched.) Here, cost of index traversal is the same as for A3, but each record may be on a different block, requiring a seek per record. Cost is potentially very high if n is large.
A5	Clustering B^+-tree Index, Comparison	$h_i * (t_T + t_S) + t_S + b * t_T$	Identical to the case of A3, equality on non-key.
A6	Secondary B^+-tree Index, Comparison	$(h_i + n) * (t_T + t_S)$	Identical to the case of A4, equality on non-key.

Figure 15.3 Cost estimates for selection algorithms.

- **A2 (clustering index, equality on key)**. For an equality comparison on a key attribute with a clustering index, we can use the index to retrieve a single record that satisfies the corresponding equality condition. Cost estimates are shown in Figure 15.3. To model the common situation that the internal nodes of the index are in the in-memory buffer, h_i can be set to 1.
- **A3 (clustering index, equality on non-key)**. We can retrieve multiple records by using a clustering index when the selection condition specifies an equality comparison on a non-key attribute, A. The only difference from the previous case is that multiple records may need to be fetched. However, the records must be stored consecutively in the file since the file is sorted on the search key. Cost estimates are shown in Figure 15.3.
- **A4 (secondary index, equality)**. Selections specifying an equality condition can use a secondary index. This strategy can retrieve a single record if the equality condition is on a key; multiple records may be retrieved if the indexing field is not a key.

  In the first case, only one record is retrieved. The cost in this case is the same as that for a clustering index (case A2).

  In the second case, each record may be resident on a different block, which may result in one I/O operation per retrieved record, with each I/O operation requiring a seek and a block transfer. The worst-case cost in this case is $(h_i + n) * (t_S + t_T)$, where n is the number of records fetched, if each record is in a different disk block, and the block fetches are randomly ordered. The worst-case cost could become even worse than that of linear search if a large number of records are retrieved.

  If the in-memory buffer is large, the block containing the record may already be in the buffer. It is possible to construct an estimate of the *average* or *expected* cost of the selection by taking into account the probability of the block containing the record already being in the buffer. For large buffers, that estimate will be much less than the worst-case estimate.

In certain algorithms, including A2, the use of a B^+-tree file organization can save one access since records are stored at the leaf level of the tree.

As described in Section 14.4.2, when records are stored in a B^+-tree file organization or other file organizations that may require relocation of records, secondary indices usually do not store pointers to the records.[4] Instead, secondary indices store the values of the attributes used as the search key in a B^+-tree file organization. Accessing a record through such a secondary index is then more expensive: First the secondary index is searched to find the B^+-tree file organization search-key values, then the B^+-tree file organization is looked up to find the records. The cost formulae described for secondary indices have to be modified appropriately if such indices are used.

[4]Recall that if B^+-tree file organizations are used to store relations, records may be moved between blocks when leaf nodes are split or merged, and when records are redistributed.

15.3.2 Selections Involving Comparisons

Consider a selection of the form $\sigma_{A \leq v}(r)$. We can implement the selection either by using linear search or by using indices in one of the following ways:

- **A5 (clustering index, comparison)**. A clustering ordered index (for example, a clustering B^+-tree index) can be used when the selection condition is a comparison. For comparison conditions of the form $A > v$ or $A \geq v$, a clustering index on A can be used to direct the retrieval of tuples, as follows: For $A \geq v$, we look up the value v in the index to find the first tuple in the file that has a value of $A \geq v$. A file scan starting from that tuple up to the end of the file returns all tuples that satisfy the condition. For $A > v$, the file scan starts with the first tuple such that $A > v$. The cost estimate for this case is identical to that for case A3.

  For comparisons of the form $A < v$ or $A \leq v$, an index lookup is not required. For $A < v$, we use a simple file scan starting from the beginning of the file, and continuing up to (but not including) the first tuple with attribute $A = v$. The case of $A \leq v$ is similar, except that the scan continues up to (but not including) the first tuple with attribute $A > v$. In either case, the index is not useful.

- **A6 (secondary index, comparison)**. We can use a secondary ordered index to guide retrieval for comparison conditions involving $<$, $\leq$, $\geq$, or $>$. The lowest-level index blocks are scanned, either from the smallest value up to v (for $<$ and $\leq$), or from v up to the maximum value (for $>$ and $\geq$).

  The secondary index provides pointers to the records, but to get the actual records we have to fetch the records by using the pointers. This step may require an I/O operation for each record fetched, since consecutive records may be on different disk blocks; as before, each I/O operation requires a disk seek and a block transfer. If the number of retrieved records is large, using the secondary index may be even more expensive than using linear search. Therefore, the secondary index should be used only if very few records are selected.

As long as the number of matching tuples is known ahead of time, a query optimizer can choose between using a secondary index or using a linear scan based on the cost estimates. However, if the number of matching tuples is not known accurately at compilation time, either choice may lead to bad performance, depending on the actual number of matching tuples.

To deal with the above situation, PostgreSQL uses a hybrid algorithm that it calls a *bitmap index scan*,[5] when a secondary index is available, but the number of matching records is not known precisely. The bitmap index scan algorithm first creates a bitmap with as many bits as the number of blocks in the relation, with all bits initialized to 0. The algorithm then uses the secondary index to find index entries for matching tuples, but instead of fetching the tuples immediately, it does the following. As each index

[5]This algorithm should not be confused with a scan using a bitmap index.

entry is found, the algorithm gets the block number from the index entry, and sets the corresponding bit in the bitmap to 1.

Once all index entries have been processed, the bitmap is scanned to find all blocks whose bit is set to 1. These are exactly the blocks containing matching records. The relation is then scanned linearly, but blocks whose bit is not set to 1 are skipped; only blocks whose bit is set to 1 are fetched, and then a scan within each block is used to retrieve all matching records in the block.

In the worst case, this algorithm is only slightly more expensive than linear scan, but in the best case it is much cheaper than linear scan. Similarly, in the worst case it is only slightly more expensive than using a secondary index scan to directly fetch tuples, but in the best case it is much cheaper than a secondary index scan. Thus, this hybrid algorithm ensures that performance is never much worse than the best plan for that database instance.

A variant of this algorithm collects all the index entries, and sorts them (using sorting algorithms which we study later in this chapter), and then performs a relation scan that skips blocks that do not have any matching entries. Using a bitmap as above can be cheaper than sorting the index entries.

15.3.3 Implementation of Complex Selections

So far, we have considered only simple selection conditions of the form *A op B*, where *op* is an equality or comparison operation. We now consider more complex selection predicates.

- **Conjunction:** A **conjunctive selection** is a selection of the form:

$$\sigma_{\theta_1 \wedge \theta_2 \wedge \cdots \wedge \theta_n}(r)$$

- **Disjunction:** A **disjunctive selection** is a selection of the form:

$$\sigma_{\theta_1 \vee \theta_2 \vee \cdots \vee \theta_n}(r)$$

  A disjunctive condition is satisfied by the union of all records satisfying the individual, simple conditions θ_i.

- **Negation:** The result of a selection $\sigma_{\neg\theta}(r)$ is the set of tuples of r for which the condition θ evaluates to false. In the absence of nulls, this set is simply the set of tuples in r that are not in $\sigma_\theta(r)$.

We can implement a selection operation involving either a conjunction or a disjunction of simple conditions by using one of the following algorithms:

- **A7 (conjunctive selection using one index).** We first determine whether an access path is available for an attribute in one of the simple conditions. If one is, one of the

selection algorithms A2 through A6 can retrieve records satisfying that condition. We complete the operation by testing, in the memory buffer, whether or not each retrieved record satisfies the remaining simple conditions.

To reduce the cost, we choose a θ_i and one of algorithms A1 through A6 for which the combination results in the least cost for $\sigma_{\theta_i}(r)$. The cost of algorithm A7 is given by the cost of the chosen algorithm.

- **A8 (conjunctive selection using composite index)**. An appropriate composite index (that is, an index on multiple attributes) may be available for some conjunctive selections. If the selection specifies an equality condition on two or more attributes, and a composite index exists on these combined attribute fields, then the index can be searched directly. The type of index determines which of algorithms A2, A3, or A4 will be used.

- **A9 (conjunctive selection by intersection of identifiers)**. Another alternative for implementing conjunctive selection operations involves the use of record pointers or record identifiers. This algorithm requires indices with record pointers, on the fields involved in the individual conditions. The algorithm scans each index for pointers to tuples that satisfy an individual condition. The intersection of all the retrieved pointers is the set of pointers to tuples that satisfy the conjunctive condition. The algorithm then uses the pointers to retrieve the actual records. If indices are not available on all the individual conditions, then the algorithm tests the retrieved records against the remaining conditions.

  The cost of algorithm A9 is the sum of the costs of the individual index scans, plus the cost of retrieving the records in the intersection of the retrieved lists of pointers. This cost can be reduced by sorting the list of pointers and retrieving records in the sorted order. Thereby, (1) all pointers to records in a block come together, hence all selected records in the block can be retrieved using a single I/O operation, and (2) blocks are read in sorted order, minimizing disk-arm movement. Section 15.4 describes sorting algorithms.

- **A10 (disjunctive selection by union of identifiers)**. If access paths are available on all the conditions of a disjunctive selection, each index is scanned for pointers to tuples that satisfy the individual condition. The union of all the retrieved pointers yields the set of pointers to all tuples that satisfy the disjunctive condition. We then use the pointers to retrieve the actual records.

  However, if even one of the conditions does not have an access path, we have to perform a linear scan of the relation to find tuples that satisfy the condition. Therefore, if there is even one such condition in the disjunct, the most efficient access method is a linear scan, with the disjunctive condition tested on each tuple during the scan.

The implementation of selections with negation conditions is left to you as an exercise (Practice Exercise 15.6).

15.4 Sorting

Sorting of data plays an important role in database systems for two reasons. First, SQL queries can specify that the output be sorted. Second, and equally important for query processing, several of the relational operations, such as joins, can be implemented efficiently if the input relations are first sorted. Thus, we discuss sorting here before discussing the join operation in Section 15.5.

We can sort a relation by building an index on the sort key and then using that index to read the relation in sorted order. However, such a process orders the relation only *logically*, through an index, rather than *physically*. Hence, the reading of tuples in the sorted order may lead to a disk access (disk seek plus block transfer) for each record, which can be very expensive, since the number of records can be much larger than the number of blocks. For this reason, it may be desirable to order the records physically.

The problem of sorting has been studied extensively, both for relations that fit entirely in main memory and for relations that are bigger than memory. In the first case, standard sorting techniques such as quick-sort can be used. Here, we discuss how to handle the second case.

15.4.1 External Sort-Merge Algorithm

Sorting of relations that do not fit in memory is called **external sorting**. The most commonly used technique for external sorting is the **external sort–merge** algorithm. We describe the external sort–merge algorithm next. Let M denote the number of blocks in the main memory buffer available for sorting, that is, the number of disk blocks whose contents can be buffered in available main memory.

1. In the first stage, a number of sorted **runs** are created; each run is sorted but contains only some of the records of the relation.

```
i = 0;
repeat
    read M blocks of the relation, or the rest of the relation,
         whichever is smaller;
    sort the in-memory part of the relation;
    write the sorted data to run file R_i;
    i = i + 1;
until the end of the relation
```

2. In the second stage, the runs are *merged*. Suppose, for now, that the total number of runs, N, is less than M, so that we can allocate one block to each run and have space left to hold one block of output. The merge stage operates as follows:

read one block of each of the N files R_i into a buffer block in memory;
repeat
    choose the first tuple (in sort order) among all buffer blocks;
    write the tuple to the output, and delete it from the buffer block;
    if the buffer block of any run R_i is empty **and not** end-of-file(R_i)
        then read the next block of R_i into the buffer block;
until all input buffer blocks are empty

The output of the merge stage is the sorted relation. The output file is buffered to reduce the number of disk write operations. The preceding merge operation is a generalization of the two-way merge used by the standard in-memory sort–merge algorithm; it merges N runs, so it is called an **N-way merge**.

In general, if the relation is much larger than memory, there may be M or more runs generated in the first stage, and it is not possible to allocate a block for each run during the merge stage. In this case, the merge operation proceeds in multiple passes. Since there is enough memory for $M-1$ input buffer blocks, each merge can take $M-1$ runs as input.

The initial *pass* functions in this way: It merges the first $M-1$ runs (as described in item 2 above) to get a single run for the next pass. Then, it merges the next $M-1$ runs similarly, and so on, until it has processed all the initial runs. At this point, the number of runs has been reduced by a factor of $M-1$. If this reduced number of runs is still greater than or equal to M, another pass is made, with the runs created by the first pass as input. Each pass reduces the number of runs by a factor of $M-1$. The passes repeat as many times as required, until the number of runs is less than M; a final pass then generates the sorted output.

Figure 15.4 illustrates the steps of the external sort–merge for an example relation. For illustration purposes, we assume that only one tuple fits in a block ($f_r = 1$), and we assume that memory holds at most three blocks. During the merge stage, two blocks are used for input and one for output.

15.4.2 Cost Analysis of External Sort-Merge

We compute the disk-access cost for the external sort–merge in this way: Let b_r denote the number of blocks containing records of relation r. The first stage reads every block of the relation and writes them out again, giving a total of $2b_r$ block transfers. The initial number of runs is $\lceil b_r/M \rceil$. During the merge pass, reading in each run one block at a time leads to a large number of seeks; to reduce the number of seeks, a larger number of blocks, denoted b_b, are read or written at a time, requiring b_b buffer blocks to be allocated to each input run and to the output run. Then, $\lfloor M/b_b \rfloor - 1$ runs can be merged in each merge pass, decreasing the number of runs by a factor of $\lfloor M/b_b \rfloor - 1$. The total number of merge passes required is $\lceil \log_{\lfloor M/b_b \rfloor - 1}(b_r/M) \rceil$. Each of these passes reads every block of the relation once and writes it out once, with two exceptions. First, the final pass can produce the sorted output without writing its result

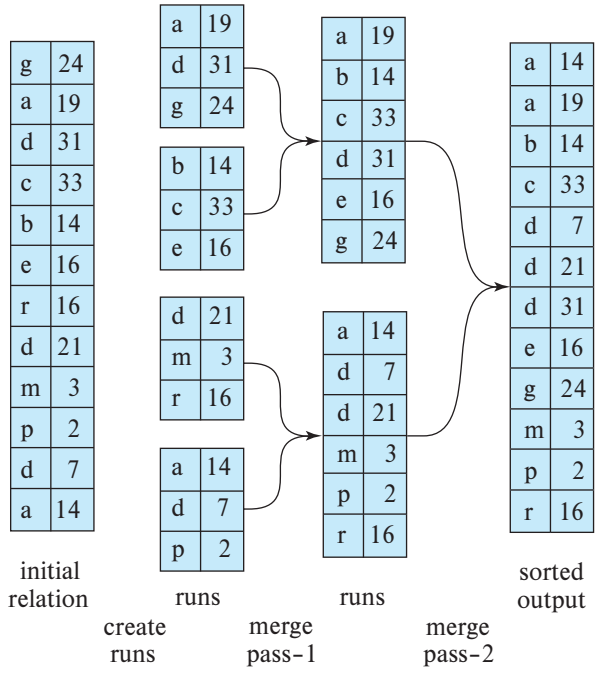


Figure 15.4 External sorting using sort-merge.

to disk. Second, there may be runs that are not read in or written out during a pass —for example, if there are $\lfloor M/b_b \rfloor$ runs to be merged in a pass, $\lfloor M/b_b \rfloor - 1$ are read in and merged, and one run is not accessed during the pass. Ignoring the (relatively small) savings due to the latter effect, the total number of block transfers for external sorting of the relation is:

$$b_r(2\lceil \log_{\lfloor M/b_b \rfloor - 1}(b_r/M) \rceil + 1)$$

Applying this equation to the example in Figure 15.4, with b_b set to 1, we get a total of $12 * (4 + 1) = 60$ block transfers, as you can verify from the figure. Note that these above numbers do not include the cost of writing out the final result.

We also need to add the disk-seek costs. Run generation requires seeks for reading data for each of the runs as well as for writing the runs. Each merge pass requires around $\lceil b_r/b_b \rceil$ seeks for reading data.[6] Although the output is written sequentially, if it is on the same disk as the input runs, the head may have moved away between writes of consecutive blocks. Thus we would have to add a total of $2\lceil b_r/b_b \rceil$ seeks for each merge pass, except the final pass (since we assume the final result is not written back to disk).

[6]To be more precise, since we read each run separately and may get fewer than b_b blocks when reading the end of a run, we may require an extra seek for each run. We ignore this detail for simplicity.

$$2\lceil b_r/M\rceil + \lceil b_r/b_b\rceil(2\lceil \log_{\lfloor M/b_b\rfloor - 1}(b_r/M)\rceil - 1)$$

Applying this equation to the example in Figure 15.4, we get a total of $8 + 12 * (2 * 2 - 1) = 44$ disk seeks if we set the number of buffer blocks per run b_b to 1.

15.5 Join Operation

In this section, we study several algorithms for computing the join of relations, and we analyze their respective costs.

We use the term **equi-join** to refer to a join of the form $r \bowtie_{r.A=s.B} s$, where A and B are attributes or sets of attributes of relations r and s, respectively.

We use as a running example the expression:

$$student \bowtie takes$$

using the same relation schemas that we used in Chapter 2. We assume the following information about the two relations:

- Number of records of *student*: $n_{student} = 5000$.
- Number of blocks of *student*: $b_{student} = 100$.
- Number of records of *takes*: $n_{takes} = 10,000$.
- Number of blocks of *takes*: $b_{takes} = 400$.

15.5.1 Nested-Loop Join

Figure 15.5 shows a simple algorithm to compute the theta join, $r \bowtie_\theta s$, of two relations r and s. This algorithm is called the **nested-loop join** algorithm, since it basically consists of a pair of nested **for** loops. Relation r is called the **outer relation** and relation s the **inner relation** of the join, since the loop for r encloses the loop for s. The algorithm uses the notation $t_r \cdot t_s$, where t_r and t_s are tuples; $t_r \cdot t_s$ denotes the tuple constructed by concatenating the attribute values of tuples t_r and t_s.

Like the linear file-scan algorithm for selection, the nested-loop join algorithm requires no indices, and it can be used regardless of what the join condition is. Extending the algorithm to compute the natural join is straightforward, since the natural join can be expressed as a theta join followed by elimination of repeated attributes by a projection. The only change required is an extra step of deleting repeated attributes from the tuple $t_r \cdot t_s$, before adding it to the result.

The nested-loop join algorithm is expensive, since it examines every pair of tuples in the two relations. Consider the cost of the nested-loop join algorithm. The number of pairs of tuples to be considered is $n_r * n_s$, where n_r denotes the number of tuples in r, and n_s denotes the number of tuples in s. For each record in r, we have to perform

```
for each tuple t_r in r do begin
    for each tuple t_s in s do begin
        test pair (t_r, t_s) to see if they satisfy the join condition θ
        if they do, add t_r · t_s to the result;
    end
end
```

Figure 15.5 Nested-loop join.

a complete scan on s. In the worst case, the buffer can hold only one block of each relation, and a total of $n_r * b_s + b_r$ block transfers would be required, where b_r and b_s denote the number of blocks containing tuples of r and s, respectively. We need only one seek for each scan on the inner relation s since it is read sequentially, and a total of b_r seeks to read r, leading to a total of $n_r + b_r$ seeks. In the best case, there is enough space for both relations to fit simultaneously in memory, so each block would have to be read only once; hence, only $b_r + b_s$ block transfers would be required, along with two seeks.

If one of the relations fits entirely in main memory, it is beneficial to use that relation as the inner relation, since the inner relation would then be read only once. Therefore, if s is small enough to fit in main memory, our strategy requires only a total $b_r + b_s$ block transfers and two seeks—the same cost as that for the case where both relations fit in memory.

Now consider the natural join of *student* and *takes*. Assume for now that we have no indices whatsoever on either relation, and that we are not willing to create any index. We can use the nested loops to compute the join; assume that *student* is the outer relation and *takes* is the inner relation in the join. We will have to examine 5000 $*$ 10,000 = 50 $*$ 10^6 pairs of tuples. In the worst case, the number of block transfers is 5000 $*$ 400 + 100 = 2,000,100, plus 5000 + 100 = 5100 seeks. In the best-case scenario, however, we can read both relations only once and perform the computation. This computation requires at most 100 + 400 = 500 block transfers, plus two seeks —a significant improvement over the worst-case scenario. If we had used *takes* as the relation for the outer loop and *student* for the inner loop, the worst-case cost of our final strategy would have been 10,000 $*$ 100 + 400 = 1,000,400 block transfers, plus 10,400 disk seeks. The number of block transfers is significantly less, and although the number of seeks is higher, the overall cost is reduced, assuming $t_S = 4$ milliseconds and $t_T = 0.1$ milliseconds.

15.5.2 Block Nested-Loop Join

If the buffer is too small to hold either relation entirely in memory, we can still obtain a major saving in block accesses if we process the relations on a per-block basis, rather

than on a per-tuple basis. Figure 15.6 shows **block nested-loop join**, which is a variant of the nested-loop join where every block of the inner relation is paired with every block of the outer relation. Within each pair of blocks, every tuple in one block is paired with every tuple in the other block, to generate all pairs of tuples. As before, all pairs of tuples that satisfy the join condition are added to the result.

The primary difference in cost between the block nested-loop join and the basic nested-loop join is that, in the worst case, each block in the inner relation s is read only once for each *block* in the outer relation, instead of once for each *tuple* in the outer relation. Thus, in the worst case, there will be a total of $b_r * b_s + b_r$ block transfers, where b_r and b_s denote the number of blocks containing records of r and s, respectively. Each scan of the inner relation requires one seek, and the scan of the outer relation requires one seek per block, leading to a total of $2 * b_r$ seeks. It is more efficient to use the smaller relation as the outer relation, in case neither of the relations fits in memory. In the best case, where the inner relation fits in memory, there will be $b_r + b_s$ block transfers and just two seeks (we would choose the smaller relation as the inner relation in this case).

Now return to our example of computing *student* $\bowtie$ *takes*, using the block nested-loop join algorithm. In the worst case, we have to read each block of *takes* once for each block of *student*. Thus, in the worst case, a total of $100 * 400 + 100 = 40{,}100$ block transfers plus $2 * 100 = 200$ seeks are required. This cost is a significant improvement over the $5000 * 400 + 100 = 2{,}000{,}100$ block transfers plus 5100 seeks needed in the worst case for the basic nested-loop join. The best-case cost remains the same—namely, $100 + 400 = 500$ block transfers and two seeks.

The performance of the nested-loop and block nested-loop procedures can be further improved:

```
for each block B_r of r do begin
    for each block B_s of s do begin
        for each tuple t_r in B_r do begin
            for each tuple t_s in B_s do begin
                test pair (t_r, t_s) to see if they satisfy the join condition
                if they do, add t_r · t_s to the result;
            end
        end
    end
end
```

Figure 15.6 Block nested-loop join.

- If the join attributes in a natural join or an equi-join form a key on the inner relation, then for each outer relation tuple the inner loop can terminate as soon as the first match is found.
- In the block nested-loop algorithm, instead of using disk blocks as the blocking unit for the outer relation, we can use the biggest size that can fit in memory, while leaving enough space for the buffers of the inner relation and the output. In other words, if memory has M blocks, we read in $M - 2$ blocks of the outer relation at a time, and when we read each block of the inner relation we join it with all the $M - 2$ blocks of the outer relation. This change reduces the number of scans of the inner relation from b_r to $\lceil b_r/(M - 2) \rceil$, where b_r is the number of blocks of the outer relation. The total cost is then $\lceil b_r/(M - 2) \rceil * b_s + b_r$ block transfers and $2\lceil b_r/(M - 2) \rceil$ seeks.
- We can scan the inner loop alternately forward and backward. This scanning method orders the requests for disk blocks so that the data remaining in the buffer from the previous scan can be reused, thus reducing the number of disk accesses needed.
- If an index is available on the inner loop's join attribute, we can replace file scans with more efficient index lookups. Section 15.5.3 describes this optimization.

15.5.3 Indexed Nested-Loop Join

In a nested-loop join (Figure 15.5), if an index is available on the inner loop's join attribute, index lookups can replace file scans. For each tuple t_r in the outer relation r, the index is used to look up tuples in s that will satisfy the join condition with tuple t_r.

This join method is called an **indexed nested-loop join**; it can be used with existing indices, as well as with temporary indices created for the sole purpose of evaluating the join.

Looking up tuples in s that will satisfy the join conditions with a given tuple t_r is essentially a selection on s. For example, consider *student* $\bowtie$ *takes*. Suppose that we have a *student* tuple with *ID* "00128". Then, the relevant tuples in *takes* are those that satisfy the selection "*ID* = 00128".

The cost of an indexed nested-loop join can be computed as follows: For each tuple in the outer relation r, a lookup is performed on the index for s, and the relevant tuples are retrieved. In the worst case, there is space in the buffer for only one block of r and one block of the index. Then, b_r I/O operations are needed to read relation r, where b_r denotes the number of blocks containing records of r; each I/O requires a seek and a block transfer, since the disk head may have moved in between each I/O. For each tuple in r, we perform an index lookup on s. Then, the cost of the join can be computed as $b_r(t_T + t_S) + n_r * c$, where n_r is the number of records in relation r, and c is the cost of a single selection on s using the join condition. We have seen in Section 15.3 how to estimate the cost of a single selection algorithm (possibly using indices); that estimate gives us the value of c.

The cost formula indicates that, if indices are available on both relations r and s, it is generally most efficient to use the one with fewer tuples as the outer relation.

For example, consider an indexed nested-loop join of *student* $\bowtie$ *takes*, with *student* as the outer relation. Suppose also that *takes* has a clustering B^+-tree index on the join attribute *ID*, which contains 20 entries on average in each index node. Since *takes* has 10,000 tuples, the height of the tree is 4, and one more access is needed to find the actual data. Since $n_{student}$ is 5000, the total cost is $100 + 5000 * 5 = 25{,}100$ disk accesses, each of which requires a seek and a block transfer. In contrast, as we saw before, 40,100 block transfers plus 200 seeks were needed for a block nested-loop join. Although the number of block transfers has been reduced, the seek cost has actually increased, increasing the total cost since a seek is considerably more expensive than a block transfer. However, if we had a selection on the *student* relation that reduces the number of rows significantly, indexed nested-loop join could be significantly faster than block nested-loop join.

15.5.4 Merge Join

The **merge-join** algorithm (also called the **sort-merge-join** algorithm) can be used to compute natural joins and equi-joins. Let $r(R)$ and $s(S)$ be the relations whose natural join is to be computed, and let $R \cap S$ denote their common attributes. Suppose that both relations are sorted on the attributes $R \cap S$. Then, their join can be computed by a process much like the merge stage in the merge–sort algorithm.

15.5.4.1 Merge-Join Algorithm

Figure 15.7 shows the merge-join algorithm. In the algorithm, *JoinAttrs* refers to the attributes in $R \cap S$, and $t_r \bowtie t_s$, where t_r and t_s are tuples that have the same values for *JoinAttrs*, denotes the concatenation of the attributes of the tuples, followed by projecting out repeated attributes. The merge-join algorithm associates one pointer with each relation. These pointers point initially to the first tuple of the respective relations. As the algorithm proceeds, the pointers move through the relation. A group of tuples of one relation with the same value on the join attributes is read into S_s. The algorithm in Figure 15.7 *requires* that every set of tuples S_s fit in main memory; we discuss extensions of the algorithm to avoid this requirement shortly. Then, the corresponding tuples (if any) of the other relation are read in and are processed as they are read.

Figure 15.8 shows two relations that are sorted on their join attribute $a1$. It is instructive to go through the steps of the merge-join algorithm on the relations shown in the figure.

The merge-join algorithm of Figure 15.7 requires that each set S_s of all tuples with the same value for the join attributes must fit in main memory. This requirement can usually be met, even if the relation s is large. If there are some join attribute values for

```
pr := address of first tuple of r;
ps := address of first tuple of s;
while (ps ≠ null and pr ≠ null) do
    begin
        t_s := tuple to which ps points;
        S_s := {t_s};
        set ps to point to next tuple of s;
        done := false;
        while (not done and ps ≠ null) do
            begin
                t_s' := tuple to which ps points;
                if (t_s'[JoinAttrs] = t_s[JoinAttrs])
                    then begin
                            S_s := S_s ∪ {t_s'};
                            set ps to point to next tuple of s;
                         end
                    else done := true;
            end
        t_r := tuple to which pr points;
        while (pr ≠ null and t_r[JoinAttrs] < t_s[JoinAttrs]) do
            begin
                set pr to point to next tuple of r;
                t_r := tuple to which pr points;
            end
        while (pr ≠ null and t_r[JoinAttrs] = t_s[JoinAttrs]) do
            begin
                for each t_s in S_s do
                    begin
                        add t_s ⋈ t_r to result;
                    end
                set pr to point to next tuple of r;
                t_r := tuple to which pr points;
            end
    end.
```

Figure 15.7 Merge join.

which S_s is larger than available memory, a block nested-loop join can be performed for such sets S_s, matching them with corresponding blocks of tuples in r with the same values for the join attributes.

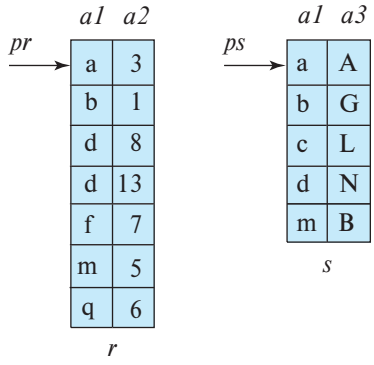


Figure 15.8 Sorted relations for merge join.

If either of the input relations r and s is not sorted on the join attributes, they can be sorted first, and then the merge-join algorithm can be used. The merge-join algorithm can also be easily extended from natural joins to the more general case of equi-joins.

15.5.4.2 Cost Analysis

Once the relations are in sorted order, tuples with the same value on the join attributes are in consecutive order. Thereby, each tuple in the sorted order needs to be read only once, and, as a result, each block is also read only once. Since it makes only a single pass through both files (assuming all sets S_s fit in memory), the merge-join method is efficient; the number of block transfers is equal to the sum of the number of blocks in both files, $b_r + b_s$.

Assuming that b_b buffer blocks are allocated to each relation, the number of disk seeks required would be $\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil$ disk seeks. Since seeks are much more expensive than data transfer, it makes sense to allocate multiple buffer blocks to each relation, provided extra memory is available. For example, with $t_T = 0.1$ milliseconds per 4-kilobyte block, and $t_S = 4$ milliseconds, the buffer size is 400 blocks (or 1.6 megabytes), so the seek time would be 4 milliseconds for every 40 milliseconds of transfer time; in other words, seek time would be just 10 percent of the transfer time.

If either of the input relations r and s is not sorted on the join attributes, they must be sorted first; the cost of sorting must then be added to the above costs. If some sets S_s do not fit in memory, the cost would increase slightly.

Suppose the merge-join scheme is applied to our example of *student* $\bowtie$ *takes*. The join attribute here is *ID*. Suppose that the relations are already sorted on the join attribute *ID*. In this case, the merge join takes a total of $400+100 = 500$ block transfers. If we assume that in the worst case only one buffer block is allocated to each input relation (that is, $b_b = 1$), a total of $400 + 100 = 500$ seeks would also be required; in reality b_b can be set much higher since we need to buffer blocks for only two relations, and the seek cost would be significantly less.

Suppose the relations are not sorted, and the memory size is the worst case, only three blocks. The cost is as follows:

1. Using the formulae that we developed in Section 15.4, we can see that sorting relation *takes* requires $\lceil \log_{3-1}(400/3) \rceil = 8$ merge passes. Sorting of relation *takes* then takes $400 * (2\lceil \log_{3-1}(400/3) \rceil + 1)$, or 6800, block transfers, with 400 more transfers to write out the result. The number of seeks required is $2 * \lceil 400/3 \rceil + 400 * (2 * 8 - 1)$ or 6268 seeks for sorting, and 400 seeks for writing the output, for a total of 6668 seeks, since only one buffer block is available for each run.
2. Similarly, sorting relation *student* takes $\lceil \log_{3-1}(100/3) \rceil = 6$ merge passes and $100 * (2\lceil \log_{3-1}(100/3) \rceil + 1)$, or 1300, block transfers, with 100 more transfers to write it out. The number of seeks required for sorting *student* is $2 * \lceil 100/3 \rceil + 100 * (2 * 6 - 1) = 1168$, and 100 seeks are required for writing the output, for a total of 1268 seeks.
3. Finally, merging the two relations takes $400 + 100 = 500$ block transfers and 500 seeks.

Thus, the total cost is 9100 block transfers plus 8932 seeks if the relations are not sorted, and the memory size is just 3 blocks.

With a memory size of 25 blocks, and the relations not sorted, the cost of sorting followed by merge join would be as follows:

1. Sorting the relation *takes* can be done with just one merge step and takes a total of just $400 * (2\lceil \log_{24}(400/25) \rceil + 1) = 1200$ block transfers. Similarly, sorting *student* takes 300 block transfers. Writing the sorted output to disk requires $400 + 100 = 500$ block transfers, and the merge step requires 500 block transfers to read the data back. Adding up these costs gives a total cost of 2500 block transfers.
2. If we assume that only one buffer block is allocated for each run, the number of seeks required in this case is $2 * \lceil 400/25 \rceil + 400 + 400 = 832$ seeks for sorting *takes* and writing the sorted output to disk, and similarly $2 * \lceil 100/25 \rceil + 100 + 100 = 208$ for *student*, plus $400 + 100$ seeks for reading the sorted data in the merge-join step. Adding up these costs gives a total cost of 1640 seeks.

   The number of seeks can be significantly reduced by setting aside more buffer blocks for each run. For example, if 5 buffer blocks are allocated for each run and for the output from merging the 4 runs of *student*, the cost is reduced to $2 * \lceil 100/25 \rceil + \lceil 100/5 \rceil + \lceil 100/5 \rceil = 48$ seeks, from 208 seeks. If the merge-join step sets aside 12 blocks each for buffering *takes* and *student*, the number of seeks for the merge-join step goes down to $\lceil 400/12 \rceil + \lceil 100/12 \rceil = 43$, from 500. The total number of seeks is then 251.

Thus, the total cost is 2500 block transfers plus 251 seeks if the relations are not sorted, and the memory size is 25 blocks.

15.5.4.3 Hybrid Merge Join

It is possible to perform a variation of the merge-join operation on unsorted tuples, if secondary indices exist on both join attributes. The algorithm scans the records through the indices, resulting in their being retrieved in sorted order. This variation presents a significant drawback, however, since records may be scattered throughout the file blocks. Hence, each tuple access could involve accessing a disk block, and that is costly.

To avoid this cost, we can use a hybrid merge-join technique that combines indices with merge join. Suppose that one of the relations is sorted; the other is unsorted, but has a secondary B^+-tree index on the join attributes. The **hybrid merge-join algorithm** merges the sorted relation with the leaf entries of the secondary B^+-tree index. The result file contains tuples from the sorted relation and addresses for tuples of the unsorted relation. The result file is then sorted on the addresses of tuples of the unsorted relation, allowing efficient retrieval of the corresponding tuples, in physical storage order, to complete the join. Extensions of the technique to handle two unsorted relations are left as an exercise for you.

15.5.5 Hash Join

Like the merge-join algorithm, the hash-join algorithm can be used to implement natural joins and equi-joins. In the hash-join algorithm, a hash function h is used to partition tuples of both relations. The basic idea is to partition the tuples of each of the relations into sets that have the same hash value on the join attributes.

We assume that:

- h is a hash function mapping *JoinAttrs* values to $\{0, 1, \ldots, n_h\}$, where *JoinAttrs* denotes the common attributes of r and s used in the natural join.
- $r_0, r_1, \ldots, r_{n_h}$ denote partitions of r tuples, each initially empty. Each tuple $t_r \in r$ is put in partition r_i, where $i = h(t_r[JoinAttrs])$.
- $s_0, s_1, \ldots, s_{n_h}$ denote partitions of s tuples, each initially empty. Each tuple $t_s \in s$ is put in partition s_i, where $i = h(t_s[JoinAttrs])$.

The hash function h should have the "goodness" properties of randomness and uniformity that we discussed in Chapter 14. Figure 15.9 depicts the partitioning of the relations.

15.5.5.1 Basics

The idea behind the hash-join algorithm is this: Suppose that an r tuple and an s tuple satisfy the join condition; then, they have the same value for the join attributes. If that value is hashed to some value i, the r tuple has to be in r_i and the s tuple in s_i. Therefore,

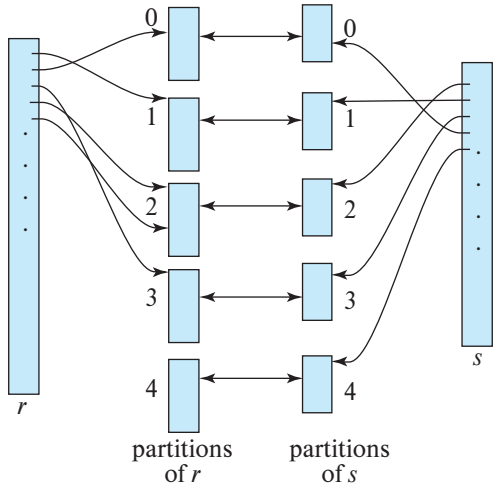


Figure 15.9 Hash partitioning of relations.

r tuples in r_i need only be compared with s tuples in s_i; they do not need to be compared with s tuples in any other partition.

For example, if d is a tuple in *student*, c a tuple in *takes*, and h a hash function on the *ID* attributes of the tuples, then d and c must be tested only if $h(c) = h(d)$. If $h(c) \neq h(d)$, then c and d must have different values for *ID*. However, if $h(c) = h(d)$, we must test c and d to see whether the values in their join attributes are the same, since it is possible that c and d have different *iid*s that have the same hash value.

Figure 15.10 shows the details of the **hash-join** algorithm to compute the natural join of relations r and s. As in the merge-join algorithm, $t_r \bowtie t_s$ denotes the concatenation of the attributes of tuples t_r and t_s, followed by projecting out repeated attributes. After the partitioning of the relations, the rest of the hash-join code performs a separate indexed nested-loop join on each of the partition pairs i, for $i = 0, \ldots, n_h$. To do so, it first **builds** a hash index on each s_i, and then **probes** (that is, looks up s_i) with tuples from r_i. The relation s is the **build input**, and r is the **probe input**.

The hash index on s_i is built in memory, so there is no need to access the disk to retrieve the tuples. The hash function used to build this hash index must be different from the hash function h used earlier, but it is still applied to only the join attributes. In the course of the indexed nested-loop join, the system uses this hash index to retrieve records that match records in the probe input.

The build and probe phases require only a single pass through both the build and probe inputs. It is straightforward to extend the hash-join algorithm to compute general equi-joins.

The value n_h must be chosen to be large enough such that, for each i, the tuples in the partition s_i of the build relation, along with the hash index on the partition, fit in memory. It is not necessary for the partitions of the probe relation to fit in memory. It is

```
/* Partition s */
for each tuple t_s in s do begin
  i := h(t_s[JoinAttrs]);
  H_{s_i} := H_{s_i} ∪ {t_s};
end
/* Partition r */
for each tuple t_r in r do begin
  i := h(t_r[JoinAttrs]);
  H_{r_i} := H_{r_i} ∪ {t_r};
end
/* Perform join on each partition */
for i := 0 to n_h do begin
  read H_{s_i} and build an in-memory hash index on it;
  for each tuple t_r in H_{r_i} do begin
    probe the hash index on H_{s_i} to locate all tuples t_s
      such that t_s[JoinAttrs] = t_r[JoinAttrs];
    for each matching tuple t_s in H_{s_i} do begin
      add t_r ⋈ t_s to the result;
    end
  end
end
```

Figure 15.10 Hash join.

best to use the smaller input relation as the build relation. If the size of the build relation is b_s blocks, then, for each of the n_h partitions to be of size less than or equal to M, n_h must be at least $\lceil b_s/M \rceil$. More precisely stated, we have to account for the extra space occupied by the hash index on the partition as well, so n_h should be correspondingly larger. For simplicity, we sometimes ignore the space requirement of the hash index in our analysis.

15.5.5.2 Recursive Partitioning

If the value of n_h is greater than or equal to the number of blocks of memory, the relations cannot be partitioned in one pass, since there will not be enough buffer blocks. Instead, partitioning has to be done in repeated passes. In one pass, the input can be split into at most as many partitions as there are blocks available for use as output buffers. Each bucket generated by one pass is separately read in and partitioned again in the next pass, to create smaller partitions. The hash function used in a pass is different from the one used in the previous pass. The system repeats this splitting of the

input until each partition of the build input fits in memory. Such partitioning is called **recursive partitioning**.

A relation does not need recursive partitioning if $M > n_h + 1$, or equivalently $M > (b_s/M)+1$, which simplifies (approximately) to $M > \sqrt{b_s}$. For example, consider a memory size of 12 megabytes, divided into 4-kilobyte blocks; it would contain a total of 3-kilobyte (3072) blocks. We can use a memory of this size to partition relations of size up to 3-kilobyte $*$ 3-kilobyte blocks, which is 36 gigabytes. Similarly, a relation of size 1 gigabyte requires just over $\sqrt{256\text{K}}$ blocks, or 2 megabytes, to avoid recursive partitioning.

15.5.5.3 Handling of Overflows

Hash-table overflow occurs in partition i of the build relation s if the hash index on s_i is larger than main memory. Hash-table overflow can occur if there are many tuples in the build relation with the same values for the join attributes, or if the hash function does not have the properties of randomness and uniformity. In either case, some of the partitions will have more tuples than the average, whereas others will have fewer; partitioning is then said to be **skewed**.

We can handle a small amount of skew by increasing the number of partitions so that the expected size of each partition (including the hash index on the partition) is somewhat less than the size of memory. The number of partitions is therefore increased by a small value, called the **fudge factor**, that is usually about 20 percent of the number of hash partitions computed as described in Section 15.5.5.

Even if, by using a fudge factor, we are conservative on the sizes of the partitions, overflows can still occur. Hash-table overflows can be handled by either *overflow resolution* or *overflow avoidance*. **Overflow resolution** is performed during the build phase if a hash-index overflow is detected. Overflow resolution proceeds in this way: If s_i, for any i, is found to be too large, it is further partitioned into smaller partitions by using a different hash function. Similarly, r_i is also partitioned using the new hash function, and only tuples in the matching partitions need to be joined.

In contrast, **overflow avoidance** performs the partitioning carefully, so that overflows never occur during the build phase. In overflow avoidance, the build relation s is initially partitioned into many small partitions, and then some partitions are combined in such a way that each combined partition fits in memory. The probe relation r is partitioned in the same way as the combined partitions on s, but the sizes of r_i do not matter.

If a large number of tuples in s have the same value for the join attributes, the resolution and avoidance techniques may fail on some partitions. In that case, instead of creating an in-memory hash index and using a nested-loop join to join the partitions, we can use other join techniques, such as block nested-loop join, on those partitions.

15.5.5.4 Cost of Hash Join

We now consider the cost of a hash join. Our analysis assumes that there is no hash-table overflow. First, consider the case where recursive partitioning is not required.

- The partitioning of the two relations r and s calls for a complete reading of both relations and a subsequent writing back of them. This operation requires $2(b_r + b_s)$ block transfers, where b_r and b_s denote the number of blocks containing records of relations r and s, respectively. The build and probe phases read each of the partitions once, calling for further $b_r + b_s$ block transfers. The number of blocks occupied by partitions could be slightly more than $b_r + b_s$, as a result of partially filled blocks. Accessing such partially filled blocks can add an overhead of at most $2n_h$ for each of the relations, since each of the n_h partitions could have a partially filled block that has to be written and read back. Thus, a hash join is estimated to require:

  $$3(b_r + b_s) + 4n_h$$

  block transfers. The overhead $4n_h$ is usually quite small compared to $b_r + b_s$ and can be ignored.

- Assuming b_b blocks are allocated for the input buffer and each output buffer, partitioning requires a total of $2(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil)$ seeks. The build and probe phases require only one seek for each of the n_h partitions of each relation, since each partition can be read sequentially. The hash join thus requires $2(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil) + 2n_h$ seeks.

Now consider the case where recursive partitioning is required. Again we assume that b_b blocks are allocated for buffering each partition. Each pass then reduces the size of each of the partitions by an expected factor of $\lfloor M/b_b \rfloor - 1$; and passes are repeated until each partition is of size at most M blocks. The expected number of passes required for partitioning s is therefore $\lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil$.

- Since, in each pass, every block of s is read in and written out, the total number of block transfers for partitioning of s is $2b_s \lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil$. The number of passes for partitioning of r is the same as the number of passes for partitioning of s, therefore the join is estimated to require

  $$2(b_r + b_s) \lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil + b_r + b_s$$

  block transfers.

- Ignoring the relatively small number of seeks during the build and probe phases, hash join with recursive partitioning requires

  $$2(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil) \lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil$$

  disk seeks.

Consider, for example, the natural join *takes* $\bowtie$ *student*. With a memory size of 20 blocks, the *student* relation can be partitioned into five partitions, each of size 20 blocks, which size will fit into memory. Only one pass is required for the partitioning. The

relation *takes* is similarly partitioned into five partitions, each of size 80. Ignoring the cost of writing partially filled blocks, the cost is $3(100 + 400) = 1500$ block transfers. There is enough memory to allocate the buffers for the input and each of the five outputs during partitioning (i.e, $b_b = 3$) leading to $2(\lceil 100/3 \rceil + \lceil 400/3 \rceil) = 336$ seeks.

The hash join can be improved if the main memory size is large. When the entire build input can be kept in main memory, n_h can be set to 0; then, the hash-join algorithm executes quickly, without partitioning the relations into temporary files, regardless of the probe input's size. The cost estimate goes down to $b_r + b_s$ block transfers and two seeks.

Indexed nested loops join can have a much lower cost than hash join in case the outer relation is small, and the index lookups fetch only a few tuples from the inner (indexed) relation. However, in case a secondary index is used, and the number of tuples in the outer relation is large, indexed nested loops join can have a very high cost, as compared to hash join. If the number of tuples in the outer relation is known at query optimization time, the best join algorithm can be chosen at that time. However, in some cases, for example, when there is a selection condition on the outer input, the optimizer makes a decision based on an estimate that may potentially be imprecise. The number of tuples in the outer relation may be found only at runtime, for example, after executing selection. Some systems allow a dynamic choice between the two algorithms at run time, after finding the number of tuples in the outer input.

15.5.5.5 Hybrid Hash Join

The **hybrid hash-join** algorithm performs another optimization; it is useful when memory sizes are relatively large but not all of the build relation fits in memory. The partitioning phase of the hash-join algorithm needs a minimum of one block of memory as a buffer for each partition that is created, and one block of memory as an input buffer. To reduce the impact of seeks, a larger number of blocks would be used as a buffer; let b_b denote the number of blocks used as a buffer for the input and for each partition. Hence, a total of $(n_h + 1) * b_b$ blocks of memory are needed for partitioning the two relations. If memory is larger than $(n_h + 1) * b_b$, we can use the rest of memory ($M - (n_h + 1) * b_b$ blocks) to buffer the first partition of the build input (i.e, s_0) so that it will not need to be written out and read back in. Further, the hash function is designed in such a way that the hash index on s_0 fits in $M - (n_h + 1) * b_b$ blocks, in order that, at the end of partitioning of s, s_0 is completely in memory and a hash index can be built on s_0.

When the system partitions r, it again does not write tuples in r_0 to disk; instead, as it generates them, the system uses them to probe the memory-resident hash index on s_0, and to generate output tuples of the join. After they are used for probing, the tuples can be discarded, so the partition r_0 does not occupy any memory space. Thus, a write and a read access have been saved for each block of both r_0 and s_0. The system writes out tuples in the other partitions as usual and joins them later. The savings of hybrid hash join can be significant if the build input is only slightly bigger than memory.

If the size of the build relation is b_s, n_h is approximately equal to b_s/M. Thus, hybrid hash join is most useful if $M >> (b_s/M) * b_b$, or $M >> \sqrt{b_s * b_b}$, where the notation $>>$ denotes *much larger than*. For example, suppose the block size is 4 kilobytes, the build relation size is 5 gigabytes, and b_b is 20. Then, the hybrid hash-join algorithm is useful if the size of memory is significantly more than 20 megabytes; memory sizes of gigabytes or more are common on computers today. If we devote 1 gigabyte for the join algorithm, s_0 would be nearly 1 gigabyte, and hybrid hash join would be nearly 20 percent cheaper than hash join.

15.5.6 Complex Joins

Nested-loop and block nested-loop joins can be used regardless of the join conditions. The other join techniques are more efficient than the nested-loop join and its variants, but they can handle only simple join conditions, such as natural joins or equi-joins. We can implement joins with complex join conditions, such as conjunctions and disjunctions, by using the efficient join techniques, if we apply the techniques developed in Section 15.3.3 for handling complex selections.

Consider the following join with a conjunctive condition:

$$r \bowtie_{\theta_1 \wedge \theta_2 \wedge \cdots \wedge \theta_n} s$$

One or more of the join techniques described earlier may be applicable for joins on the individual conditions $r \bowtie_{\theta_1} s$, $r \bowtie_{\theta_2} s$, $r \bowtie_{\theta_3} s$, and so on. We can compute the overall join by first computing the result of one of these simpler joins $r \bowtie_{\theta_i} s$; each pair of tuples in the intermediate result consists of one tuple from r and one from s. The result of the complete join consists of those tuples in the intermediate result that satisfy the remaining conditions:

$$\theta_1 \wedge \cdots \wedge \theta_{i-1} \wedge \theta_{i+1} \wedge \cdots \wedge \theta_n$$

These conditions can be tested as tuples in $r \bowtie_{\theta_i} s$ are being generated.

A join whose condition is disjunctive can be computed in this way. Consider:

$$r \bowtie_{\theta_1 \vee \theta_2 \vee \cdots \vee \theta_n} s$$

The join can be computed as the union of the records in individual joins $r \bowtie_{\theta_i} s$:

$$(r \bowtie_{\theta_1} s) \cup (r \bowtie_{\theta_2} s) \cup \cdots \cup (r \bowtie_{\theta_n} s)$$

Section 15.6 describes algorithms for computing the union of relations.

15.5.7 Joins over Spatial Data

The join algorithms we have presented make no specific assumptions about the type of data being joined, but they do assume the use of standard comparison operations such as equality, less than, or greater than, where the values are linearly ordered.

Selection and join conditions on spatial data involve comparison operators that check if one region contains or overlaps another, or whether a region contains a particular point; and the regions may be multi-dimensional. Comparisons may pertain also to the distance between points, for example, finding a set of points closest to a given point in a two-dimensional space.

Merge-join cannot be used with such comparison operations, since there is no simple sort order over spatial data in two or more dimensions. Partitioning of data based on hashing is also not applicable, since there is no way to ensure that tuples that satisfy an overlap or containment predicate are hashed to the same value. Nested loops join can always be used regardless of the complexity of the conditions, but can be very inefficient on large datasets.

Indexed nested-loops join can however be used, if appropriate spatial indices are available. In Section 14.10, we saw several types of indices for spatial data, including R-trees, k-d trees, k-d-B trees, and quadtrees. Additional details on those indices appear in Section 24.4. These index structures enable efficient retrieval of spatial data based on predicates such as contains, contained in, or overlaps, and can also be effectively used to find nearest neighbors.

Most major database systems today incorporate support for indexing spatial data, and make use of them when processing queries using spatial comparison conditions.

15.6 Other Operations

Other relational operations and extended relational operations—such as duplicate elimination, projection, set operations, outer join, and aggregation—can be implemented as outlined in Section 15.6.1 through Section 15.6.5.

15.6.1 Duplicate Elimination

We can implement duplicate elimination easily by sorting. Identical tuples will appear adjacent to each other as a result of sorting, and all but one copy can be removed. With external sort–merge, duplicates found while a run is being created can be removed before the run is written to disk, thereby reducing the number of block transfers. The remaining duplicates can be eliminated during merging, and the final sorted run has no duplicates. The worst-case cost estimate for duplicate elimination is the same as the worst-case cost estimate for sorting of the relation.

We can also implement duplicate elimination by hashing, as in the hash-join algorithm. First, the relation is partitioned on the basis of a hash function on the whole tuple. Then, each partition is read in, and an in-memory hash index is constructed.

While constructing the hash index, a tuple is inserted only if it is not already present. Otherwise, the tuple is discarded. After all tuples in the partition have been processed, the tuples in the hash index are written to the result. The cost estimate is the same as that for the cost of processing (partitioning and reading each partition) of the build relation in a hash join.

Because of the relatively high cost of duplicate elimination, SQL requires an explicit request by the user to remove duplicates; otherwise, the duplicates are retained.

15.6.2 Projection

We can implement projection easily by performing projection on each tuple, which gives a relation that could have duplicate records, and then removing duplicate records. Duplicates can be eliminated by the methods described in Section 15.6.1. If the attributes in the projection list include a key of the relation, no duplicates will exist; hence, duplicate elimination is not required. Generalized projection can be implemented in the same way as projection.

15.6.3 Set Operations

We can implement the *union*, *intersection*, and *set-difference* operations by first sorting both relations, and then scanning once through each of the sorted relations to produce the result. In $r \cup s$, when a concurrent scan of both relations reveals the same tuple in both files, only one of the tuples is retained. The result of $r \cap s$ will contain only those tuples that appear in both relations. We implement *set difference*, $r - s$, similarly, by retaining tuples in r only if they are absent in s.

For all these operations, only one scan of the two sorted input relations is required, so the cost is $b_r + b_s$ block transfers if the relations are sorted in the same order. Assuming a worst case of one block buffer for each relation, a total of $b_r + b_s$ disk seeks would be required in addition to $b_r + b_s$ block transfers. The number of seeks can be reduced by allocating extra buffer blocks.

If the relations are not sorted initially, the cost of sorting has to be included. Any sort order can be used in the evaluation of set operations, provided that both inputs have that same sort order.

Hashing provides another way to implement these set operations. The first step in each case is to partition the two relations by the same hash function and thereby create the partitions $r_0, r_1, \ldots, r_{n_h}$ and $s_0, s_1, \ldots, s_{n_h}$. Depending on the operation, the system then takes these steps on each partition $i = 0, 1, \ldots, n_h$:

- $r \cup s$
  1. Build an in-memory hash index on r_i.
  2. Add the tuples in s_i to the hash index only if they are not already present.
  3. Add the tuples in the hash index to the result.

Note 15.1 Answering Keyword Queries

Keyword search on documents is widely used in the context of web search. In its simplest form, a keyword query provides a set of words $K_1, K_2, \ldots, K_n$, and the goal is to find documents d_i from a collection of documents D such that d_i contains all the keywords in the query. Real-life keyword search is more complicated, since it requires ranking of documents based on various metrics such TF-IDF and PageRank, as we saw earlier in Section 8.3.

Documents that contain a specified keyword can be located efficiently by using an index (often referred to as an **inverted index**) that maps each keyword K_i to a list S_i of identifiers of the documents that contain K_i. The list is kept sorted. For example, if documents d_1, d_9 and d_{21} contain the term "Silberschatz", the inverted list for the keyword Silberschatz would be "$d_1; d_9; d_{21}$". Compression techniques are used to reduce the size of the inverted lists. A B^+-tree index can be used to map each keyword K_i to its associated inverted list S_i.

To answer a query with keyword $K_1, K_2, \ldots, K_n$, we retrieve the inverted list S_i for each keyword K_i, and then compute the intersection $S_1 \cap S_2 \cap \cdots \cap S_n$ to find documents that appear in all the lists. Since the lists are sorted, the intersection can be efficiently implemented by merging the lists using concurrent scans of all the lists. Many information-retrieval systems return documents that contain several, even if not all, of the keywords; the merge step can be easily modified to output documents that contain at least k of the n keywords.

To support ranking of keyword-query results, extra information can be stored in each inverted list, including the inverse document frequency of the term, and for each document the PageRank, the term frequency of the term, as well as the positions within the document where the term occurs. This information can be used to compute scores that are then used to rank the documents. For example, documents where the keywords occur close to each other may receive a higher score for keyword proximity than those where they occur farther from each other. The keyword proximity score may be combined with the TF-IDF score, and PageRank to compute an overall score. Documents are then ranked on this score. Since most web searches retrieve only the top few answers, search engines incorporate a number of optimizations that help to find the top few answers efficiently, without computing the full list and then finding the ranking. References providing further details may be found in the Further Reading section at the end of the chapter.

- $r \cap s$

  1. Build an in-memory hash index on r_i.
  2. For each tuple in s_i, probe the hash index and output the tuple to the result only if it is already present in the hash index.

- $r - s$

  1. Build an in-memory hash index on r_i.
  2. For each tuple in s_i, probe the hash index, and, if the tuple is present in the hash index, delete it from the hash index.
  3. Add the tuples remaining in the hash index to the result.

15.6.4 Outer Join

Recall the *outer-join operations* described in Section 4.1.3. For example, the natural left outer join *takes* $\bowtie$ *student* contains the join of *takes* and *student*, and, in addition, for each *takes* tuple t that has no matching tuple in *student* (i.e, where *ID* is not in *student*), the following tuple t_1 is added to the result. For all attributes in the schema of *takes*, tuple t_1 has the same values as tuple t. The remaining attributes (from the schema of *student*) of tuple t_1 contain the value null.

We can implement the outer-join operations by using one of two strategies:

1. Compute the corresponding join, and then add further tuples to the join result to get the outer-join result. Consider the left outer-join operation and two relations: $r(R)$ and $s(S)$. To evaluate $r \bowtie_\theta s$, we first compute $r \bowtie_\theta s$ and save that result as temporary relation q_1. Next, we compute $r - \Pi_R(q_1)$ to obtain those tuples in r that do not participate in the theta join. We can use any of the algorithms for computing the joins, projection, and set difference described earlier to compute the outer joins. We pad each of these tuples with null values for attributes from s, and add it to q_1 to get the result of the outer join.

   The right outer-join operation $r \bowtie\!\!\!\subset_\theta s$ is equivalent to $s \bowtie_\theta r$ and can therefore be implemented in a symmetric fashion to the left outer join. We can implement the full outer-join operation $r \bowtie\!\!\!\subset\!\!\!\supset_\theta s$ by computing the join $r \bowtie s$ and then adding the extra tuples of both the left and right outer-join operations, as before.

2. Modify the join algorithms. It is easy to extend the nested-loop join algorithms to compute the left outer join: Tuples in the outer relation that do not match any tuple in the inner relation are written to the output after being padded with null values. However, it is hard to extend the nested-loop join to compute the full outer join.

   Natural outer joins and outer joins with an equi-join condition can be computed by extensions of the merge-join and hash-join algorithms. Merge join can be extended to compute the full outer join as follows: When the merge of the two relations is being done, tuples in either relation that do not match any tuple in the other relation can be padded with nulls and written to the output. Similarly, we can extend merge join to compute the left and right outer joins by writing out nonmatching tuples (padded with nulls) from only one of the relations. Since the relations are sorted, it is easy to detect whether or not a tuple matches any tuples

from the other relation. For example, when a merge join of *takes* and *student* is done, the tuples are read in sorted order of *ID*, and it is easy to check, for each tuple, whether there is a matching tuple in the other.

The cost estimates for implementing outer joins using the merge-join algorithm are the same as are those for the corresponding join. The only difference lies in the size of the result, and therefore in the block transfers for writing it out, which we did not count in our earlier cost estimates.

The extension of the hash-join algorithm to compute outer joins is left for you to do as an exercise (Exercise 15.21).

15.6.5 Aggregation

Recall the aggregation function (operator), discussed in Section 3.7. For example, the function

select *dept_name*, **avg** (*salary*)
from *instructor*
group by *dept_name*;

computes the average salary in each university department.

The aggregation operation can be implemented in the same way as duplicate elimination. We use either sorting or hashing, just as we did for duplicate elimination, but based on the grouping attributes (*dept_name* in the preceding example). However, instead of eliminating tuples with the same value for the grouping attribute, we gather them into groups and apply the aggregation operations on each group to get the result.

The cost estimate for implementing the aggregation operation is the same as the cost of duplicate elimination for aggregate functions such as **min**, **max**, **sum**, **count**, and **avg**.

Instead of gathering all the tuples in a group and then applying the aggregation operations, we can implement the aggregation operations **sum**, **min**, **max**, **count**, and **avg** on the fly as the groups are being constructed. For the case of **sum**, **min**, and **max**, when two tuples in the same group are found, the system replaces them with a single tuple containing the **sum**, **min**, or **max**, respectively, of the columns being aggregated. For the **count** operation, it maintains a running count for each group for which a tuple has been found. Finally, we implement the **avg** operation by computing the sum and the count values on the fly, and finally dividing the sum by the count to get the average.

If all tuples of the result fit in memory, the sort-based and the hash-based implementations do not need to write any tuples to disk. As the tuples are read in, they can be inserted in a sorted tree structure or in a hash index. When we use on-the-fly aggregation techniques, only one tuple needs to be stored for each of the groups. Hence, the sorted tree structure or hash index fits in memory, and the aggregation can be processed with just b_r block transfers (and 1 seek) instead of the $3b_r$ transfers (and a worst case of up to $2b_r$ seeks) that would be required otherwise.

15.7 Evaluation of Expressions

So far, we have studied how individual relational operations are carried out. Now we consider how to evaluate an expression containing multiple operations. The obvious way to evaluate an expression is simply to evaluate one operation at a time, in an appropriate order. The result of each evaluation is **materialized** in a temporary relation for subsequent use. A disadvantage to this approach is the need to construct the temporary relations, which (unless they are small) must be written to disk. An alternative approach is to evaluate several operations simultaneously in a **pipeline**, with the results of one operation passed on to the next, without the need to store a temporary relation.

In Section 15.7.1 and Section 15.7.2, we consider both the *materialization* approach and the *pipelining* approach. We shall see that the costs of these approaches can differ substantially, but also that there are cases where only the materialization approach is feasible.

15.7.1 Materialization

It is easiest to understand intuitively how to evaluate an expression by looking at a pictorial representation of the expression in an **operator tree**. Consider the expression:

$$\Pi_{name}(\sigma_{building = \text{“Watson”}}(department) \bowtie instructor)$$

in Figure 15.11.

If we apply the materialization approach, we start from the lowest-level operations in the expression (at the bottom of the tree). In our example, there is only one such operation: the selection operation on *department*. The inputs to the lowest-level operations are relations in the database. We execute these operations using the algorithms that we studied earlier, and we store the results in temporary relations. We can use these temporary relations to execute the operations at the next level up in the tree, where the inputs now are either temporary relations or relations stored in the database. In our

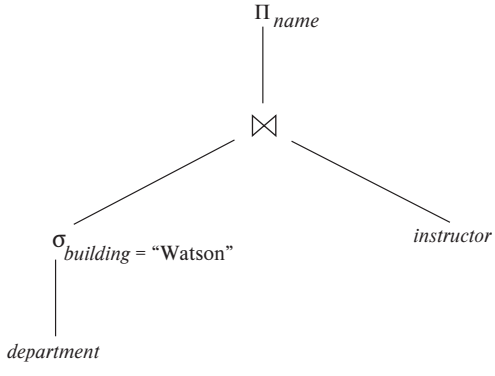


Figure 15.11 Pictorial representation of an expression.

example, the inputs to the join are the *instructor* relation and the temporary relation created by the selection on *department*. The join can now be evaluated, creating another temporary relation.

By repeating the process, we will eventually evaluate the operation at the root of the tree, giving the final result of the expression. In our example, we get the final result by executing the projection operation at the root of the tree, using as input the temporary relation created by the join.

Evaluation as just described is called **materialized evaluation**, since the results of each intermediate operation are created (materialized) and then are used for evaluation of the next-level operations.

The cost of a materialized evaluation is not simply the sum of the costs of the operations involved. When we computed the cost estimates of algorithms, we ignored the cost of writing the result of the operation to disk. To compute the cost of evaluating an expression as done here, we have to add the costs of all the operations, as well as the cost of writing the intermediate results to disk. We assume that the records of the result accumulate in a buffer, and, when the buffer is full, they are written to disk. The number of blocks written out, b_r, can be estimated as n_r/f_r, where n_r is the estimated number of tuples in the result relation r and f_r is the *blocking factor* of the result relation, that is, the number of records of r that will fit in a block. In addition to the transfer time, some disk seeks may be required, since the disk head may have moved between successive writes. The number of seeks can be estimated as $\lceil b_r/b_b \rceil$ where b_b is the size of the output buffer (measured in blocks).

Double buffering (using two buffers, with one continuing execution of the algorithm while the other is being written out) allows the algorithm to execute more quickly by performing CPU activity in parallel with I/O activity. The number of seeks can be reduced by allocating extra blocks to the output buffer and writing out multiple blocks at once.

15.7.2 Pipelining

We can improve query-evaluation efficiency by reducing the number of temporary files that are produced. We achieve this reduction by combining several relational operations into a *pipeline* of operations, in which the results of one operation are passed along to the next operation in the pipeline. Evaluation as just described is called **pipelined evaluation**.

For example, consider the expression $(\Pi_{a1,a2}(r \bowtie s))$. If materialization were applied, evaluation would involve creating a temporary relation to hold the result of the join and then reading back in the result to perform the projection. These operations can be combined: When the join operation generates a tuple of its result, it passes that tuple immediately to the project operation for processing. By combining the join and the projection, we avoid creating the intermediate result and instead create the final result directly.

Creating a pipeline of operations can provide two benefits:

1. It eliminates the cost of reading and writing temporary relations, reducing the cost of query evaluation. Note that the cost formulae that we saw earlier for each operation included the cost of reading the result from disk. If the input to an operator o_i is pipelined from a preceding operator o_j, the cost of o_i should not include the cost of reading the input from disk; the cost formulae that we saw earlier can be modified accordingly.
2. It can start generating query results quickly, if the root operator of a query-evaluation plan is combined in a pipeline with its inputs. This can be quite useful if the results are displayed to a user as they are generated, since otherwise there may be a long delay before the user sees any query results.

15.7.2.1 Implementation of Pipelining

We can implement a pipeline by constructing a single, complex operation that combines the operations that constitute the pipeline. Although this approach may be feasible for some frequently occurring situations, it is desirable in general to reuse the code for individual operations in the construction of a pipeline.

In the example of Figure 15.11, all three operations can be placed in a pipeline, which passes the results of the selection to the join as they are generated. In turn, it passes the results of the join to the projection as they are generated. The memory requirements are low, since results of an operation are not stored for long. However, as a result of pipelining, the inputs to the operations are not available all at once for processing.

Pipelines can be executed in either of two ways:

1. In a **demand-driven pipeline**, the system makes repeated requests for tuples from the operation at the top of the pipeline. Each time that an operation receives a request for tuples, it computes the next tuple (or tuples) to be returned and then returns that tuple. If the inputs of the operation are not pipelined, the next tuple(s) to be returned can be computed from the input relations, while the system keeps track of what has been returned so far. If it has some pipelined inputs, the operation also makes requests for tuples from its pipelined inputs. Using the tuples received from its pipelined inputs, the operation computes tuples for its output and passes them up to its parent.
2. In a **producer-driven pipeline**, operations do not wait for requests to produce tuples, but instead generate the tuples **eagerly**. Each operation in a producer-driven pipeline is modeled as a separate process or thread within the system that takes a stream of tuples from its pipelined inputs and generates a stream of tuples for its output.

We describe next how demand-driven and producer-driven pipelines can be implemented.

Each operation in a demand-driven pipeline can be implemented as an **iterator** that provides the following functions: *open*(), *next*(), and *close*(). After a call to *open*(), each call to *next*() returns the next output tuple of the operation. The implementation of the operation in turn calls *open*() and *next*() on its inputs, to get its input tuples when required. The function *close*() tells an iterator that no more tuples are required. The iterator maintains the **state** of its execution in between calls so that successive *next*() requests receive successive result tuples.

For example, for an iterator implementing the select operation using linear search, the *open*() operation starts a file scan, and the iterator's state records the point to which the file has been scanned. When the *next*() function is called, the file scan continues from after the previous point; when the next tuple satisfying the selection is found by scanning the file, the tuple is returned after storing the point where it was found in the iterator state. A merge-join iterator's *open*() operation would open its inputs, and if they are not already sorted, it would also sort the inputs. On calls to *next*(), it would return the next pair of matching tuples. The state information would consist of up to where each input had been scanned. Details of the implementation of iterators are left for you to complete in Practice Exercise 15.7.

Producer-driven pipelines, on the other hand, are implemented in a different manner. For each pair of adjacent operations in a producer-driven pipeline, the system creates a buffer to hold tuples being passed from one operation to the next. The processes or threads corresponding to different operations execute concurrently. Each operation at the bottom of a pipeline continually generates output tuples, and puts them in its output buffer, until the buffer is full. An operation at any other level of a pipeline generates output tuples when it gets input tuples from lower down in the pipeline until its output buffer is full. Once the operation uses a tuple from a pipelined input, it removes the tuple from its input buffer. In either case, once the output buffer is full, the operation waits until its parent operation removes tuples from the buffer so that the buffer has space for more tuples. At this point, the operation generates more tuples until the buffer is full again. The operation repeats this process until all the output tuples have been generated.

It is necessary for the system to switch between operations only when an output buffer is full or when an input buffer is empty and more input tuples are needed to generate any more output tuples. In a parallel-processing system, operations in a pipeline may be run concurrently on distinct processors (see Section 22.5.1).

Using producer-driven pipelining can be thought of as **pushing** data up an operation tree from below, whereas using demand-driven pipelining can be thought of as **pulling** data up an operation tree from the top. Whereas tuples are generated *eagerly* in producer-driven pipelining, they are generated **lazily**, on demand, in demand-driven pipelining. Demand-driven pipelining is used more commonly than producer-driven pipelining because it is easier to implement. However, producer-driven pipelining is very useful in parallel processing systems. Producer-driven pipelining has also been

found to be more efficient than demand-driven pipelining on modern CPUs since it reduces the number of function call invocations as compared to demand-driven pipelining. Producer-driven pipelining is increasingly used in systems that generate machine code for high performance query evaluation.

15.7.2.2 Evaluation Algorithms for Pipelining

Query plans can be annotated to mark edges that are pipelined; such edges are called **pipelined edges**. In contrast, non-pipelined edges are referred to as **blocking edges** or **materialized edges**. The two operators connected by a pipelined edge must be executed concurrently, since one consumes tuples as the other generates them. Since a plan can have multiple pipelined edges, the set of all operators that are connected by pipelined edges must be executed concurrently. A query plan can be divided into subtrees such that each subtree has only pipelined edges, and the edges between the subtrees are non-pipelined. Each such subtree is called a **pipeline stage**. The query processor executes the plan one pipeline stage at a time, and concurrently executes all the operators in a single pipeline stage.

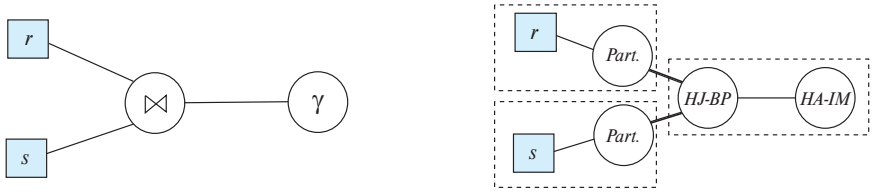
Some operations, such as sorting, are inherently **blocking operations**, that is, they may not be able to output any results until all tuples from their inputs have been examined.[7] But interestingly, blocking operators can consume tuples as they are generated, and can output tuples to their consumers as they are generated; such operations actually execute in two or more stages, and blocking actually happens between two stages of the operation.

For example, the external sort-merge operation actually has two steps: (i) run-generation, followed by (ii) merging. The run-generation step can accept tuples as they are generated by the input to the sort, and can thus be pipelined with the sort input. The merge step, on the other hand, can send tuples to its consumer as they are generated, and can thus be pipelined with the consumer of the sort operation. But the merge step can start only after the run-generation step has finished. We can thus model the sort-merge operator as two sub-operators connected to each other by a non-pipelined edge, but each of the sub-operators can be connected by pipelined edges to their input and output respectively.

Other operations, such as join, are not inherently blocking, but specific evaluation algorithms may be blocking. For example, the indexed nested loops join algorithm can output result tuples as it gets tuples for the outer relation. It is therefore pipelined on its outer (left-hand side) relation; however, it is blocking on its indexed (right-hand side) input, since the index must be fully constructed before the indexed nested-loop join algorithm can execute.

The hash-join algorithm is a blocking operation on both inputs, since it requires both its inputs to be fully retrieved and partitioned before it outputs any tuples. How-

[7] Blocking operations such as sorting may be able to output tuples early if the input is known to satisfy some special properties such as being sorted, or partially sorted, already. However, in the absence of such information, blocking operations cannot output tuples early.



(a) Logical Query (b) Pipelined Plan

Figure 15.12 Query plan with pipelining.

ever, hash-join partitions each of its inputs, and then performs multiple build-probe steps, once per partition. Thus, the hash-join algorithm has 3 steps: (i) partitioning of the first input, (ii) partitioning of the second input, and (iii) the build-probe step. The partitioning step for each input can accept tuples as they are generated by the input, and can thus be pipelined with its input. The build-probe step can output tuples to its consumer as the tuples are generated, and can thus be pipelined with its consumer. But the two partitioning steps are connected to the build-probe step by non-pipelined edges, since build-probe can start only after partitioning has been completed on both inputs.

Hybrid hash join can be viewed as partially pipelined on the probe relation, since it can output tuples from the first partition as tuples are received for the probe relation. However, tuples that are not in the first partition will be output only after the entire pipelined input relation is received. Hybrid hash join thus provides fully pipelined evaluation on its probe input if the build input fits entirely in memory, or nearly pipelined evaluation if most of the build input fits in memory.

Figure 15.12a shows a query that joins two relations r and s, and then performs an aggregation on the result; details of the join predicate, group by attributes and aggregation functions are omitted for simplicity. Figure 15.12b shows a pipelined plan for the query using hash join and in-memory hash aggregation. Pipelined edges are shown using a normal line, while blocking edges are shown using a bold line. Pipeline stages are enclosed in dashed boxes. Note that hash join has been split into three suboperators. Two of suboperators, shown abbreviated to *Part.*, partition r and s respectively. The third, abbreviated to *HJ-BP*, performs the build and probe phase of the hash join. The *HA-IM* operator is the in-memory hash aggregation operator. The edges from the partition operators to the *HJ-BP* operator are blocking edges, since the *HJ-BP* operator can start execution only after the partition operators have completed execution. The edges from the relations (assumed to be scanned using a relation scan operator) to the partition operators are pipelined, as is the edge from the *HJ-BP* operator to the *HA-IM* operator. The resultant pipeline stages are shown enclosed in dashed boxes.

In general, for each materialized edge we need to add the cost of writing the data to disk, and the cost of the consumer operator should include the cost of reading the data from disk. However, when a materialized edge is between suboperators of a single

```
done_r := false;
done_s := false;
r := Ø;
s := Ø;
result := Ø;
while not done_r or not done_s do
   begin
      if queue is empty, then wait until queue is not empty;
      t := top entry in queue;
      if t = End_r then done_r := true
        else if t = End_s then done_s := true
           else if t is from input r
              then
                 begin
                    r := r ∪ {t};
                    result := result ∪ ({t} ⋈ s);
                 end
              else /* t is from input s */
                 begin
                    s := s ∪ {t};
                    result := result ∪ (r ⋈ {t});
                 end
   end
```

Figure 15.13 Double-pipelined join algorithm.

operator, for example between run generation and merge, the materialization cost has already been accounted for in the operators cost, and should not be added again.

In some applications, a join algorithm that is pipelined on both its inputs and its output is desirable. If both inputs are sorted on the join attribute, and the join condition is an equi-join, merge join can be used, with both its inputs and its output pipelined.

However, in the more common case that the two inputs that we desire to pipeline into the join are not already sorted, another alternative is the **double-pipelined join** technique, shown in Figure 15.13. The algorithm assumes that the input tuples for both input relations, r and s, are pipelined. Tuples made available for both relations are queued for processing in a single queue. Special queue entries, called End_r and End_s, which serve as end-of-file markers, are inserted in the queue after all tuples from r and s (respectively) have been generated. For efficient evaluation, appropriate indices should be built on the relations r and s. As tuples are added to r and s, the indices must be kept

up to date. When hash indices are used on r and s, the resultant algorithm is called the **double-pipelined hash-join** technique.

The double-pipelined join algorithm in Figure 15.13 assumes that both inputs fit in memory. In case the two inputs are larger than memory, it is still possible to use the double-pipelined join technique as usual until available memory is full. When available memory becomes full, r and s tuples that have arrived up to that point can be treated as being in partition r_0 and s_0, respectively. Tuples for r and s that arrive subsequently are assigned to partitions r_1 and s_1, respectively, which are written to disk, and are not added to the in-memory index. However, tuples assigned to r_1 and s_1 are used to probe s_0 and r_0, respectively, before they are written to disk. Thus, the join of r_1 with s_0, and s_1 with r_0, is also carried out in a pipelined fashion. After r and s have been fully processed, the join of r_1 tuples with s_1 tuples must be carried out to complete the join; any of the join techniques we have seen earlier can be used to join r_1 with s_1.

15.7.3 Pipelines for Continuous-Stream Data

Pipelining is also applicable in situations where data are entered into the database in a continuous manner, as is the case, for example, for inputs from sensors that are continuously monitoring environmental data. Such data are called *data streams*, as we saw earlier in Section 10.5. Queries may be written over stream data in order to respond to data as they arrive. Such queries are called *continuous queries*.

The operations in a continuous query should be implemented using pipelined algorithms, so that results from the pipeline can be output without blocking. Producer-driven pipelines (which we discussed earlier in Section 15.7.2.1) are the best suited for continuous query evaluation.

Many such queries perform aggregation with windowing; tumbling windows which divide time into fixed size intervals, such as 1 minute, or 1 hour, are commonly used. Grouping and aggregation is performed separately on each window, as tuples are received; assuming memory size is large enough, an in-memory hash index is used to perform aggregation.

The result of aggregation on a window can be output once the system knows that no further tuples in that window will be received in future. If tuples are guaranteed to arrive sorted by timestamp, the arrival of a tuple of a following window indicates no more tuples will be received for an earlier window. If tuples may arrive out of order, streams must carry punctuations that indicate that all future tuples will have a timestamp greater than some specified value. The arrival of a punctuation allows the output of aggregates of windows whose end-timestamp is less than or equal to the timestamp specified by the punctuation.

15.8 Query Processing in Memory

The query processing algorithms that we have described so far focus on minimizing I/O cost. In this section, we discuss extensions to the query processing techniques that

help minimize memory access costs by using cache-conscious query processing algorithms and query compilation. We then discuss query processing with column-oriented storage. The algorithms we describe in this section give significant benefits for memory resident data; they are also very useful with disk-resident data, since they can speed up processing once data has been brought into the in-memory buffer.

15.8.1 Cache-Conscious Algorithms

When data is resident in memory, access is much faster than if data were resident on magnetic disks, or even SSDs. However, it must be kept in mind that data already in CPU cache can be accessed as much as 100 times faster than data in memory. Modern CPUs have several levels of cache. Commonly used CPUs today have an L1 cache of size around 64 kilobytes, with a latency of about 1 nanosecond, an L2 cache of size around 256 kilobytes, with a latency of around 5 nanoseconds, and an L3 cache of having a size of around 10 megabytes, with a latency of 10 to 15 nanoseconds. In contrast, reading data in memory results in a latency of around 50 to 100 nanoseconds. For simplicity in the rest of this section we ignore the difference between the L1, L2 and L3 cache levels, and assume that there is only a single cache level.

As we saw in Section 14.4.7, the speed difference between cache memory and main memory, and the fact that data are transferred between main memory and cache in units of a *cache-line* (typically about 64 bytes), results in a situation where the relationship between cache and main memory is not dissimilar to the relationship between main memory and disk (although with smaller speed differences). But there is a difference: while the contents of the main memory buffers disk-based data are controlled by the database system, CPU cache is controlled by the algorithms built into the computer hardware. Thus, the database system cannot directly control what is kept in cache.

However, query processing algorithms can be designed in a way that the makes the best use of cache, to optimize performance. Here are some ways this can be done:

- To sort a relation that is in-memory, we use the external merge-sort algorithm, with the run size chosen such that the run fits into the cache; assuming we focus on the L3 cache, each run should be a few megabytes in size. We then use an in-memory sorting algorithm on each run; since the run fits in cache, cache misses are likely to be minimal when the run is sorted. The sorted runs (all of which are in memory) are then merged. Merging is cache efficient, since access to the runs is sequential: when a particular word is accessed from memory, the cache line that is fetched will contain the words that would be accessed next from that run.

  To sort a relation larger than memory, we can use external sort-merge with much larger run sizes, but use the in-memory merge-sort technique we just described to perform the in-memory sort of the large runs.

- Hash-join requires probing of an index on the build relation. If the build relation fits in memory, an index could be built on the whole relation; however, cache hits during probe can be maximized by partitioning the relations into smaller pieces

such that each partition of the build-relation along with the index fits in the cache. Each partition is processed separately, with a build and a probe phase; since the build partition and its index fit in cache, cache misses are minimized during the build as well as the probe phase.

For relations larger than memory, the first stage of hash-join should partition the two relations such that for each partition, the partitions of the two relations together fit in memory. The technique just described can then be used to perform the hash join on each of these partitions, after fetching the contents into memory.

- Attributes in a tuple can be arranged such that attributes that tend to be accessed together are laid out consecutively. For example, if a relation is often used for aggregation, those attributes used as group by attributes, and those that are aggregated upon, can be stored consecutively. As a result, if there is a cache miss on one attribute, the cache line that is fetched would contain attributes that are likely to be used immediately.

Cache-aware algorithms are of increasing importance in modern database systems, since memory sizes are often large enough that much of the data is memory-resident.

In cases where the requisite data item is not in cache, there is a processing **stall** while the data item is retrieved from memory and loaded into cache. In order to continue to make use of the core that made the request resulting in the stall, the operating system maintains multiple threads of execution on which a core may work. Parallel query processing algorithms, which we study in Chapter 22 can use multiple threads running on a single CPU core; if one thread is stalled, another can start execution so the CPU core is utilized better.

15.8.2 Query Compilation

With data resident in memory, CPU cost becomes the bottleneck, and minimizing CPU cost can give significant benefits. Traditional databases query processors act as interpreters that execute a query plan. However, there is a significant overhead due to interpretation: for example, to access an attribute of a record, the query execution engine may repeatedly look up the relation meta-data to find the offset of the attribute within the record, since the same code must work for all relations. There is also significant overhead due to function calls that are performed for each record processed by an operation.

To avoid overhead due to interpretation, modern main-memory databases compile query plans into machine code or intermediate level byte-code. For example, the compiler can compute the offset of an attribute at compile time, and generate code where the offset is a constant. The compiler can also combine the code for multiple functions in a way that minimizes function calls. With these, and other related optimizations, compiled code has been found to execute faster, by up to a factor of 10, than interpreted code.

15.8.3 Column-Oriented Storage

In Section 13.6, we saw that in data-analytic applications, only a few attributes of a large schema may be needed, and that in such cases, storing a relation by column instead of by row may be advantageous. Selection operations on a single attribute (or small number of attributes) have significantly lower cost in a column store since only the relevant attributes need to be accessed. However, since accessing each attribute requires its own data access, the cost of retrieving many attributes is higher and may incur additional seeks if data are stored on disk.

Because column stores permit efficient access to many values for a given attribute at once, they are well suited to exploit the vector-processing capabilities of modern processors. This capability allows certain operations (such as comparisons and aggregations) to be performed in a parallel on multiple attribute values. When compiling query plans to machine code, the compiler can generate vector-processing instructions supported by the processor.

15.9 Summary

- The first action that the system must perform on a query is to translate the query into its internal form, which (for relational database systems) is usually based on the relational algebra. In the process of generating the internal form of the query, the parser checks the syntax of the user's query, verifies that the relation names appearing in the query are names of relations in the database, and so on. If the query was expressed in terms of a view, the parser replaces all references to the view name with the relational-algebra expression to compute the view.
- Given a query, there are generally a variety of methods for computing the answer. It is the responsibility of the query optimizer to transform the query as entered by the user into an equivalent query that can be computed more efficiently. Chapter 16 covers query optimization.
- We can process simple selection operations by performing a linear scan or by making use of indices. We can handle complex selections by computing unions and intersections of the results of simple selections.
- We can sort relations larger than memory by the external sort – merge algorithm.
- Queries involving a natural join may be processed in several ways, depending on the availability of indices and the form of physical storage for the relations.
    - If the join result is almost as large as the Cartesian product of the two relations, a *block nested-loop* join strategy may be advantageous.
    - If indices are available, the *indexed nested-loop* join can be used.

  - If the relations are sorted, a *merge join* may be desirable. It may be advantageous to sort a relation prior to join computation (so as to allow use of the merge-join strategy).
  - The *hash-join* algorithm partitions the relations into several pieces, such that each piece of one of the relations fits in memory. The partitioning is carried out with a hash function on the join attributes so that corresponding pairs of partitions can be joined independently.
- Duplicate elimination, projection, set operations (union, intersection, and difference), and aggregation can be done by sorting or by hashing.
- Outer-join operations can be implemented by simple extensions of join algorithms.
- Hashing and sorting are dual, in the sense that any operation such as duplicate elimination, projection, aggregation, join, and outer join that can be implemented by hashing can also be implemented by sorting, and vice versa; that is, any operation that can be implemented by sorting can also be implemented by hashing.
- An expression can be evaluated by means of materialization, where the system computes the result of each subexpression and stores it on disk and then uses it to compute the result of the parent expression.
- Pipelining helps to avoid writing the results of many subexpressions to disk by using the results in the parent expression even as they are being generated.

Review Terms

- Query processing
- Evaluation primitive
- Query-execution plan
- Query-evaluation plan
- Query-execution engine
- Measures of query cost
- Sequential I/O
- Random I/O
- File scan
- Linear search
- Selections using indices
- Access paths
- Index scans
- Conjunctive selection
- Disjunctive selection
- Composite index
- Intersection of identifiers
- External sorting
- External sort–merge
- Runs
- *N*-way merge
- Equi-join
- Nested-loop join
- Block nested-loop join
- Indexed nested-loop join
- Merge join
- Sort-merge join
- Hybrid merge join

- Hash-join
  - Build
  - Probe
  - Build input
  - Probe input
  - Recursive partitioning
  - Hash-table overflow
  - Skew
  - Fudge factor
  - Overflow resolution
  - Overflow avoidance
- Hybrid hash-join
- Spatial join
- Operator tree
- Materialized evaluation
- Double buffering
- Pipelined evaluation
  - Demand-driven pipeline (lazy, pulling)
  - Producer-driven pipeline (eager, pushing)
  - Iterator
  - Pipeline stages
- Double-pipelined join
- Continuous query evaluation

Practice Exercises

15.1 Assume (for simplicity in this exercise) that only one tuple fits in a block and memory holds at most three blocks. Show the runs created on each pass of the sort-merge algorithm when applied to sort the following tuples on the first attribute: (kangaroo, 17), (wallaby, 21), (emu, 1), (wombat, 13), (platypus, 3), (lion, 8), (warthog, 4), (zebra, 11), (meerkat, 6), (hyena, 9), (hornbill, 2), (baboon, 12).

15.2 Consider the bank database of Figure 15.14, where the primary keys are underlined, and the following SQL query:

select *T.branch_name*
from *branch T*, *branch S*
where *T.assets* > *S.assets* **and** *S.branch_city* = "Brooklyn"

Write an efficient relational-algebra expression that is equivalent to this query. Justify your choice.

15.3 Let relations $r_1(A, B, C)$ and $r_2(C, D, E)$ have the following properties: r_1 has 20,000 tuples, r_2 has 45,000 tuples, 25 tuples of r_1 fit on one block, and 30 tuples of r_2 fit on one block. Estimate the number of block transfers and seeks required using each of the following join strategies for $r_1 \bowtie r_2$:

a. Nested-loop join.

b. Block nested-loop join.

c. Merge join.

d. Hash join.

15.4 The indexed nested-loop join algorithm described in Section 15.5.3 can be inefficient if the index is a secondary index and there are multiple tuples with the same value for the join attributes. Why is it inefficient? Describe a way, using sorting, to reduce the cost of retrieving tuples of the inner relation. Under what conditions would this algorithm be more efficient than hybrid merge join?

15.5 Let r and s be relations with no indices, and assume that the relations are not sorted. Assuming infinite memory, what is the lowest-cost way (in terms of I/O operations) to compute $r \bowtie s$? What is the amount of memory required for this algorithm?

15.6 Consider the bank database of Figure 15.14, where the primary keys are underlined. Suppose that a B^+-tree index on *branch_city* is available on relation *branch*, and that no other index is available. List different ways to handle the following selections that involve negation:

a. $\sigma_{\neg(branch\_city < \text{“Brooklyn”})}(branch)$

b. $\sigma_{\neg(branch\_city = \text{“Brooklyn”})}(branch)$

c. $\sigma_{\neg(branch\_city < \text{“Brooklyn”} \vee assets < 5000)}(branch)$

15.7 Write pseudocode for an iterator that implements indexed nested-loop join, where the outer relation is pipelined. Your pseudocode must define the standard iterator functions *open*( ), *next*( ), and *close*( ). Show what state information the iterator must maintain between calls.

15.8 Design sort-based and hash-based algorithms for computing the relational division operation (see Practice Exercise 2.9 for a definition of the division operation).

---

branch(*branch_name*, *branch_city, assets*)
customer (*customer_name*, *customer_street, customer_city*)
loan (*loan_number*, *branch_name, amount*)
borrower (*customer_name*, *loan_number*)
account (*account_number*, *branch_name, balance*)
depositor (*customer_name*, *account_number*)

Figure 15.14 Bank database.

15.9 What is the effect on the cost of merging runs if the number of buffer blocks per run is increased while overall memory available for buffering runs remains fixed?

15.10 Consider the following extended relational-algebra operators. Describe how to implement each operation using sorting and using hashing.

a. **Semijoin** ($\ltimes_\theta$): The multiset semijoin operator $r \ltimes_\theta s$ is defined as follows: if a tuple r_i appears n times in r, it appears n times in the result of $r \ltimes_\theta$ if there is at least one tuple s_j such that r_i and s_j satisfy predicate θ; otherwise r_i does not appear in the result.

b. **Anti-semijoin** ($\overline{\ltimes}_\theta$): The multiset anti-semijoin operator $r \overline{\ltimes}_\theta s$ is defined as follows: if a tuple r_i appears n times in r, it appears n times in the result of $r \ltimes_\theta$ if there does not exist any tuple s_j in s such that r_i and s_j satisfy predicate θ; otherwise r_i does not appear in the result.

15.11 Suppose a query retrieves only the first K results of an operation and terminates after that. Which choice of demand-driven or producer-driven pipelining (with buffering) would be a good choice for such a query? Explain your answer.

15.12 Current generation CPUs include an *instruction cache*, which caches recently used instructions. A function call then has a significant overhead because the set of instructions being executed changes, resulting in cache misses on the instruction cache.

a. Explain why producer-driven pipelining with buffering is likely to result in a better instruction cache hit rate, as compared to demand-driven pipelining.

b. Explain why modifying demand-driven pipelining by generating multiple results on one call to *next*(), and returning them together, can improve the instruction cache hit rate.

15.13 Suppose you want to find documents that contain at least k of a given set of n keywords. Suppose also you have a keyword index that gives you a (sorted) list of identifiers of documents that contain a specified keyword. Give an efficient algorithm to find the desired set of documents.

15.14 Suggest how a document containing a word (such as "leopard") can be indexed such that it is efficiently retrieved by queries using a more general concept (such as "carnivore" or "mammal"). You can assume that the concept hierarchy is not very deep, so each concept has only a few generalizations (a concept can, however, have a large number of specializations). You can also assume that you are provided with a function that returns the concept for each word in a document. Also suggest how a query using a specialized concept can retrieve documents using a more general concept.

15.15 Explain why the nested-loops join algorithm (see Section 15.5.1) would work poorly on a database stored in a column-oriented manner. Describe an alternative algorithm that would work better, and explain why your solution is better.

15.16 Consider the following queries. For each query, indicate if column-oriented storage is likely to be beneficial or not, and explain why.

a. Fetch ID, *name* and *dept_name* of the student with ID 12345.

b. Group the *takes* relation by *year* and *course_id*, and find the total number of students for each (*year*, *course_id*) combination.

Exercises

15.17 Suppose you need to sort a relation of 40 gigabytes, with 4-kilobyte blocks, using a memory size of 40 megabytes. Suppose the cost of a seek is 5 milliseconds, while the disk transfer rate is 40 megabytes per second.

a. Find the cost of sorting the relation, in seconds, with $b_b = 1$ and with $b_b = 100$.

b. In each case, how many merge passes are required?

c. Suppose a flash storage device is used instead of a disk, and it has a latency of 20 microsecond and a transfer rate of 400 megabytes per second. Recompute the cost of sorting the relation, in seconds, with $b_b = 1$ and with $b_b = 100$, in this setting.

15.18 Why is it not desirable to force users to make an explicit choice of a query-processing strategy? Are there cases in which it *is* desirable for users to be aware of the costs of competing query-processing strategies? Explain your answer.

15.19 Design a variant of the hybrid merge-join algorithm for the case where both relations are not physically sorted, but both have a sorted secondary index on the join attributes.

15.20 Estimate the number of block transfers and seeks required by your solution to Exercise 15.19 for $r_1 \bowtie r_2$, where r_1 and r_2 are as defined in Exercise 15.3.

15.21 The hash-join algorithm as described in Section 15.5.5 computes the natural join of two relations. Describe how to extend the hash-join algorithm to compute the natural left outer join, the natural right outer join, and the natural full outer join. (Hint: Keep extra information with each tuple in the hash index to detect whether any tuple in the probe relation matches the tuple in the hash index.) Try out your algorithm on the *takes* and *student* relations.

15.22 Suppose you have to compute ${}_{A}\gamma_{sum(C)}(r)$ as well as ${}_{A,B}\gamma_{sum(C)}(r)$. Describe how to compute these together using a single sorting of r.

15.23 Write pseudocode for an iterator that implements a version of the sort – merge algorithm where the result of the final merge is pipelined to its consumers. Your pseudocode must define the standard iterator functions *open*(), *next*(), and *close*(). Show what state information the iterator must maintain between calls.

15.24 Explain how to split the hybrid hash-join operator into sub-operators to model pipelining. Also explain how this split is different from the split for a hash-join operator.

15.25 Suppose you need to sort relation r using sort–merge and merge–join the result with an already sorted relation s.

a. Describe how the sort operator is broken into suboperators to model the pipelining in this case.

b. The same idea is applicable even if both inputs to the merge join are the outputs of sort–merge operations. However, the available memory has to be shared between the two merge operations (the merge–join algorithm itself needs very little memory). What is the effect of having to share memory on the cost of each sort-merge operation?

Further Reading

[Graefe (1993)] presents an excellent survey of query-evaluation techniques. [Faerber et al. (2017)] describe main-memory database implementation techniques, including query processing techniques for main-memory databases, while [Kemper et al. (2012)] describes techniques for query processing with in-memory columnar data. [Samet (2006)] provides a textbook description of spatial data structures, while [Shekhar and Chawla (2003)] provides a textbook description of spatial databases, including indexing and query processing techniques. Textbook descriptions of techniques for indexing documents, and efficiently computing ranked answers to keyword queries may be found in [Manning et al. (2008)].

Bibliography

[Faerber et al. (2017)] F. Faerber, A. Kemper, P.-A. Larson, J. Levandoski, T. Neumann, and A. Pavlo, "Main Memory Database Systems", *Foundations and Trends in Databases*, Volume 8, Number 1-2 (2017), pages 1–130.

[Graefe (1993)] G. Graefe, "Query Evaluation Techniques for Large Databases", *ACM Computing Surveys*, Volume 25, Number 2 (1993).

[Kemper et al. (2012)] A. Kemper, T. Neumann, F. Funke, V. Leis, and H. Mühe, "HyPer: Adapting Columnar Main-Memory Data Management for Transaction AND Query Processing", *IEEE Data Engineering Bulletin*, Volume 35, Number 1 (2012), pages 46–51.

[Manning et al. (2008)] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press (2008).

[Samet (2006)] H. Samet, *Foundations of Multidimensional and Metric Data Structures*, Morgan Kaufmann (2006).

[Shekhar and Chawla (2003)] S. Shekhar and S. Chawla, *Spatial Databases: A TOUR*, Pearson (2003).

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

CHAPTER 16

Query Optimization

Query optimization is the process of selecting the most efficient query-evaluation plan from among the many strategies usually possible for processing a given query, especially if the query is complex. We do not expect users to write their queries so that they can be processed efficiently. Rather, we expect the system to construct a query-evaluation plan that minimizes the cost of query evaluation. This is where query optimization comes into play.

One aspect of optimization occurs at the relational-algebra level, where the system attempts to find an expression that is equivalent to the given expression, but more efficient to execute. Another aspect is selecting a detailed strategy for processing the query, such as choosing the algorithm to use for executing an operation, choosing the specific indices to use, and so on.

The difference in cost (in terms of evaluation time) between a good strategy and a bad strategy is often substantial and may be several orders of magnitude. Hence, it is worthwhile for the system to spend a substantial amount of time on the selection of a good strategy for processing a query, even if the query is executed only once.

16.1 Overview

Consider the following relational-algebra expression, for the query "Find the names of all instructors in the Music department together with the course title of all the courses that the instructors teach."[1]

$$\Pi_{name,title}\,(\sigma_{dept\_name = \text{"Music"}}\,(instructor \bowtie (teaches \bowtie \Pi_{course\_id,title}(course))))$$

The subexpression $instructor \bowtie teaches \bowtie \Pi_{course\_id,title}(course)$ in the preceding expression can create a very large intermediate result. However, we are interested in only a few tuples of this intermediate result, namely, those pertaining to instructors in the

[1]Note that the projection of *course* on (*course_id*, *title*) is required since *course* shares an attribute *dept_name* with *instructor*; if we did not remove this attribute using the projection, the above expression using natural joins would return only courses from the Music department, even if some Music department instructors taught courses in other departments.

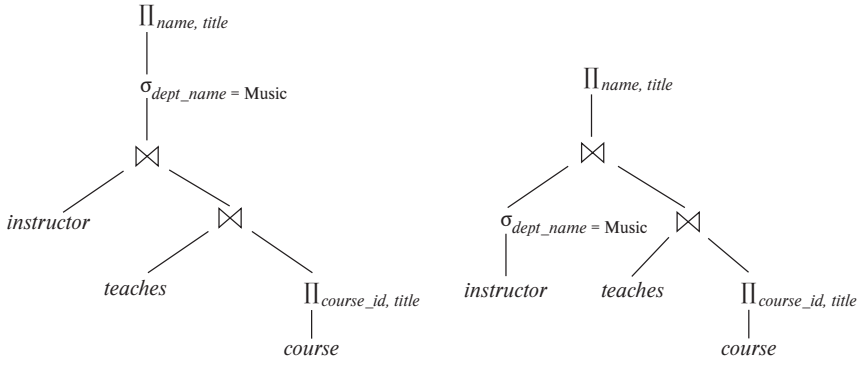


Figure 16.1 Equivalent expressions.

Music department, and in only two of the nine attributes of this relation. Since we are concerned with only those tuples in the *instructor* relation that pertain to the Music department, we do not need to consider those tuples that do not have *dept_name* = "Music". By reducing the number of tuples of the *instructor* relation that we need to access, we reduce the size of the intermediate result. Our query is now represented by the relational-algebra expression:

$$\Pi_{name,title}\left((\sigma_{dept\_name = \text{"Music"}}(instructor)) \bowtie (teaches \bowtie \Pi_{course\_id,title}(course))\right)$$

which is equivalent to our original algebra expression, but which generates smaller intermediate relations. Figure 16.1 depicts the initial and transformed expressions.

An evaluation plan defines exactly what algorithm should be used for each operation and how the execution of the operations should be coordinated. Figure 16.2 illustrates one possible evaluation plan for the expression from Figure 16.1(b). As we have seen, several different algorithms can be used for each relational operation, giving rise to alternative evaluation plans. In the figure, hash join has been chosen for one of the join operations, while the other uses merge join, after sorting the relations on the join attribute, which is ID. All edges are assumed to be pipelined, unless marked as materialized. With pipelined edges the output of the producer is sent directly to the consumer, without being written out to disk; with materialized edges, on the other hand, the output is written to disk, and then read from the disk by the consumer. There are no materialized edges in the evaluation plan in Figure 16.2, although some of the operators, such as sort and hash join, can be represented using suboperators with materialized edges between the suboperators, as we saw in Section 15.7.2.2.

Given a relational-algebra expression, it is the job of the query optimizer to come up with a query-evaluation plan that computes the same result as the given expression, and is the least costly way of generating the result (or, at least, is not much costlier than the least costly way).

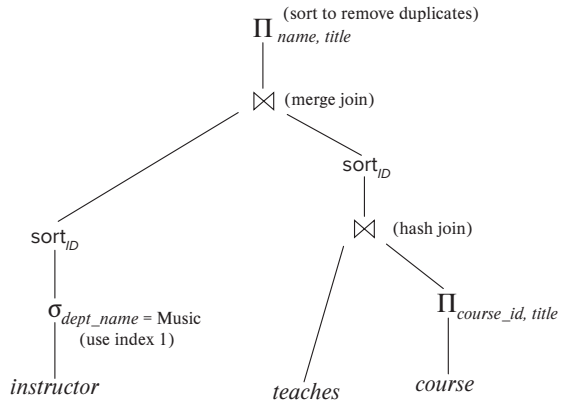


Figure 16.2 An evaluation plan.

The expression that we saw in Figure 16.1 may not necessarily lead to the least-cost evaluation plan for computing the result, since it still computes the join of the entire *teaches* relation with the *course* relation. The following expression gives the same final result, but generates smaller intermediate results, since it joins *teaches* with only *instructor* tuples corresponding to the Music department, and then joins that result with *course*.

$$\Pi_{name,title}\ ((\sigma_{dept\_name = \text{“Music”}}\ (instructor) \bowtie\ teaches) \bowtie\ \Pi_{course\_id,title}(course))$$

Regardless of the way the query is written, it is the job of the optimizer to find the least-cost plan for the query.

To find the least costly query-evaluation plan, the optimizer needs to generate alternative plans that produce the same result as the given expression and to choose the least costly one. Generation of query-evaluation plans involves three steps: (1) generating expressions that are logically equivalent to the given expression, (2) annotating the resultant expressions in alternative ways to generate alternative query-evaluation plans, and (3) estimating the cost of each evaluation plan, and choosing the one whose estimated cost is the least.

Steps (1), (2), and (3) are interleaved in the query optimizer—some expressions are generated and annotated to generate evaluation plans, then further expressions are generated and annotated, and so on. As evaluation plans are generated, their costs are estimated by using statistical information about the relations, such as relation sizes and index depths.

To implement the first step, the query optimizer must generate expressions equivalent to a given expression. It does so by means of *equivalence rules* that specify how to transform an expression into a logically equivalent one. We describe these rules in Section 16.2.

In Section 16.3 we describe how to estimate statistics of the results of each operation in a query plan. Using these statistics with the cost formulae in Chapter 15 allows

Note 16.1 VIEWING QUERY EVALUATION PLANS

Most database systems provide a way to view the evaluation plan chosen to execute a given query. It is usually best to use the GUI provided with the database system to view evaluation plans. However, if you use a command line interface, many databases support variations of a command "**explain** <query>", which displays the execution plan chosen for the specified query <query>. The exact syntax varies with different databases:

- PostgreSQL uses the syntax shown above.
- Oracle uses the syntax **explain plan for**. However, the command stores the resultant plan in a table called *plan_table*, instead of displaying it. The query "**select * from table**(*dbms_xplan.display*);" displays the stored plan.
- DB2 follows a similar approach to Oracle, but requires the program `db2exfmt` to be executed to display the stored plan.
- SQL Server requires the command **set showplan_text on** to be executed before submitting the query; then, when a query is submitted, instead of executing the query, the evaluation plan is displayed.
- MySQL uses the same **explain** <query> syntax as PostgreSQL, but the output is a table whose contents are not easy to understand. However, executing **show warnings** after the **explain** command displays the evaluation plan in a more human-readable format.

The estimated costs for the plan are also displayed along with the plan. It is worth noting that the costs are usually not in any externally meaningful unit, such as seconds or I/O operations, but rather in units of whatever cost model the optimizer uses. Some optimizers such as PostgreSQL display two cost-estimate numbers; the first indicates the estimated cost for outputting the first result, and the second indicates the estimated cost for outputting all results.

us to estimate the costs of individual operations. The individual costs are combined to determine the estimated cost of evaluating a given relational-algebra expression, as outlined in Section 15.7.

In Section 16.4, we describe how to choose a query-evaluation plan. We can choose one based on the estimated cost of the plans. Since the cost is an estimate, the selected plan is not necessarily the least costly plan; however, as long as the estimates are good, the plan is likely to be the least costly one, or not much more costly than it.

Finally, materialized views help to speed up processing of certain queries. In Section 16.5, we study how to "maintain" materialized views—that is, to keep them up-to-date—and how to perform query optimization with materialized views.

16.2 Transformation of Relational Expressions

A query can be expressed in several different ways, with different costs of evaluation. In this section, rather than take the relational expression as given, we consider alternative, equivalent expressions.

Two relational-algebra expressions are said to be **equivalent** if, on every legal database instance, the two expressions generate the same set of tuples. (Recall that a legal database instance is one that satisfies all the integrity constraints specified in the database schema.) Note that the order of the tuples is irrelevant; the two expressions may generate the tuples in different orders, but would be considered equivalent as long as the set of tuples is the same.

In SQL, the inputs and outputs are multisets of tuples, and the multiset version of the relational algebra (described in Note 3.1 on page 80, Note 3.2 on page 97, and Note 3.3 on page 108) is used for evaluating SQL queries. Two expressions in the *multiset* version of the relational algebra are said to be equivalent if on every legal database the two expressions generate the same multiset of tuples. The discussion in this chapter is based on the relational algebra. We leave extensions to the multiset version of the relational algebra to you as exercises.

16.2.1 Equivalence Rules

An **equivalence rule** says that expressions of two forms are equivalent. We can replace an expression of the first form with an expression of the second form, or vice versa—that is, we can replace an expression of the second form by an expression of the first form—since the two expressions generate the same result on any valid database. The optimizer uses equivalence rules to transform expressions into other logically equivalent expressions.

We now describe several equivalence rules on relational-algebra expressions. Some of the equivalences listed appear in Figure 16.3. We use $\theta, \theta_1, \theta_2$, and so on to denote predicates, L_1, L_2, L_3, and so on to denote lists of attributes, and E, E_1, E_2, and so on to denote relational-algebra expressions. A relation name r is simply a special case of a relational-algebra expression and can be used wherever E appears.

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections. This transformation is referred to as a cascade of σ.

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are **commutative**.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

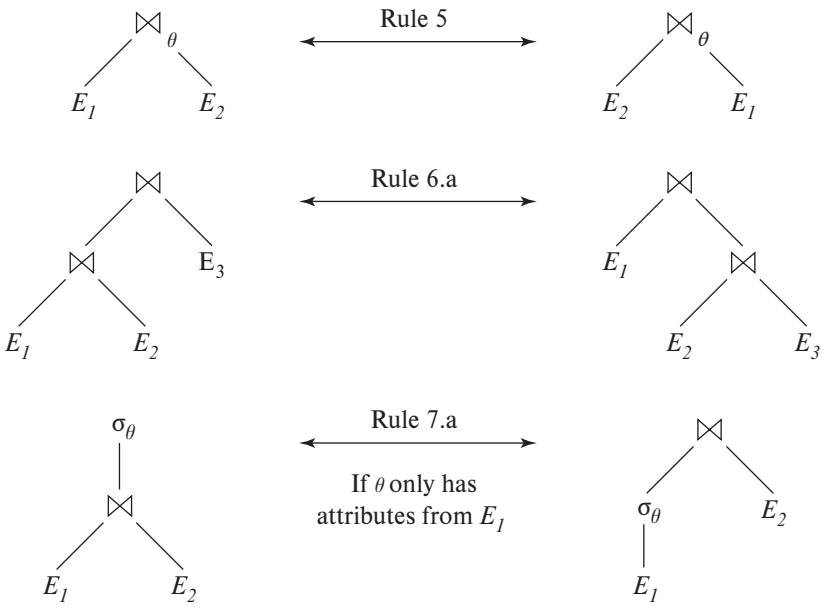


Figure 16.3 Pictorial representation of equivalences.

3. Only the final operations in a sequence of projection operations are needed; the others can be omitted. This transformation can also be referred to as a cascade of Π.

$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{L_n}(E))\ldots)) \equiv \Pi_{L_1}(E)$$

where $L_1 \subseteq L_2 \subseteq \ldots \subseteq L_n$.

4. Selections can be combined with Cartesian products and theta joins.

   a. $\sigma_\theta(E_1 \times E_2) \equiv E_1 \bowtie_\theta E_2$

      This expression is just the definition of the theta join.

   b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

5. Theta-join operations are commutative.

$$E_1 \bowtie_\theta E_2 \equiv E_2 \bowtie_\theta E_1$$

Recall that the natural-join operator is simply a special case of the theta-join operator; hence, natural joins are also commutative.

The order of attributes differs between the left-hand side and right-hand side of the commutativity rule, so the equivalence does not hold if the order of attributes is taken into account. Since we use a version of relational algebra where every attribute must have a name for it to be referenced, the order of attributes

does not actually matter, except when the result is finally displayed. When the order does matter, a projection operation can be added to one of the sides of the equivalence to appropriately reorder attributes. However, for simplicity, we omit the projection and ignore the attribute order in all our equivalence rules.

6. a. Natural-join operations are **associative**.

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

b. Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3. Any of these conditions may be empty; hence, it follows that the Cartesian product ($\times$) operation is also associative. The commutativity and associativity of join operations are important for join reordering in query optimization.

7. The selection operation distributes over the theta-join operation under the following two conditions:

a. Selection distributes over the theta-join operation when all the attributes in selection condition θ_1 involve only the attributes of one of the expressions (say, E_1) being joined.

$$\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} E_2$$

b. Selection distributes over the theta-join operation when selection condition θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2.

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

8. The projection operation distributes over the theta-join operation under the following conditions.

a. Let L_1 and L_2 be attributes of E_1 and E_2, respectively. Suppose that the join condition θ involves only attributes in $L_1 \cup L_2$. Then,

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$$

b. Consider a join $E_1 \bowtie_{\theta} E_2$. Let L_1 and L_2 be sets of attributes from E_1 and E_2, respectively. Let L_3 be attributes of E_1 that are involved in join condition θ, but are not in L_1 and let L_4 be attributes of E_2 that are involved in join condition θ, but are not in L_2. Then,

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$$

Similar equivalences hold for outer join operations $\mathbin{⟕}$, $\mathbin{⟗}$ and $\mathbin{⟖}$.

9. The set operations union and intersection are commutative.

   a. $E_1 \cup E_2 \equiv E_2 \cup E_1$

   b. $E_1 \cap E_2 \equiv E_2 \cap E_1$

   Set difference is not commutative.

10. Set union and intersection are associative.

    a. $(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$

    b. $(E_1 \cap E_2) \cap E_3 \equiv E_1 \cap (E_2 \cap E_3)$

11. The selection operation distributes over the union, intersection, and set-difference operations.

    a. $\sigma_\theta(E_1 \cup E_2) \equiv \sigma_\theta(E_1) \cup \sigma_\theta(E_2)$

    b. $\sigma_\theta(E_1 \cap E_2) \equiv \sigma_\theta(E_1) \cap \sigma_\theta(E_2)$

    c. $\sigma_\theta(E_1 - E_2) \equiv \sigma_\theta(E_1) - \sigma_\theta(E_2)$

    d. $\sigma_\theta(E_1 \cap E_2) \equiv \sigma_\theta(E_1) \cap E_2$

    e. $\sigma_\theta(E_1 - E_2) \equiv \sigma_\theta(E_1) - E_2$

    The preceding equivalence does not hold if $-$ is replaced by $\cup$.

12. The projection operation distributes over the union operation

$$\Pi_L(E_1 \cup E_2) \equiv (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

    provided E_1 and E_2 have the same schema.

13. Selection distributes over aggregation under the following conditions. Let G be a set of group by attributes, and A a set of aggregate expressions. When θ only involves attributes in G, the following equivalence holds.

$$\sigma_\theta({}_G\gamma_A(E) \equiv {}_G\gamma_A(\sigma_\theta(E))$$

14. a. Full outer join is commutative.

$$E_1 ⟗ E_2 \equiv E_2 ⟗ E_2$$

    b. Left and right outer join are not commutative. However, left outer join and right outer join can be exchanged as follows.

$$E_1 ⟕ E_2 \equiv E_2 ⟖ E_1$$

15. Selection distributes over left and right outer join under some conditions. Specifically, when the selection condition θ_1 involves only the attributes of one of the expressions being joined, say E_1, the following equivalences hold.

a. $\sigma_{\theta_1}(E_1 \rtimes_\theta E_2) \equiv (\sigma_{\theta_1}(E_1)) \rtimes_\theta E_2$

b. $\sigma_{\theta_1}(E_2 \ltimes_\theta E_1) \equiv (E_2 \ltimes_\theta (\sigma_{\theta_1}(E_1)))$

16. Outer joins can be replaced by inner joins under some conditions. Specifically, if θ_1 has the property that it evaluates to false or unknown whenever the attributes of E_2 are null, then the following equivalences hold.

a. $\sigma_{\theta_1}(E_1 \rtimes_\theta E_2) \equiv \sigma_{\theta_1}(E_1 \bowtie_\theta E_2)$

b. $\sigma_{\theta_1}(E_2 \ltimes_\theta E_1) \equiv \sigma_{\theta_1}(E_2 \bowtie_\theta E_2)$

A predicate θ_1 satisfying the above property is said to be **null rejecting** on E_2. For example, if θ_1 is of the form $A < 4$ where A is an attribute from E_2, then θ_1 would evaluate to unknown whenever A is null, and as a result any tuples in $E_1 \rtimes_\theta E_2$ that are not in $E_1 \bowtie_\theta E_2$ would be rejected by σ_{θ_1}. We can therefore replace the outer join by an inner join (or vice versa).

More generally, the condition would hold if θ_1 is of the form $\theta_1^1 \wedge \theta_1^2 \wedge \ldots \wedge \theta_1^k$, and at least one of the terms θ_1^i is of the form e_1 *relop* e_2, where e_1 and e_2 are arithmetic or string expressions involving at least one attribute from E_2, and *relop* is any of $<, \leq, =, \geq, >$.

This is only a partial list of equivalences. More equivalences are discussed in the exercises.

Some equivalences that hold for joins do not hold for outer joins. For example, the selection operation does not distribute over outer join when the conditions specified in rule 15.a or rule 15.b do hold. To see this, we look at the expression:

$$\sigma_{year=2017}(instructor \rtimes teaches)$$

and consider the case of an instructor who teaches no courses at all, regardless of year. In the above expression, the left outer join retains a tuple for each such instructor with a null value for *year*. Then the selection operation removes those tuples since the predicate *null*=2017 evaluates to *unknown*, and such instructors do not appear in the result. However, if we push the selection operation down to *teaches*, the resulting expression:

$$instructor \rtimes \sigma_{year=2017}(teaches)$$

is syntactically correct since the selection predicate includes only attributes from *teaches*, but the result is different. For an instructor that does not teach at all, the *instructor* tuple appears in the result of $instructor \rtimes \sigma_{year=2017}(teaches)$, but not in the result of $\sigma_{year=2017}(instructor \rtimes teaches)$. The following equivalence, however, does hold:

$$\sigma_{year=2017}(instructor \rtimes teaches) \equiv \sigma_{year=2017}(instructor \bowtie teaches)$$

As another example, unlike inner joins, outer joins are not associative. We show thus using an example for the natural left outer join. Similar examples can be con-

structed for natural right and natural full outer join, as well as for the corresponding theta-join versions of the outer join operations.

Let relation $r(A, B)$ be a relation consisting of the single tuple $(1, 1)$, $s(B, C)$ be a relation consisting of the single tuple $(1, 1)$, and $t(A, C)$ be an empty relation with no tuples. We shall show that for this example,

$$(r \rtimes s) \rtimes t \not\equiv r \rtimes (s \rtimes t)$$

To see this, note first that $(r \rtimes s)$ produces a result with schema (A, B, C) having one tuple $(1, 1, 1)$. Computing the left outer join of that result with relation t produces a result with schema (A, B, C) having one tuple $(1, 1, 1)$. Next, we look at the expression $r \rtimes (s \rtimes t)$, and note that $s \rtimes t$ produces a result with schema (A, B, C) having one tuple $(null, 1, 1)$. Computing the left outer join of r with that result produces a result with schema (A, B, C) having one tuple $(1, 1, null)$.

16.2.2 Examples of Transformations

We now illustrate the use of the equivalence rules. We use our university example with the relation schemas:

instructor(*ID*, *name*, *dept_name*, *salary*)
teaches(*ID*, *course_id*, *sec_id*, *semester*, *year*)
course(*course_id*, *title*, *dept_name*, *credits*)

In our example in Section 16.1, the expression:

$$\Pi_{name,title}\,(\sigma_{dept\_name = \text{“Music”}}\,(instructor \bowtie (teaches \bowtie \Pi_{course\_id,title}(course))))$$

was transformed into the following expression:

$$\Pi_{name,title}\,((\sigma_{dept\_name = \text{“Music”}}\,(instructor)) \bowtie (teaches \bowtie \Pi_{course\_id,title}(course)))$$

which is equivalent to our original algebra expression but generates smaller intermediate relations. We can carry out this transformation by using rule 7.a. Remember that the rule merely says that the two expressions are equivalent; it does not say that one is better than the other.

Multiple equivalence rules can be used, one after the other, on a query or on parts of the query. As an illustration, suppose that we modify our original query to restrict attention to instructors who have taught a course in 2017. The new relational-algebra query is:

$$\Pi_{name,title}\,(\sigma_{dept\_name = \text{“Music”} \wedge year\ =\ 2017}\,(instructor \bowtie (teaches \bowtie \Pi_{course\_id,title}(course))))$$

We cannot apply the selection predicate directly to the *instructor* relation, since the predicate involves attributes of both the *instructor* and *teaches* relations. However, we can first apply rule 6.a (associativity of natural join) to transform the join $instructor \bowtie (teaches \bowtie \Pi_{course\_id,title}(course))$ into $(instructor \bowtie teaches) \bowtie \Pi_{course\_id,title}(course)$:

$$\Pi_{name,title}\ (\sigma_{dept\_name = \text{“Music”} \wedge year\ =\ 2017} ((instructor \bowtie teaches) \bowtie \Pi_{course\_id,title}(course)))$$

Then, using rule 7.a, we can rewrite our query as:

$$\Pi_{name,title}\ ((\sigma_{dept\_name = \text{“Music”} \wedge year\ =\ 2017} (instructor \bowtie teaches)) \bowtie \Pi_{course\_id,title}(course))$$

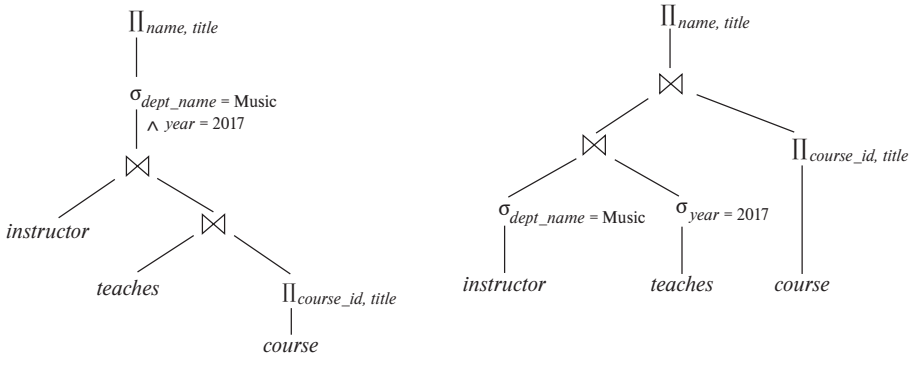
Let us examine the selection subexpression within this expression. Using rule 1, we can break the selection into two selections to get the following subexpression:

$$\sigma_{dept\_name = \text{“Music”}}\ (\sigma_{year\ =\ 2017}\ (instructor \bowtie teaches))$$

Both of the preceding expressions select tuples with *dept_name* = “Music” and *course_id* = 2017. However, the latter form of the expression provides a new opportunity to apply rule 7.a (“perform selections early”), resulting in the subexpression:

$$\sigma_{dept\_name = \text{“Music”}}\ (instructor) \bowtie \sigma_{year\ =\ 2017}\ (teaches)$$

Figure 16.4 depicts the initial expression and the final expression after all these transformations. We could equally well have used rule 7.b to get the final expression directly, without using rule 1 to break the selection into two selections. In fact, rule 7.b can itself be derived from rules 1 and 7.a.



(a) Initial expression tree

(b) Tree after multiple transformations

Figure 16.4 Multiple transformations.

A set of equivalence rules is said to be **minimal** if no rule can be derived from any combination of the others. The preceding example illustrates that the set of equivalence rules in Section 16.2.1 is not minimal. An expression equivalent to the original expression may be generated in different ways; the number of different ways of generating an expression increases when we use a nonminimal set of equivalence rules. Query optimizers therefore use minimal sets of equivalence rules.

Now consider the following form of our example query:

$$\Pi_{name,title}\ ((\sigma_{dept\_name = \text{“Music”}}\ (instructor) \bowtie teaches) \bowtie \Pi_{course\_id,title}(course))$$

When we compute the subexpression:

$$(\sigma_{dept\_name = \text{“Music”}}\ (instructor) \bowtie teaches)$$

we obtain a relation whose schema is:

$$(ID, name, dept\_name, salary, course\_id, sec\_id, semester, year)$$

We can eliminate several attributes from the schema by pushing projections based on equivalence rules 8.a and 8.b. The only attributes that we must retain are those that either appear in the result of the query or are needed to process subsequent operations. By eliminating unneeded attributes, we reduce the number of columns of the intermediate result. Thus, we reduce the size of the intermediate result. In our example, the only attributes we need from the join of *instructor* and *teaches* are *name* and *course_id*. Therefore, we can modify the expression to:

$$\Pi_{name,title}\ ((\Pi_{name,course\_id}\ ((\sigma_{dept\_name = \text{“Music”}}\ (instructor)) \bowtie teaches)) \bowtie \Pi_{course\_id,title}(course))$$

The projection $\Pi_{name,course\_id}$ reduces the size of the intermediate join results.

16.2.3 Join Ordering

A good ordering of join operations is important for reducing the size of temporary results; hence, most query optimizers pay a lot of attention to the join order. As mentioned in equivalence rule 6.a, the natural-join operation is associative. Thus, for all relations r_1, r_2, and r_3:

$$(r_1 \bowtie r_2) \bowtie r_3 \equiv r_1 \bowtie (r_2 \bowtie r_3)$$

Although these expressions are equivalent, the costs of computing them may differ. Consider again the expression:

$$\Pi_{name,title}\ ((\sigma_{dept\_name = \text{“Music”}}\ (instructor)) \bowtie teaches \bowtie \Pi_{course\_id,title}(course))$$

We could choose to compute $teaches \bowtie \Pi_{course\_id,title}(course)$ first, and then to join the result with:

$$\sigma_{dept\_name = \text{“Music”}}(instructor)$$

However, $teaches \bowtie \Pi_{course\_id,title}(course)$ is likely to be a large relation, since it contains one tuple for every course taught. In contrast:

$$\sigma_{dept\_name = \text{“Music”}}(instructor) \bowtie teaches$$

is probably a small relation. To see that it is, we note that a university has fewer instructors than courses and, since a university has a large number of departments, it is likely that only a small fraction of the university instructors are associated with the Music department. Thus, the preceding expression results in one tuple for each course taught by an instructor in the Music department. Therefore, the temporary relation that we must store is smaller than it would have been had we computed $teaches \bowtie \Pi_{course\_id,title}(course)$ first.

There are other options to consider for evaluating our query. We do not care about the order in which attributes appear in a join, since it is easy to change the order before displaying the result. Thus, for all relations r_1 and r_2:

$$r_1 \bowtie r_2 \equiv r_2 \bowtie r_1$$

That is, natural join is commutative (equivalence rule 5).

Using the associativity and commutativity of the natural join (rules 5 and 6), consider the following relational-algebra expression:

$$(instructor \bowtie \Pi_{course\_id,title}(course)) \bowtie teaches$$

Note that there are no attributes in common between $\Pi_{course\_id,title}(course)$ and *instructor*, so the join is just a Cartesian product. If there are a tuples in *instructor* and b tuples in $\Pi_{course\_id,title}(course)$, this Cartesian product generates $a * b$ tuples, one for every possible pair of instructor tuple and course (without regard for whether the instructor taught the course). This Cartesian product would produce a very large temporary relation. However, if the user had entered the preceding expression, we could use the associativity and commutativity of the natural join to transform this expression to the more efficient expression:

$$(instructor \bowtie teaches) \bowtie \Pi_{course\_id,title}(course)$$

16.2.4 Enumeration of Equivalent Expressions

Query optimizers can use equivalence rules to systematically generate expressions equivalent to the given query expression. The cost of an expression is computed based

```
procedure genAllEquivalent(E)
EQ = {E}
repeat
    Match each expression E_i in EQ with each equivalence rule R_j
    if any subexpression e_i of E_i matches one side of R_j
        Create a new expression E' which is identical to E_i, except that
            e_i is transformed to match the other side of R_j
        Add E' to EQ if it is not already present in EQ
until no new expression can be added to EQ
```

Figure 16.5 Procedure to generate all equivalent expressions.

on statistics that are discussed in Section 16.3. Cost-based query optimizers, described in Section 16.4 compute the cost of each alternative and pick the least cost alternative.

Conceptually, enumeration of equivalent expressions can be done as outlined in Figure 16.5. The process proceeds as follows: Given a query expression E, the set of equivalent expressions EQ initially contains only E. Now, each expression in EQ is matched with each equivalence rule. If a subexpression e_j of any expression $E_i \in EQ$ (as a special case, e_j could be E_i itself) matches one side of an equivalence rule, the optimizer generates a copy E_k of E_i, in which e_j is transformed to match the other side of the rule, and adds E_k to EQ. This process continues until no more new expressions can be generated. With a properly chosen set of equivalence rules, the set of equivalent expressions is finite, and the process can be guaranteed to terminate.

For example, given an expression $r \bowtie (s \bowtie t)$, the commutativity rule can match the subexpression $(s \bowtie t)$, and would create a new expression $r \bowtie (t \bowtie s)$. The commutativity rule also matches the join at the root of $r \bowtie (s \bowtie t)$, and creates a new expression $(s \bowtie t) \bowtie r$. Associativity and commutativity rules can continue to be applied to generate new expressions. But eventually applying any equivalence rule will only generate expressions that were already generated earlier, and the process will terminate.

The preceding process is extremely costly both in space and in time, but optimizers can greatly reduce both the space and time cost, using two key ideas.

1. If we generate an expression E' from an expression E_1 by using an equivalence rule on subexpression e_i, then E' and E_1 have identical subexpressions except for e_i and its transformation. Even e_i and its transformed version usually share many identical subexpressions. Expression-representation techniques that allow both expressions to point to shared subexpressions can reduce the space requirement significantly.

2. It is not always necessary to generate every expression that can be generated with the equivalence rules. If an optimizer takes cost estimates of evaluation into account, it may be able to avoid examining some of the expressions, as we shall see in Section 16.4. We can reduce the time required for optimization by using techniques such as these.

With these and other techniques to reduce the optimization time, equivalence rules can be used to enumerate alternative plans, whose costs can be computed; the lowest-cost plan amongst the alternatives is then chosen. We discuss efficient implementation of cost-based query optimization based on equivalence rules in Section 16.4.2.

Some query optimizers use equivalence rules in a heuristic manner. With such an approach, if the left-hand side of an equivalence rule matches a subtree in a query plan, the subtree is rewritten to match the right-hand side of the rule. This process is repeated till the query plan cannot be further rewritten. Rules must be carefully chosen such that the cost decreases when a rule is applied, and rewriting must eventually terminate. Although this approach can be implemented to execute quite fast, there is no guarantee that it will find the optimal plan.

Yet other query optimizers focus on join order selection, which is often a key factor in query cost. We discuss algorithms for join-order optimization in Section 16.4.1.

16.3 Estimating Statistics of Expression Results

The cost of an operation depends on the size and other statistics of its inputs. Given an expression such as $r \bowtie (s \bowtie t)$ to estimate the cost of joining r with $(s \bowtie t)$, we need to have estimates of statistics such as the size of $s \bowtie t$.

In this section, we first list some statistics about database relations that are stored in database-system catalogs, and then show how to use the stored statistics to estimate statistics on the results of various relational operations.

Given a query expression, we consider it as a tree; we can start from the bottom-level operations, and estimate their statistics, and continue the process on higher-level operations, till we reach the root of the tree. The size estimates that we compute as part of these statistics can be used to compute the cost of algorithms for individual operations in the tree, and these costs can be added up to find the cost of an entire query plan, as we saw in Chapter 15.

One thing that will become clear later in this section is that the estimates are not very accurate, since they are based on assumptions that may not hold exactly. A query-evaluation plan that has the lowest estimated execution cost may therefore not actually have the lowest actual execution cost. However, real-world experience has shown that even if estimates are not precise, the plans with the lowest estimated costs usually have actual execution costs that are either the lowest actual execution costs or are close to the lowest actual execution costs.

16.3.1 Catalog Information

The database-system catalog stores the following statistical information about database relations:

- n_r, the number of tuples in the relation r.
- b_r, the number of blocks containing tuples of relation r.
- l_r, the size of a tuple of relation r in bytes.
- f_r, the blocking factor of relation r—that is, the number of tuples of relation r that fit into one block.
- $V(A, r)$, the number of distinct values that appear in the relation r for attribute A. This value is the same as the size of $\Pi_A(r)$. If A is a key for relation r, $V(A, r)$ is n_r.

The last statistic, $V(A, r)$, can also be maintained for sets of attributes, if desired, instead of just for individual attributes. Thus, given a set of attributes, $\mathcal{A}$, $V(\mathcal{A}, r)$ is the size of $\Pi_{\mathcal{A}}(r)$.

If we assume that the tuples of relation r are stored together physically in a file, the following equation holds:

$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

Statistics about indices, such as the heights of B^+-tree indices and number of leaf pages in the indices, are also maintained in the catalog.

If we wish to maintain accurate statistics, then every time a relation is modified, we must also update the statistics. This update incurs a substantial amount of overhead. Therefore, most systems do not update the statistics on every modification. Instead, they update the statistics during periods of light system load. As a result, the statistics used for choosing a query-processing strategy may not be completely accurate. However, if not too many updates occur in the intervals between the updates of the statistics, the statistics will be sufficiently accurate to provide a good estimation of the relative costs of the different plans.

The statistical information noted here is simplified. Real-world optimizers often maintain further statistical information to improve the accuracy of their cost estimates of evaluation plans. For instance, most databases store the distribution of values for each attribute as a **histogram**: in a histogram, the values for the attribute are divided into a number of ranges, and with each range the histogram associates the number of tuples whose attribute value lies in that range. Figure 16.6 shows an example of a histogram for an integer-valued attribute that takes values in the range 1 to 25.

As an example of a histogram, the range of values for an attribute *age* of a relation *person* could be divided into 0–9, 10–19, . . . , 90–99 (assuming a maximum age of 99). With each range we store a count of the number of *person* tuples whose *age* values lie in that range.

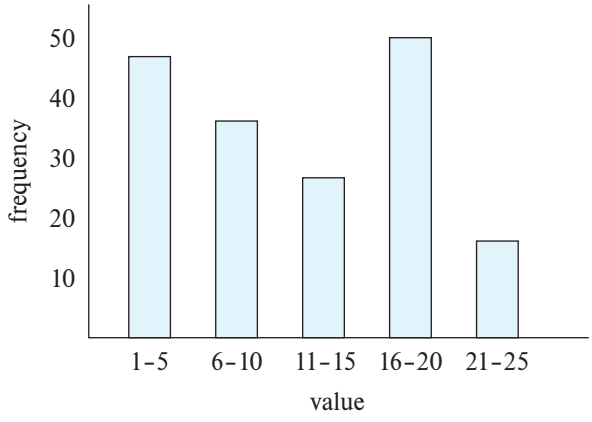


Figure 16.6 Example of histogram.

The histogram shown in Figure 16.6, is an **equi-width histogram** since it divides the range of values into equal-sized ranges. In contrast, an **equi-depth** histogram adjusts the boundaries of the ranges such that each range has the same number of values. Thus, an equi-depth histogram merely stores the boundaries of partitions of the range, and need not store the number of values. For example, the following could be the equidepth histogram for the data whose equi-width histogram is shown in Figure 16.6:

(4, 8, 14, 19)

The histogram indicates that 1/5th of the tuples have age less than 4, another 1/5th have age ≥ 4 but < 8, and so on, with the last 1/5th having age ≥ 19. Information about the total number of tuples is also stored with the equi-width histogram. Equi-depth histograms are preferred to equi-width histograms since they provide better estimates, and occupy less space.

Histograms used in database systems can also record the number of distinct values in each range, in addition to the number of tuples with attribute values in that range. In our example, the histogram could store the number of distinct age values that lie in each range. Without such histogram information, an optimizer would have to assume that the distribution of values is uniform; that is, each range has the same number of distinct values.

In many database applications, some values are very frequent, compared to other values. To get better estimates for queries that specify these values, many databases store a list of n most frequent values for some n (say 5 or 10), along with the number of times each value appears. In our example, if ages 4, 7, 18, 19, and 23 are the five most frequently occurring values, the database could store the number of persons having each of these ages. The histogram then only stores statistics for age values other than these five values, since we have now have exact counts for these values.

A histogram takes up only a little space, so histograms on several different attributes can be stored in the system catalog.

16.3.2 Selection Size Estimation

The size estimate of the result of a selection operation depends on the selection predicate. We first consider a single equality predicate, then a single comparison predicate, and finally combinations of predicates.

- $\sigma_{A=a}(r)$: If a is a frequently occurring value for which the occurrence count is available, we can use that value directly as the size estimate for the selection.

  Otherwise if there is no histogram available, we assume uniform distribution of values (i.e., each value appears with equal probability), the selection result is estimated to have $n_r/V(A,r)$ tuples, assuming that the value a appears in attribute A of some record of r. The assumption that the value a in the selection appears in some record is generally true, and cost estimates often make it implicitly. However, it is often not realistic to assume that each value appears with equal probability. The *course_id* attribute in the *takes* relation is an example where the assumption is not valid. It is reasonable to expect that a popular undergraduate course will have many more students than a smaller specialized graduate course. Therefore, certain *course_id* values appear with greater probability than do others. Despite the fact that the uniform-distribution assumption is often not correct, it is a reasonable approximation of reality in many cases, and it helps us to keep our presentation relatively simple.

  If a histogram is available on attribute A, we can locate the range that contains the value a, and modify the above-mentioned estimate $n_r/V(A,r)$ by using the frequency count for that range instead of n_r, and the number of distinct values that occurs in that range instead of $V(A,r)$.

- $\sigma_{A\leq v}(r)$: Consider a selection of the form $\sigma_{A\leq v}(r)$. Suppose that the lowest and highest values ($\min(A,r)$ and $\max(A,r)$) for the attribute are stored in the catalog. Assuming that values are uniformly distributed, we can estimate the number of records that will satisfy the condition $A \leq v$ as:

  - 0 if $v < \min(A,r)$
  - n_r if $v \geq \max(A,r)$, and,
  - $n_r \cdot \frac{v-\min(A,r)}{\max(A,r)-\min(A,r)}$, otherwise.

  If a histogram is available on attribute A, we can get a more accurate estimate; we leave the details as an exercise for you.

  In some cases, such as when the query is part of a stored procedure, the value v may not be available when the query is optimized. In such cases, we assume that approximately one-half the records will satisfy the comparison condition. That is,

Note 16.2 COMPUTING AND MAINTAINING STATISTICS

Conceptually, statistics on relations can be thought of as materialized views, which should be automatically maintained when relations are modified. Unfortunately, keeping statistics up-to-date on every insert, delete or update to the database can be very expensive. On the other hand, optimizers generally do not need exact statistics: an error of a few percent may result in a plan that is not quite optimal being chosen, but the alternative plan chosen is likely to have a cost which is within a few percent of the optimal cost. Thus, it is acceptable to have statistics that are approximate.

Database systems reduce the cost of generating and maintaining statistics, as outlined below, by exploiting the fact that statistics can be approximate.

- Statistics are often computed from a sample of the underlying data, instead of examining the entire collection of data. For example, a fairly accurate histogram can be computed from a sample of a few thousand tuples, even on a relation that has millions, or hundreds of millions of records. However, the sample used must be a **random sample**; a sample that is not random may have an excessive representation of one part of the relation and can give misleading results. For example, if we used a sample of instructors to compute a histogram on salaries, if the sample has an overrepresentation of lower-paid instructors the histogram would result in wrong estimates. Database systems today routinely use random sampling to create statistics. See the bibliographical notes online for references on sampling.
- Statistics are not maintained on every update to the database. In fact, some database systems never update statistics automatically. They rely on database administrators periodically running a command to update statistics. Oracle and PostgreSQL provide an SQL command called **analyze** that generates statistics on specified relations, or on all relations. IBM DB2 supports an equivalent command called **runstats**. See the system manuals for details. You should be aware that optimizers sometimes choose very bad plans due to incorrect statistics. Many database systems, such as IBM DB2, Oracle, and SQL Server, update statistics automatically at certain points of time. For example, the system can keep approximate track of how many tuples there are in a relation and recompute statistics if this number changes significantly. Another approach is to compare estimated cardinalities of a relation scan with actual cardinalities when a query is executed, and if they differ significantly, initiate an update of statistics for that relation.

we assume the result has $n_r/2$ tuples; the estimate may be very inaccurate, but it is the best we can do without any further information.

- Complex selections:
  - **Conjunction:** A *conjunctive selection* is a selection of the form:

$$\sigma_{\theta_1 \wedge \theta_2 \wedge \cdots \wedge \theta_n}(r)$$

We can estimate the result size of such a selection: For each θ_i, we estimate the size of the selection $\sigma_{\theta_i}(r)$, denoted by s_i, as described previously. Thus, the probability that a tuple in the relation satisfies selection condition θ_i is s_i/n_r.

The preceding probability is called the **selectivity** of the selection $\sigma_{\theta_i}(r)$. Assuming that the conditions are *independent* of each other, the probability that a tuple satisfies all the conditions is simply the product of all these probabilities. Thus, we estimate the number of tuples in the full selection as:

$$n_r * \frac{s_1 * s_2 * \cdots * s_n}{n_r^n}$$

  - **Disjunction:** A *disjunctive selection* is a selection of the form:

$$\sigma_{\theta_1 \vee \theta_2 \vee \cdots \vee \theta_n}(r)$$

A disjunctive condition is satisfied by the union of all records satisfying the individual, simple conditions θ_i.

As before, let s_i/n_r denote the probability that a tuple satisfies condition θ_i. The probability that the tuple will satisfy the disjunction is then 1 minus the probability that it will satisfy *none* of the conditions:

$$1 - (1 - \frac{s_1}{n_r}) * (1 - \frac{s_2}{n_r}) * \cdots * (1 - \frac{s_n}{n_r})$$

Multiplying this value by n_r gives us the estimated number of tuples that satisfy the selection.

  - **Negation:** In the absence of nulls, the result of a selection $\sigma_{\neg\theta}(r)$ is simply the tuples of r that are not in $\sigma_\theta(r)$. We already know how to estimate the number of tuples in $\sigma_\theta(r)$. The number of tuples in $\sigma_{\neg\theta}(r)$ is therefore estimated to be n_r minus the estimated number of tuples in $\sigma_\theta(r)$.

We can account for nulls by estimating the number of tuples for which the condition θ would evaluate to *unknown*, and subtracting that number from the above estimate, ignoring nulls. Estimating that number would require extra statistics to be maintained in the catalog.

16.3.3 Join Size Estimation

In this section, we see how to estimate the size of the result of a join.

The Cartesian product $r \times s$ contains $n_r * n_s$ tuples. Each tuple of $r \times s$ occupies $l_r + l_s$ bytes, from which we can calculate the size of the Cartesian product.

Estimating the size of a natural join is somewhat more complicated than estimating the size of a selection or of a Cartesian product. Let $r(R)$ and $s(S)$ be relations.

- If $R \cap S = \emptyset$—that is, the relations have no attribute in common—then $r \bowtie s$ is the same as $r \times s$, and we can use our estimation technique for Cartesian products.
- If $R \cap S$ is a key for R, then we know that a tuple of s will join with at most one tuple from r. Therefore, the number of tuples in $r \bowtie s$ is no greater than the number of tuples in s. The case where $R \cap S$ is a key for S is symmetric to the case just described. If $R \cap S$ forms a foreign key of S, referencing R, the number of tuples in $r \bowtie s$ is exactly the same as the number of tuples in s.
- The most difficult case is when $R \cap S$ is a key for neither R nor S. In this case, we assume, as we did for selections, that each value appears with equal probability. Consider a tuple t of r, and assume $R \cap S = \{A\}$. We estimate that tuple t produces

$$\frac{n_s}{V(A, s)}$$

tuples in $r \bowtie s$, since this number is the average number of tuples in s with a given value for the attributes A. Considering all the tuples in r, we estimate that there are

$$\frac{n_r * n_s}{V(A, s)}$$

tuples in $r \bowtie s$. Observe that, if we reverse the roles of r and s in the preceding estimate, we obtain an estimate of

$$\frac{n_r * n_s}{V(A, r)}$$

tuples in $r \bowtie s$. These two estimates differ if $V(A, r) \neq V(A, s)$. If this situation occurs, there are likely to be dangling tuples that do not participate in the join. Thus, the lower of the two estimates is probably the more accurate one.

The preceding estimate of join size may be too high if the $V(A, r)$ values for attribute A in r have few values in common with the $V(A, s)$ values for attribute A in s. However, this situation is unlikely to happen in the real world, since dangling tuples either do not exist or constitute only a small fraction of the tuples, in most real-world relations.

More important, the preceding estimate depends on the assumption that each value appears with equal probability. More sophisticated techniques for size estimation have to be used if this assumption does not hold. For example, if we have histograms on the join attributes of both relations, and both histograms have the same ranges, then we can use the above estimation technique within each range,

using the number of rows with values in the range instead of n_r or n_s, and the number of distinct values in that range, instead of $V(A, r)$ or $V(A, s)$. We then add up the size estimates obtained for each range to get the overall size estimate. We leave the case where both relations have histograms on the join attribute, but the histograms have different ranges, as an exercise for you.

We can estimate the size of a theta join $r \bowtie_\theta s$ by rewriting the join as $\sigma_\theta(r \times s)$ and using the size estimates for Cartesian products along with the size estimates for selections, which we saw in Section 16.3.2.

To illustrate all these ways of estimating join sizes, consider the expression:

$$student \bowtie takes$$

Assume the following catalog information about the two relations:

- $n_{student} = 5000$.
- $n_{takes} = 10000$.
- $V(ID, takes) = 2500$, which implies that only half the students have taken any course (this is unrealistic, but we use it to show that our size estimates are correct even in this case), and on average, each student who has taken a course has taken four courses.

Note that since *ID* is a primary key of *student*, $V(ID, student) = n_{student} = 5000$.

The attribute *ID* in *takes* is a foreign key on *student*, and null values do not occur in *takes.ID*, since *ID* is part of the primary key of *takes*; thus, the size of $student \bowtie takes$ is exactly n_{takes}, which is 10000.

We now compute the size estimates for $student \bowtie takes$ without using information about foreign keys. Since $V(ID, takes) = 2500$ and $V(ID, student) = 5000$, the two estimates we get are $5000 * 10000/2500 = 20000$ and $5000 * 10000/5000 = 10000$, and we choose the lower one. In this case, the lower of these estimates is the same as that which we computed earlier from information about foreign keys.

16.3.4 Size Estimation for Other Operations

Next we outline how to estimate the sizes of the results of other relational-algebra operations.

- **Projection:** The estimated size (number of records or number of tuples) of a projection of the form $\Pi_A(r)$ is $V(A, r)$, since projection eliminates duplicates.
- **Aggregation:** The size of ${}_G\gamma_A(r)$ is simply $V(G, r)$, since there is one tuple in ${}_G\gamma_A(r)$ for each distinct value of G.
- **Set operations:** If the two inputs to a set operation are selections on the same relation, we can rewrite the set operation as disjunctions, conjunctions, or negations. For example, $\sigma_{\theta_1}(r) \cup \sigma_{\theta_2}(r)$ can be rewritten as $\sigma_{\theta_1 \vee \theta_2}(r)$. Similarly, we can rewrite

intersections as conjunctions, and we can rewrite set difference by using negation, so long as the two relations participating in the set operations are selections on the same relation. We can then use the estimates for selections involving conjunctions, disjunctions, and negation in Section 16.3.2.

If the inputs are not selections on the same relation, we estimate the sizes this way: The estimated size of $r \cup s$ is the sum of the sizes of r and s. The estimated size of $r \cap s$ is the minimum of the sizes of r and s. The estimated size of $r - s$ is the same size as r. All three estimates may be inaccurate, but provide upper bounds on the sizes.

- **Outer join:** The estimated size of $r \rtimes s$ is the size of $r \bowtie s$ plus the size of r; that of $r \ltimes s$ is symmetric, while that of $r \rtimes\ltimes s$ is the size of $r \bowtie s$ plus the sizes of r and s. All three estimates may be inaccurate, but provide upper bounds on the sizes.

16.3.5 Estimation of Number of Distinct Values

The size estimates discussed earlier depend on statistics such as histograms, or at a minimum, the number of distinct values for an attribute. While these statistics can be precomputed and stored for relations in the database, we need to compute them for intermediate results. Note that estimation of the number of sizes and the number of distinct values of attributes in an intermediate result E_i helps us estimate the sizes and number of distinct values of attributes in the next level intermediate results that use E_i.

For selections, the number of distinct values of an attribute (or set of attributes) A in the result of a selection, $V(A, \sigma_\theta(r))$, can be estimated in these ways:

- If the selection condition θ forces A to take on a specified value (e.g., $A = 3$), $V(A, \sigma_\theta(r)) = 1$.
- If θ forces A to take on one of a specified set of values (e.g., $(A = 1 \vee A = 3 \vee A = 4)$), then $V(A, \sigma_\theta(r))$ is set to the number of specified values.
- If the selection condition θ is of the form A *op* v, where *op* is a comparison operator, $V(A, \sigma_\theta(r))$ is estimated to be $V(A, r) * s$, where s is the selectivity of the selection.
- In all other cases of selections, we assume that the distribution of A values is independent of the distribution of the values on which selection conditions are specified, and we use an approximate estimate of $\min(V(A, r), n_{\sigma_\theta(r)})$. A more accurate estimate can be derived for this case using probability theory, but the preceding approximation works fairly well.

For joins, the number of distinct values of an attribute (or set of attributes) A in the result of a join, $V(A, r \bowtie s)$, can be estimated in these ways:

- If all attributes in A are from r, $V(A, r \bowtie s)$ is estimated as $\min(V(A, r), n_{r \bowtie s})$, and similarly if all attributes in A are from s, $V(A, r \bowtie s)$ is estimated to be $\min(V(A, s), n_{r \bowtie s})$.
- If A contains attributes $A1$ from r and $A2$ from s, then $V(A, r \bowtie s)$ is estimated as:

$$\min(V(A1, r) * V(A2 - A1, s), V(A1 - A2, r) * V(A2, s), n_{r \bowtie s})$$

  Note that some attributes may be in $A1$ as well as in $A2$, and $A1 - A2$ and $A2 - A1$ denote, respectively, attributes in A that are only from r and attributes in A that are only from s. Again, more accurate estimates can be derived by using probability theory, but the above approximations work fairly well.

The estimates of distinct values are straightforward for projections: They are the same in $\Pi_A(r)$ as in r. The same holds for grouping attributes of aggregation. For results of **sum**, **count**, and **average**, we can assume, for simplicity, that all aggregate values are distinct. For **min**(A) and **max**(A), the number of distinct values can be estimated as $\min(V(A, r), V(G, r))$, where G denotes the grouping attributes. We omit details of estimating distinct values for other operations.

16.4 Choice of Evaluation Plans

Generation of expressions is only part of the query-optimization process, since each operation in the expression can be implemented with different algorithms. An evaluation plan defines exactly what algorithm should be used for each operation, and how the execution of the operations should be coordinated.

Given an evaluation plan, we can estimate its cost using statistics estimated by the techniques in Section 16.3 coupled with cost estimates for various algorithms and evaluation methods described in Chapter 15.

A **cost-based optimizer** explores the space of all query-evaluation plans that are equivalent to the given query, and chooses the one with the least estimated cost. We have seen how equivalence rules can be used to generate equivalent plans. However, cost-based optimization with arbitrary equivalence rules is fairly complicated. We first cover a simpler version of cost-based optimization, which involves only join-order and join algorithm selection, in Section 16.4.1. Then, in Section 16.4.2, we briefly sketch how a general-purpose optimizer based on equivalence rules can be built, without going into details.

Exploring the space of all possible plans may be too expensive for complex queries. Most optimizers include heuristics to reduce the cost of query optimization, at the potential risk of not finding the optimal plan. We study some such heuristics in Section 16.4.3.

16.4.1 Cost-Based Join-Order Selection

The most common type of query in SQL consists of a join of a few relations, with join predicates and selections specified in the **where** clause. In this section we consider the problem of choosing the optimal join order for such a query.

For a complex join query, the number of different query plans that are equivalent to the query can be large. As an illustration, consider the expression:

$$r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$$

where the joins are expressed without any ordering. With $n = 3$, there are 12 different join orderings:

$$\begin{array}{llll} r_1 \bowtie (r_2 \bowtie r_3) & r_1 \bowtie (r_3 \bowtie r_2) & (r_2 \bowtie r_3) \bowtie r_1 & (r_3 \bowtie r_2) \bowtie r_1 \\ r_2 \bowtie (r_1 \bowtie r_3) & r_2 \bowtie (r_3 \bowtie r_1) & (r_1 \bowtie r_3) \bowtie r_2 & (r_3 \bowtie r_1) \bowtie r_2 \\ r_3 \bowtie (r_1 \bowtie r_2) & r_3 \bowtie (r_2 \bowtie r_1) & (r_1 \bowtie r_2) \bowtie r_3 & (r_2 \bowtie r_1) \bowtie r_3 \end{array}$$

In general, with n relations, there are $(2(n-1))!/(n-1)!$ different join orders. (We leave the computation of this expression for you to do in Exercise 16.12.) For joins involving small numbers of relations, this number is acceptable; for example, with $n = 5$, the number is 1680. However, as n increases, this number rises quickly. With $n = 7$, the number is 665,280; with $n = 10$, the number is greater than 17.6 billion!

Luckily, it is not necessary to generate all the expressions equivalent to a given expression. For example, suppose we want to find the best join order of the form:

$$(r_1 \bowtie r_2 \bowtie r_3) \bowtie r_4 \bowtie r_5$$

which represents all join orders where r_1, r_2, and r_3 are joined first (in some order), and the result is joined (in some order) with r_4 and r_5. There are 12 different join orders for computing $r_1 \bowtie r_2 \bowtie r_3$, and 12 orders for computing the join of this result with r_4 and r_5. Thus, there appear to be 144 join orders to examine. However, once we have found the best join order for the subset of relations $\{r_1, r_2, r_3\}$, we can use that order for further joins with r_4 and r_5, and we can ignore all costlier join orders of $r_1 \bowtie r_2 \bowtie r_3$. Thus, instead of 144 choices to examine, we need to examine only 12 + 12 choices.

Using this idea, we can develop a *dynamic-programming* algorithm for finding optimal join orders. Dynamic-programming algorithms store results of computations and reuse them, a procedure that can reduce execution time greatly.

We now consider how to find the optimal join order for a set of n relations $S = \{r_1, r_2, \ldots, r_n\}$, where each relation may have selection conditions, and a set of join conditions between the relations r_i is provided. We assume that relations have unique names.

A recursive procedure implementing the dynamic-programming algorithm appears in Figure 16.7 and is invoked as FindBestPlan(S), where S is the set of relations above. The procedure applies selections on individual relations at the earliest possible point,

```
procedure FindBestPlan(S)
    if (bestplan[S].cost ≠ ∞) /* bestplan[S] already computed */
        return bestplan[S]
    if (S contains only 1 relation)
        set bestplan[S].plan and bestplan[S].cost based on the best way of
            accessing S using selection conditions (if any) on S.
    else for each non-empty subset S1 of S such that S1 ≠ S
        P1 = FindBestPlan(S1)
        P2 = FindBestPlan(S − S1)
        for each algorithm A for joining the results of P1 and P2
            // For indexed-nested loops join, the outer relation could be P1 or P2.
            // Similarly for hash-join, the build relation could be P1 or P2.
            // We assume the alternatives are considered as separate algorithms.
            // We assume cost of A does not include cost of reading the inputs.
            if algorithm A is indexed nested loops
                Let P_o and P_i denote the outer and inner inputs of A
                if P_i has a single relation r_i, and r_i has an index on the join
                        attributes
                    plan = "execute P_o.plan; join results of P_o and r_i using A",
                            with any selection condition on P_i performed as
                            part of the join condition
                    cost = P_o.cost + cost of A
                else /* Cannot use indexed nested loops join */
                    cost = ∞
            else
                plan = "execute P1.plan; execute P2.plan;
                            join results of P1 and P2 using A"
                cost = P1.cost + P2.cost + cost of A
            if cost < bestplan[S].cost
                bestplan[S].cost = cost
                bestplan[S].plan = plan
    return bestplan[S]
```

Figure 16.7 Dynamic-programming algorithm for join-order optimization.

that is, when the relations are accessed. It is easiest to understand the procedure assuming that all joins are natural joins, although the procedure works unchanged with any join condition. With arbitrary join conditions, the join of two subexpressions is understood to include all join conditions that relate attributes from the two subexpressions.

The procedure stores the evaluation plans it computes in an associative array *bestplan*, which is indexed by sets of relations. Each element of the associative array contains two components: the cost of the best plan of S, and the plan itself. The value of $bestplan[S].cost$ is assumed to be initialized to ∞ if $bestplan[S]$ has not yet been computed.

The procedure first checks if the best plan for computing the join of the given set of relations S has been computed already (and stored in the associative array *bestplan*); if so, it returns the already computed plan.

If S contains only one relation, the best way of accessing S (taking selections on S, if any, into account) is recorded in *bestplan*. This may involve using an index to identify tuples, and then fetching the tuples (often referred to as an *index scan*), or scanning the entire relation (often referred to as a *relation scan*).[2] If there is any selection condition on S, other than those ensured by an index scan, a selection operation is added to the plan to ensure all selections on S are satisfied.

Otherwise, if S contains more than one relation, the procedure tries every way of dividing S into two disjoint subsets. For each division, the procedure recursively finds the best plans for each of the two subsets. It then considers all possible algorithms for joining the results of the two subsets. Note that since indexed nested loops join can potentially use either input $P1$ or $P2$ as the inner input, we consider the two alternatives as two different algorithms. The choice of build versus probe input also leads us to consider the two choices for hash join as two different algorithms.

The cost of each alternative is considered, and the least cost option chosen. The join cost considered should not include the cost of reading the inputs, since we assume that the input is pipelined from the preceding operators, which could be a relation/index scan, or a preceding join. Recall that some operators, such as hash join, can be treated as having suboperators with a blocking (materialized) edge between them, but with the input and output edges of the join being pipelined. The join cost formulae that we saw in Chapter 15 can be used with appropriate modifications to ignore the cost of reading the input relations. Note that indexed nested loops join is treated differently from other join techniques: the plan as well as the cost are different in this case, since we do not perform a relation/index scan of the inner input, and the index lookup cost is included in the cost of indexed nested loops join.

The procedure picks the cheapest plan from among all the alternatives for dividing S into two sets and the algorithms for joining the results of the two sets. The cheapest plan and its cost are stored in the array *bestplan* and returned by the procedure. The time complexity of the procedure can be shown to be $O(3^n)$ (see Practice Exercise 16.13).

The order in which tuples are generated by the join of a set of relations is important for finding the best overall join order, since it can affect the cost of further joins. For

[2] If an index contains all the attributes of a relation that are used in a query, it is possible to perform an *index-only scan*, which retrieves the required attribute values from the index, without fetching actual tuples.

instance, if merge join is used, a potentially expensive sort operation is required on the input, unless the input is already sorted on the join attribute.

A particular sort order of the tuples is said to be an **interesting sort order** if it could be useful for a later operation. For instance, generating the result of $r_1 \bowtie r_2 \bowtie r_3$ sorted on the attributes common with r_4 or r_5 may be useful, but generating it sorted on the attributes common to only r_1 and r_2 is not useful. Using merge join for computing $r_1 \bowtie r_2 \bowtie r_3$ may be costlier than using some other join technique, but it may provide an output sorted in an interesting sort order.

Hence, it is not sufficient to find the best join order for each subset of the set of n given relations. Instead, we have to find the best join order for each subset, for each interesting sort order of the join result for that subset. The *bestplan* array can now be indexed by $[S, o]$, where S is a set of relations, and o is an interesting sort order. The FindBestPlan function can then be modified to take interesting sort orders into consideration; we leave details as an exercise for you (see Practice Exercise 16.11).

The number of subsets of n relations is 2^n. The number of interesting sort orders is generally not large. Thus, about 2^n join expressions need to be stored. The dynamic-programming algorithm for finding the best join order can be extended to handle sort orders. Specifically, when considering sort-merge join, the cost of sorting has to be added if an input (which may be a relation, or the result of a join operation) is not sorted on the join attribute, but is not added if it is sorted.

The cost of the extended algorithm depends on the number of interesting orders for each subset of relations; since this number has been found to be small in practice, the cost remains at $O(3^n)$. With $n = 10$, this number is around 59,000, which is much better than the 17.6 billion different join orders. More important, the storage required is much less than before, since we need to store only one join order for each interesting sort order of each of 1024 subsets of $r_1, \ldots, r_{10}$. Although both numbers still increase rapidly with n, commonly occurring joins usually have less than 10 relations and can be handled easily.

The code shown in Figure 16.7 actually considers each possible way of dividing S into two disjoint subsets twice, since each of the two subsets can play the role of $S1$. Considering the division twice does not affect correctness, but wastes time. The code can be optimized as follows: find the alphabetically smallest relation r_i in $S1$, and the alphabetically smallest relation r_j in $S - S1$, and execute the loop only if $r_i < r_j$. Doing so ensures that each division is considered only once.

Further, the code also considers all possible join orders, including those that contain Cartesian products; for example, if two relations r_1 and r_3 do not have any join condition linking the two relations, the code will still consider $S = \{r_1, r_3\}$, which will result in a Cartesian product. It is possible to take join conditions into account, and modify the code to only generate divisions that do not result in Cartesian products. This optimization can save a great deal of time for many queries. See the Further Reading section at the end of the chapter for references providing more details on Cartesian-product-free join order enumeration.

16.4.2 Cost-Based Optimization with Equivalence Rules

The join-order optimization technique we just saw handles the most common class of queries, which perform an inner join of a set of relations. However, many queries use other features, such as aggregation, outer join, and nested queries, which are not addressed by join-order selection, but can be handled by using equivalence rules.

In this section we outline how to create a general-purpose cost-based optimizer based on equivalence rules. Equivalence rules can help explore alternatives with a wide variety of operations, such as outer joins, aggregations, and set operations, as we have seen earlier. Equivalence rules can be added if required for further operations, such as operators that return the top-K results in sorted order.

In Section 16.2.4, we saw how an optimizer could systematically generate all expressions equivalent to the given query. The procedure for generating equivalent expressions can be modified to generate all possible evaluation plans as follows: A new class of equivalence rules, called **physical equivalence rules**, is added that allows a logical operation, such as a join, to be transformed to a physical operation, such as a hash join, or a nested-loops join. By adding such rules to the original set of equivalence rules, the procedure can generate all possible evaluation plans. The cost estimation techniques we have seen earlier can then be used to choose the optimal (i.e., the least-cost) plan.

However, the procedure shown in Section 16.2.4 is very expensive, even if we do not consider generation of evaluation plans. To make the approach work efficiently requires the following:

1. A space-efficient representation of expressions that avoids making multiple copies of the same subexpressions when equivalence rules are applied.
2. E fficient techniques for detecting duplicate derivations of the same expression.
3. A form of dynamic programming based on **memoization**, which stores the optimal query evaluation plan for a subexpression when it is optimized for the first time; subsequent requests to optimize the same subexpression are handled by returning the already memorized plan.
4. Techniques that avoid generating all possible equivalent plans by keeping track of the cheapest plan generated for any subexpression up to any point of time, and pruning away any plan that is more expensive than the cheapest plan found so far for that subexpression.

The details are more complex than we wish to deal with here. This approach was pioneered by the Volcano research project, and the query optimizer of SQL Server is based on this approach. See the bibliographical notes for references containing further information.

16.4.3 Heuristics in Optimization

A drawback of cost-based optimization is the cost of optimization itself. Although the cost of query optimization can be reduced by clever algorithms, the number of different

evaluation plans for a query can be very large, and finding the optimal plan from this set requires a lot of computational effort. Hence, optimizers use **heuristics** to reduce the cost of optimization.

An example of a heuristic rule is the following rule for transforming relational-algebra queries:

- Perform selection operations as early as possible.

A heuristic optimizer would use this rule without finding out whether the cost is reduced by this transformation. In the first transformation example in Section 16.2, the selection operation was pushed into a join.

We say that the preceding rule is a heuristic because it usually, but not always, helps to reduce the cost. For an example of where it can result in an increase in cost, consider an expression $\sigma_\theta(r \bowtie s)$, where the condition θ refers to only attributes in s. The selection can certainly be performed before the join. However, if r is extremely small compared to s, and if there is an index on the join attributes of s, but no index on the attributes used by θ, then it is probably a bad idea to perform the selection early. Performing the selection early—that is, directly on s—would require doing a scan of all tuples in s. It is probably cheaper, in this case, to compute the join by using the index and then to reject tuples that fail the selection. (This case is specifically handled by the dynamic programming algorithm for join order optimization.)

The projection operation, like the selection operation, reduces the size of relations. Thus, whenever we need to generate a temporary relation, it is advantageous to apply immediately any projections that are possible. This advantage suggests a companion to the "perform selections early" heuristic:

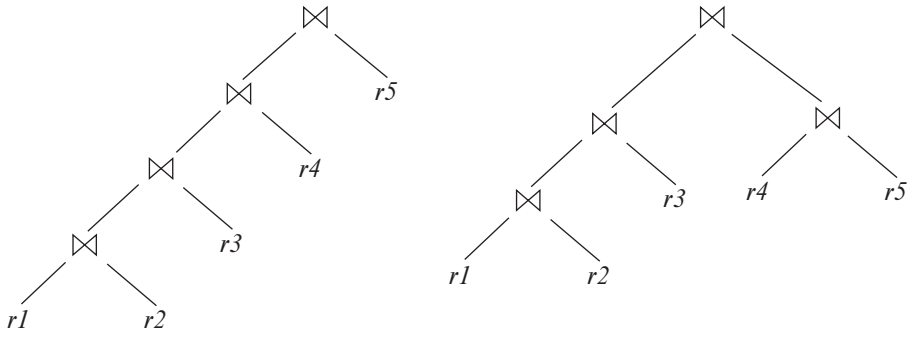
- Perform projections early.

It is usually better to perform selections earlier than projections, since selections have the potential to reduce the sizes of relations greatly, and selections enable the use of indices to access tuples. An example similar to the one used for the selection heuristic should convince you that this heuristic does not always reduce the cost.

Optimizers based on join-order enumeration typically use heuristic transformations to handle constructs other than joins, and applying the cost-based join-order selection algorithm to subexpressions involving only joins and selections. Details of such heuristics are for the most part specific to individual optimizers, and we do not cover them.

Most practical query optimizers have further heuristics to reduce the cost of optimization. For example, many query optimizers, such as the System R optimizer,[3] do not consider all join orders, but rather restrict the search to particular kinds of join or-

[3]System R was one of the first implementations of SQL, and its optimizer pioneered the idea of cost-based join-order optimization.



(a) Left-deep join tree (b) Non-left-deep join tree

Figure 16.8 Left-deep join trees.

ders. The System R optimizer considers only those join orders where the right operand of each join is one of the initial relations $r_1, \ldots, r_n$. Such join orders are called **left-deep join orders**. Left-deep join orders are particularly convenient for pipelined evaluation, since the right operand is a stored relation, and thus only one input to each join is pipelined.

Figure 16.8 illustrates the difference between left-deep join trees and non-left-deep join trees. The time it takes to consider all left-deep join orders is $O(n!)$, which is much less than the time to consider all join orders. With the use of dynamic-programming optimizations, the System R optimizer can find the best join order in time $O(n2^n)$. Contrast this cost with the $O(3^n)$ time required to find the best overall join order. The System R optimizer uses heuristics to push selections and projections down the query tree.

A heuristic approach to reduce the cost of join-order selection, which was originally used in some versions of Oracle, works roughly this way: For an n-way join, it considers n evaluation plans. Each plan uses a left-deep join order, starting with a different one of the n relations. The heuristic constructs the join order for each of the n evaluation plans by repeatedly selecting the "best" relation to join next, on the basis of a ranking of the available access paths. Either nested-loop or sort-merge join is chosen for each of the joins, depending on the available access paths. Finally, the heuristic chooses one of the n evaluation plans in a heuristic manner, on the basis of minimizing the number of nested-loop joins that do not have an index available on the inner relation and on the number of sort-merge joins.

Query-optimization approaches that apply heuristic plan choices for some parts of the query, with cost-based choice based on generation of alternative access plans on other parts of the query, have been adopted in several systems. The approach used in System R and in its successor, the Starburst project, is a hierarchical procedure based on the nested-block concept of SQL. The cost-based optimization techniques described here are used for each block of the query separately. The optimizers in several

database products, such as IBM DB2 and Oracle, are based on the above approach, with extensions to handle other operations such as aggregation. For compound SQL queries (using the ∪, ∩, or − operation), the optimizer processes each component separately and combines the evaluation plans to form the overall evaluation plan.

Most optimizers allow a cost budget to be specified for query optimization. The search for the optimal plan is terminated when the **optimization cost budget** is exceeded, and the best plan found up to that point is returned. The budget itself may be set dynamically; for example, if a cheap plan is found for a query, the budget may be reduced, on the premise that there is no point spending a lot of time optimizing the query if the best plan found so far is already quite cheap. On the other hand, if the best plan found so far is expensive, it makes sense to invest more time in optimization, which could result in a significant reduction in execution time. To best exploit this idea, optimizers usually first apply cheap heuristics to find a plan and then start full cost-based optimization with a budget based on the heuristically chosen plan.

Many applications execute the same query repeatedly, but with different values for the constants. For example, a university application may repeatedly execute a query to find the courses for which a student has registered, but each time for a different student with a different value for the student ID. As a heuristic, many optimizers optimize a query once, with whatever values were provided for the constants when the query was first submitted, and cache the query plan. Whenever the query is executed again, perhaps with new values for constants, the cached query plan is reused (using new values for the constants). The optimal plan for the new constants may differ from the optimal plan for the initial values, but as a heuristic the cached plan is reused.[4] Caching and reuse of query plans is referred to as **plan caching**.

Even with the use of heuristics, cost-based query optimization imposes a substantial overhead on query processing. However, the added cost of cost-based query optimization is usually more than offset by the saving at query-execution time, which is dominated by slow disk accesses. The difference in execution time between a good plan and a bad one may be huge, making query optimization essential. The achieved saving is magnified in those applications that run on a regular basis, where a query can be optimized once, and the selected query plan can be used each time the query is executed. Therefore, most commercial systems include relatively sophisticated optimizers. The bibliographical notes give references to descriptions of the query optimizers of actual database systems.

16.4.4 Optimizing Nested Subqueries

SQL conceptually treats nested subqueries in the **where** clause as functions that take parameters and return either a single value or a set of values (possibly an empty set). The parameters are the variables from an outer-level query that are used in the nested

[4]For the student registration query, the plan would almost certainly be the same for any student ID. But a query that took a range of student IDs, and returned registration information for all student IDs in that range, would probably have a different optimal plan if the range were very small than if the range were large.

subquery (these variables are called **correlation variables**). For instance, suppose we have the following query, to find the names of all instructors who taught a course in 2019:

```
select name
from instructor
where exists (select *
              from teaches
              where instructor.ID = teaches.ID
                    and teaches.year = 2019);
```

Conceptually, the subquery can be viewed as a function that takes a parameter (here, *instructor.ID*) and returns the set of all courses taught in 2019 by instructors (with the same *ID*).

SQL evaluates the overall query (conceptually) by computing the Cartesian product of the relations in the outer **from** clause and then testing the predicates in the **where** clause for each tuple in the product. In the preceding example, the predicate tests if the result of the subquery evaluation is empty. In practice, the predicates in the **where** clause that can be used as join predicates, or as selection predicates are evaluated as part of the selections on relations or to perform joins that avoid Cartesian products. Predicates involving nested subqueries in the **where** clause are evaluated subsequently, since they are usually expensive, by invoking the subquery as a function.

The technique of evaluating a nested subquery by invoking it as a function is called **correlated evaluation**. Correlated evaluation is not very efficient, since the subquery is separately evaluated for each tuple in the outer level query. A large number of random disk I/O operations may result.

SQL optimizers therefore attempt to transform nested subqueries into joins, where possible. Efficient join algorithms help avoid expensive random I/O. Where the transformation is not possible, the optimizer keeps the subqueries as separate expressions, optimizes them separately, and then evaluates them by correlated evaluation.

As an attempt at transforming a nested subquery into a join, the query in the preceding example can be rewritten in relational algebra as a join:

$$\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$$

Unfortunately, the above query is not quite correct, since the multiset versions of the relational algebra operators are used in SQL implementations, and as a result an instructor who teaches multiple sections in 2019 will appear multiple times in the result of the relational algebra query, although that instructor would appear only once in the SQL query result. Using the set version of the relational algebra operators will not help either, since if there are two instructors with the same name who teach in 2019, the name would appear only once with the set version of relational algebra, but would appear twice in the SQL query result. (We note that the set version of relational algebra

would give the correct result if the query output contained the primary key of *instructor*, namely *ID*.)

To properly reflect SQL semantics, the number of duplicates of a tuple in the result should not change because of the rewriting. The semijoin operator of the relational algebra provides a solution to this problem. The multiset version of the **semijoin** operator $r \ltimes_\theta s$ is defined as follows: if a tuple r_i appears n times in r, it appears n times in the result of $r \ltimes_\theta$ if there is at least one tuple s_j such that r_i and s_j together satisfy predicate θ; otherwise r_i does not appear in the result. The set version of the semijoin operator $r \ltimes_\theta s$ can be defined as $\Pi_R(r \bowtie_\theta s)$, where R is the set of attributes in the schema of r. The multiset version of the semijoin operator outputs the same tuples, but the number of duplicates of each tuple r_i in the semijoin result is the same as the number of duplicates of r_i in r.

The preceding SQL query can be translated into the following equivalent relational algebra using the multiset semijoin operator:

$$\Pi_{name}(instructor \ltimes_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$$

The above query in the multiset relational algebra gives the same result as the SQL query, including the counts of duplicates. The query can equivalently be written as:

$$\Pi_{name}(instructor \ltimes_{instructor.ID=teaches.ID} (\sigma_{teaches.year=2019}(teaches)))$$

The following SQL query using the **in** clause is equivalent to the preceding SQL query using the **exists** clause, and can be translated to the same relational algebra expression using semijoin.

```
select name
from instructor
where instructor.ID in (select teaches.ID
                        from teaches
                        where teaches.year = 2019);
```

The anti-semijoin is useful with **not exists** queries. The multiset **anti-semijoin** operator $r \overline{\ltimes}_\theta s$ is defined as follows: if a tuple r_i appears n times in r, it appears n times in the result of $r \ltimes_\theta s$ if there does not exist any tuple s_j in s such that r_i and s_j satisfy predicate θ; otherwise r_i does not appear in the result. The anti-semijoin operator is also known as the **anti-join** operator.

Consider the SQL query:

```
select name
from instructor
where not exists (select *
                  from teaches
                  where instructor.ID = teaches.ID
                        and teaches.year = 2019);
```

The preceding query can be translated into the following relational algebra using the anti-semijoin operator:

$$\Pi_{name}(instructor \overline{\ltimes}_{instructor.ID=teaches.ID}(\sigma_{teaches.year=2019}(teaches)))$$

In general, a query of the form:

```
select A
from r1, r2, ..., rn
where P1 and exists (select *
                     from s1, s2, ..., sm
                     where P2^1 and P2^2);
```

where P_2^1 are predicates that only reference the relations s_i in the subquery, and P_2^2 predicates that also reference the relations r_i from the outer query, can be translated to:

$$\Pi_A((\sigma_{P_1}(r_1 \times r_2 \times \ldots \times r_n)) \ltimes_{P_2^2} \sigma_{P_2^1}(s_1 \times s_2 \times \ldots \times s_m))$$

If **not exists** were used instead of **exists**, the semijoin should be replaced by anti-semijoin in the relational algebra query. If an **in** clause is used instead of **exists**, the relational algebra query can be appropriately modified by adding a corresponding predicate in the semijoin predicate, as our earlier example illustrated.

The process of replacing a nested query by a query with a join, semijoin, or anti-semijoin is called **decorrelation**. The semijoin and anti-semijoin operators can be efficiently implemented using modifications of the join algorithms, as explored in Practice Exercise 15.10.

Consider the following query with aggregation in a scalar subquery, that finds instructors who have taught more than one course section in 2019.

```
select name
from instructor
where 1 < (select count(*)
           from teaches
           where instructor.ID = teaches.ID
                 and teaches.year = 2019);
```

The above query can be rewritten using a semijoin as follows:

$$\Pi_{name}(instructor \ltimes_{(instructor.ID=TID)\wedge(1<cnt)}({}_{ID\ \mathbf{as}\ TID}\gamma_{\mathbf{count}(*)\ \mathbf{as}\ cnt}(\sigma_{year=2019}(teaches))))$$

Observe that the subquery has a predicate *instructor.ID*= *teaches.ID*, and aggregation without a group by clause. The decorrelated query has the predicate moved into the semijoin condition, and the aggregation is now grouped by *ID*. The predicate 1 < (subquery) has turned into a semijoin predicate. Intuitively, the subquery performs a sep-

arate count for each *instructor.ID*; grouping by *ID* ensures that counts are computed separately for each *ID*.

Decorrelation is clearly more complicated when the nested subquery uses aggregation, or when the nested subquery is used as a scalar subquery. In fact, decorrelation is not possible for certain cases of subqueries. For example, a subquery that is used as a scalar subquery is expected to return only one result; if it returns more than one result, a runtime exception can occur, which is not possible with a decorrelated query. Further, whether to decorrelate or not should ideally be done in a cost-based manner, depending on whether decorrelation reduces the cost or not. Some query optimizers represent nested subqueries using extended relational-algebra constructs, and express transformations of nested subqueries to semijoin, anti-semijoin, and so forth, as equivalence rules. We do not attempt to give algorithms for the general case, and instead refer you to relevant items in the online bibliographical notes.

Optimization of complex nested subqueries is a complicated task, as you can infer from the preceding discussion, and many optimizers do only a limited amount of decorrelation. It better to avoid using complex nested subqueries, where possible, since we cannot be sure that the query optimizer will succeed in converting them to a form that can be evaluated efficiently.

16.5 Materialized Views

When a view is defined, normally the database stores only the query defining the view. In contrast, a **materialized view** is a view whose contents are computed and stored. Materialized views constitute redundant data, in that their contents can be inferred from the view definition and the rest of the database contents. However, it is much cheaper in many cases to read the contents of a materialized view than to compute the contents of the view by executing the query defining the view.

Materialized views are important for improving performance in some applications. Consider this view, which gives the total salary in each department:

```
create view department_total_salary(dept_name, total_salary) as
select dept_name, sum (salary)
from instructor
group by dept_name;
```

Suppose the total salary amount at a department is required frequently. Computing the view requires reading every *instructor* tuple pertaining to a department and summing up the salary amounts, which can be time-consuming. In contrast, if the view definition of the total salary amount were materialized, the total salary amount could be found by looking up a single tuple in the materialized view.[5]

[5]The difference may not be all that large for a medium-sized university, but in other settings the difference can be very large. For example, if the materialized view computed total sales of each product, from a sales relation with tens

16.5.1 View Maintenance

A problem with materialized views is that they must be kept up-to-date when the data used in the view definition changes. For instance, if the *salary* value of an instructor is updated, the materialized view will become inconsistent with the underlying data, and it must be updated. The task of keeping a materialized view up-to-date with the underlying data is known as **view maintenance**.

Views can be maintained by manually written code: That is, every piece of code that updates the *salary* value can be modified to also update the total salary amount for the corresponding department. However, this approach is error prone, since it is easy to miss some places where the *salary* is updated, and the materialized view will then no longer match the underlying data.

Another option for maintaining materialized views is to define triggers on insert, delete, and update of each relation in the view definition. The triggers must modify the contents of the materialized view, to take into account the change that caused the trigger to fire. A simplistic way of doing so is to completely recompute the materialized view on every update.

A better option is to modify only the affected parts of the materialized view, which is known as **incremental view maintenance**. We describe how to perform incremental view maintenance in Section 16.5.2.

Modern database systems provide more direct support for incremental view maintenance. Database-system programmers no longer need to define triggers for view maintenance. Instead, once a view is declared to be materialized, the database system computes the contents of the view and incrementally updates the contents when the underlying data change.

Most database systems perform **immediate view maintenance**; that is, incremental view maintenance is performed as soon as an update occurs, as part of the updating transaction. Some database systems also support **deferred view maintenance**, where view maintenance is deferred to a later time; for example, updates may be collected throughout a day, and materialized views may be updated at night. This approach reduces the overhead on update transactions. However, materialized views with deferred view maintenance may not be consistent with the underlying relations on which they are defined.

16.5.2 Incremental View Maintenance

To understand how to maintain materialized views incrementally, we start off by considering individual operations, and then we see how to handle a complete expression.

The changes to a relation that can cause a materialized view to become out-of-date are inserts, deletes, and updates. To simplify our description, we replace updates to a tuple by deletion of the tuple followed by insertion of the updated tuple. Thus, we need

---

of millions of tuples, the difference between computing the aggregate from the underlying data and looking up the materialized view can be many orders of magnitude.

to consider only inserts and deletes. The changes (inserts and deletes) to a relation or expression are referred to as its **differential**.

16.5.2.1 Join Operation

Consider the materialized view $v = r \bowtie s$. Suppose we modify r by inserting a set of tuples denoted by i_r. If the old value of r is denoted by r^{old}, and the new value of r by r^{new}, $r^{new} = r^{old} \cup i_r$. Now, the old value of the view, v^{old}, is given by $r^{old} \bowtie s$, and the new value v^{new} is given by $r^{new} \bowtie s$. We can rewrite $r^{new} \bowtie s$ as $(r^{old} \cup i_r) \bowtie s$, which we can again rewrite as $(r^{old} \bowtie s) \cup (i_r \bowtie s)$. In other words:

$$v^{new} = v^{old} \cup (i_r \bowtie s)$$

Thus, to update the materialized view v, we simply need to add the tuples $i_r \bowtie s$ to the old contents of the materialized view. Inserts to s are handled in an exactly symmetric fashion.

Now suppose r is modified by deleting a set of tuples denoted by d_r. Using the same reasoning as above, we get:

$$v^{new} = v^{old} - (d_r \bowtie s)$$

Deletes on s are handled in an exactly symmetric fashion.

16.5.2.2 Selection and Projection Operations

Consider a view $v = \sigma_\theta(r)$. If we modify r by inserting a set of tuples i_r, the new value of v can be computed as:

$$v^{new} = v^{old} \cup \sigma_\theta(i_r)$$

Similarly, if r is modified by deleting a set of tuples d_r, the new value of v can be computed as:

$$v^{new} = v^{old} - \sigma_\theta(d_r)$$

Projection is a more difficult operation with which to deal. Consider a materialized view $v = \Pi_A(r)$. Suppose the relation r is on the schema $R = (A, B)$, and r contains two tuples $(a, 2)$ and $(a, 3)$. Then, $\Pi_A(r)$ has a single tuple (a). If we delete the tuple $(a, 2)$ from r, we cannot delete the tuple (a) from $\Pi_A(r)$: If we did so, the result would be an empty relation, whereas in reality $\Pi_A(r)$ still has a single tuple (a). The reason is that the same tuple (a) is derived in two ways, and deleting one tuple from r removes only one of the ways of deriving (a); the other is still present.

This reason also gives us the intuition for a solution: For each tuple in a projection such as $\Pi_A(r)$, we will keep a count of how many times it was derived.

When a set of tuples d_r is deleted from r, for each tuple t in d_r we do the following: Let $t.A$ denote the projection of t on the attribute A. We find $(t.A)$ in the materialized view and decrease the count stored with it by 1. If the count becomes 0, $(t\ A)$ is deleted from the materialized view.

Handling insertions is relatively straightforward. When a set of tuples i_r is inserted into r, for each tuple t in i_r we do the following: If $(t.A)$ is already present in the materialized view, we increase the count stored with it by 1. If not, we add $(t.A)$ to the materialized view, with the count set to 1.

16.5.2.3 Aggregation Operations

Aggregation operations proceed somewhat like projections. The aggregate operations in SQL are **count, sum, avg, min,** and **max**:

- **count**: Consider a materialized view $v = {}_G\gamma_{count(B)}(r)$, which computes the count of the attribute B, after grouping r by attribute G.

  When a set of tuples i_r is inserted into r, for each tuple t in i_r we do the following: We look for the group $t.G$ in the materialized view. If it is not present, we add $(t.G, 1)$ to the materialized view. If the group $t.G$ is present, we add 1 to the count of the group.

  When a set of tuples d_r is deleted from r, for each tuple t in d_r we do the following: We look for the group $t.G$ in the materialized view and subtract 1 from the count for the group. If the count becomes 0, we delete the tuple for the group $t.G$ from the materialized view.

- **sum**: Consider a materialized view $v = {}_G\gamma_{sum(B)}(r)$.
  When a set of tuples i_r is inserted into r, for each tuple t in i_r we do the following: We look for the group $t.G$ in the materialized view. If it is not present, we add $(t.G, t.B)$ to the materialized view; in addition, we store a count of 1 associated with $(t.G, t.B)$, just as we did for projection. If the group $t.G$ is present, we add the value of $t.B$ to the aggregate value for the group and add 1 to the count of the group.

  When a set of tuples d_r is deleted from r, for each tuple t in d_r we do the following: We look for the group $t.G$ in the materialized view and subtract $t.B$ from the aggregate value for the group. We also subtract 1 from the count for the group, and if the count becomes 0, we delete the tuple for the group $t.G$ from the materialized view.

  Without keeping the extra count value, we would not be able to distinguish a case where the sum for a group is 0 from the case where the last tuple in a group is deleted.

- **avg**: Consider a materialized view $v = {}_G\gamma_{avg(B)}(r)$.
  Directly updating the average on an insert or delete is not possible, since it depends not only on the old average and the tuple being inserted/deleted, but also on the number of tuples in the group.

Instead, to handle the case of **avg**, we maintain the **sum** and **count** aggregate values as described earlier and compute the average as the sum divided by the count.

- **min, max**: Consider a materialized view $v = {}_G\gamma_{min(B)}(r)$. (The case of **max** is exactly equivalent.)

  Handling insertions on r is straightforward, similar to the case of **sum**. Maintaining the aggregate values **min** and **max** on deletions may be more expensive. For example, if the tuple t corresponding to the minimum value for a group is deleted from r, we have to look at the other tuples of r that are in the same group to find the new minimum value. It is a good idea to create an ordered index on (G, B) since it would help us to find the new minimum value for a group very efficiently.

16.5.2.4 Other Operations

The set operation *intersection* is maintained as follows: Given materialized view $v = r \cap s$, when a tuple is inserted in r we check if it is present in s, and if so we add it to v. If a tuple is deleted from r, we delete it from the intersection if it is present. The other set operations, *union* and *set difference*, are handled in a similar fashion; we leave details to you.

Outer joins are handled in much the same way as joins, but with some extra work. In the case of deletion from r we have to handle tuples in s that no longer match any tuple in r. In the case of insertion to r, we have to handle tuples in s that did not match any tuple in r. Again we leave details to you.

16.5.2.5 Handling Expressions

So far we have seen how to update incrementally the result of a single operation. To handle an entire expression, we can derive expressions for computing the incremental change to the result of each subexpression, starting from the smallest subexpressions.

For example, suppose we wish to incrementally update a materialized view $E_1 \bowtie E_2$ when a set of tuples i_r is inserted into relation r. Let us assume r is used in E_1 alone. Suppose the set of tuples to be inserted into E_1 is given by expression D_1. Then the expression $D_1 \bowtie E_2$ gives the set of tuples to be inserted into $E_1 \bowtie E_2$.

See the online bibliographical notes for further details on incremental view maintenance with expressions.

16.5.3 Query Optimization and Materialized Views

Query optimization can be performed by treating materialized views just like regular relations. However, materialized views offer further opportunities for optimization:

- Rewriting queries to use materialized views:

  Suppose a materialized view $v = r \bowtie s$ is available, and a user submits a query $r \bowtie s \bowtie t$. Rewriting the query as $v \bowtie t$ may provide a more efficient query plan

than optimizing the query as submitted. Thus, it is the job of the query optimizer to recognize when a materialized view can be used to speed up a query.

- Replacing a use of a materialized view with the view definition:
  Suppose a materialized view $v = r \bowtie s$ is available, but without any index on it, and a user submits a query $\sigma_{A=10}(v)$. Suppose also that s has an index on the common attribute B, and r has an index on attribute A. The best plan for this query may be to replace v with $r \bowtie s$, which can lead to the query plan $\sigma_{A=10}(r) \bowtie s$; the selection and join can be performed efficiently by using the indices on $r.A$ and $s.B$, respectively. In contrast, evaluating the selection directly on v may require a full scan of v, which may be more expensive.

The online bibliographical notes give pointers to research showing how to perform query optimization efficiently with materialized views.

16.5.4 Materialized View and Index Selection

Another related optimization problem is that of **materialized view selection**, namely, "What is the best set of views to materialize?" This decision must be made on the basis of the system **workload**, which is a sequence of queries and updates that reflects the typical load on the system. One simple criterion would be to select a set of materialized views that minimizes the overall execution time of the workload of queries and updates, including the time taken to maintain the materialized views. Database administrators usually modify this criterion to take into account the importance of different queries and updates: Fast response may be required for some queries and updates, but a slow response may be acceptable for others.

Indices are just like materialized views, in that they too are derived data, can speed up queries, and may slow down updates. Thus, the problem of **index selection** is closely related to that of materialized view selection, although it is simpler. We examine index and materialized view selection in more detail in Section 25.1.4.1 and Section 25.1.4.2.

Most database systems provide tools to help the database administrator with index and materialized view selection. These tools examine the history of queries and updates and suggest indices and views to be materialized. The Microsoft SQL Server Database Tuning Assistant, the IBM DB2 Design Advisor, and the Oracle SQL Tuning Wizard are examples of such tools.

16.6 Advanced Topics in Query Optimization

There are a number of opportunities for optimizing queries, beyond those we have seen so far. We examine a few of these in this section.

16.6.1 Top-*K* Optimization

Many queries fetch results sorted on some attributes, and require only the top K results for some K. Sometimes the bound K is specified explicitly. For example, some databases support a **limit** K clause which results in only the top K results being returned by the query. Other databases support alternative ways of specifying similar limits. In other cases, the query may not specify such a limit, but the optimizer may allow a hint to be specified, indicating that only the top K results of the query are likely to be retrieved, even if the query generates more results.

When K is small, a query optimization plan that generates the entire set of results, then sorts and generates the top K, is very inefficient since it discards most of the intermediate results that it computes. Several techniques have been proposed to optimize such *top-K queries*. One approach is to use pipelined plans that can generate the results in sorted order. Another approach is to estimate what is the highest value on the sorted attributes that will appear in the top-K output, and introduce selection predicates that eliminate larger values. If extra tuples beyond the top-K are generated they are discarded, and if too few tuples are generated then the selection condition is changed and the query is re-executed. See the bibliographical notes for references to work on top-K optimization.

16.6.2 Join Minimization

When queries are generated through views, sometimes more relations are joined than are needed for computation of the query. For example, a view v may include the join of *instructor* and *department*, but a use of the view v may use only attributes from *instructor*. The join attribute *dept_name* of *instructor* is a foreign key referencing *department*. Assuming that *instructor.dept_name* has been declared **not null**, the join with *department* can be dropped, with no impact on the query. For under the above assumption, the join with *department* does not eliminate any tuples from *instructor*, nor does it result in extra copies of any *instructor* tuple.

Dropping a relation from a join as above is an example of join minimization. In fact, join minimization can be performed in other situations as well. See the bibliographical notes for references on join minimization.

16.6.3 Optimization of Updates

Update queries often involve subqueries in the **set** and **where** clauses, which must also be taken into account in optimizing the update. Updates that involve a selection on the updated column (e.g., give a 10 percent salary raise to all employees whose salary is $\geq$ \$100,000) must be handled carefully. If the update is done while the selection is being evaluated by an index scan, an updated tuple may be reinserted in the index ahead of the scan and seen again by the scan; the same employee tuple may then get incorrectly updated multiple times (an infinite number of times, in this case). A similar problem also arises with updates involving subqueries whose result is affected by the update.

The problem of an update affecting the execution of a query associated with the update is known as the **Halloween problem** (named so because it was first recognized on a Halloween day, at IBM). The problem can be avoided by executing the queries defining the update first, creating a list of affected tuples, and updating the tuples and indices as the last step. However, breaking up the execution plan in such a fashion increases the execution cost. Update plans can be optimized by checking if the Halloween problem can occur, and if it cannot occur, updates can be performed while the query is being processed, reducing the update overheads. For example, the Halloween problem cannot occur if the update does not affect index attributes. Even if it does, if the updates decrease the value while the index is scanned in increasing order, updated tuples will not be encountered again during the scan. In such cases, the index can be updated even while the query is being executed, reducing the overall cost.

Update queries that result in a large number of updates can also be optimized by collecting the updates as a batch and then applying the batch of updates separately to each affected index. When applying the batch of updates to an index, the batch is first sorted in the index order for that index; such sorting can greatly reduce the amount of random I/O required for updating indices.

Such optimizations of updates are implemented in most database systems. See the bibliographical notes for references to such optimization.

16.6.4 Multiquery Optimization and Shared Scans

When a batch of queries are submitted together, a query optimizer can potentially exploit common subexpressions between the different queries, evaluating them once and reusing them where required. Complex queries may in fact have subexpressions repeated in different parts of the query, which can be similarly exploited to reduce query evaluation cost. Such optimization is known as **multiquery optimization**.

Common subexpression elimination optimizes subexpressions shared by different expressions in a program by computing and storing the result and reusing it wherever the subexpression occurs. Common subexpression elimination is a standard optimization applied on arithmetic expressions by programming-language compilers. Exploiting common subexpressions among evaluation plans chosen for each of a batch of queries is just as useful in database query evaluation, and is implemented by some databases. However, multiquery optimization can do even better in some cases: A query typically has more than one evaluation plan, and a judiciously chosen set of query evaluation plans for the queries may provide for a greater sharing and lesser cost than that afforded by choosing the lowest cost evaluation plan for each query. More details on multiquery optimization may be found in references cited in the bibliographical notes.

Sharing of relation scans between queries is another limited form of multiquery optimization that is implemented in some databases. The **shared-scan** optimization works as follows: Instead of reading the relation repeatedly from disk, once for each query that needs to scan a relation, data are read once from disk, and pipelined to each of

the queries. The shared-scan optimization is particularly useful when multiple queries perform a scan on a single large relation (typically a "fact table").

16.6.5 Parametric Query Optimization

Plan caching, which we saw in Section 16.4.3, is used as a heuristic in many databases. Recall that with plan caching, if a query is invoked with some constants, the plan chosen by the optimizer is cached and reused if the query is submitted again, even if the constants in the query are different. For example, suppose a query takes a department name as a parameter and retrieves all courses of the department. With plan caching, a plan chosen when the query is executed for the first time, say for the Music department, is reused if the query is executed for any other department.

Such reuse of plans by plan caching is reasonable if the optimal query plan is not significantly affected by the exact value of the constants in the query. However, if the plan is affected by the value of the constants, parametric query optimization is an alternative.

In **parametric query optimization**, a query is optimized without being provided specific values for its parameters—for example, *dept_name* in the preceding example. The optimizer then outputs several plans, each optimal for a different parameter value. A plan would be output by the optimizer only if it is optimal for some possible value of the parameters. The set of alternative plans output by the optimizer are stored. When a query is submitted with specific values for its parameters, instead of performing a full optimization, the cheapest plan from the set of alternative plans computed earlier is used. Finding the cheapest such plan usually takes much less time than reoptimization. See the bibliographical notes for references on parametric query optimization.

16.6.6 Adaptive Query Processing

As we noted earlier, query optimization is based on estimates that are at best approximations. Thus, it is possible at times for the optimizer to choose a plan that turns out to perform very badly. Adaptive operators that choose the specific operator at execution time provide a partial solution to this problem. For example, SQL Server supports an adaptive join algorithm that checks the size of its outer input, and chooses either nested loops join, or hash join depending on the size of the outer input.

Many systems also include the ability to monitor the behavior of a plan during query execution, and adapt the plan accordingly. For example, suppose the statistics collected by the system during early stages of the plan's execution (or the execution of subparts of the plan) are found to differ substantially from the optimizers estimates to such an extent that it is clear that the chosen plan is suboptimal. Then an adaptive system may abort the execution, choose a new query execution plan using the statistics collected during the initial execution, and restart execution using the new plan; the statistics collected during the execution of the old plan ensure the old plan is not selected again. Further, the system must avoid repeated aborts and restarts; ideally, the system should ensure that the overall cost of query evaluation is close to that with the

plan that would be chosen if the optimizer had exact statistics. The specific criteria and mechanisms for such adaptive query processing are complex, and are referenced in the bibliographic notes available online.

16.7 Summary

- Given a query, there are generally a variety of methods for computing the answer. It is the responsibility of the system to transform the query as entered by the user into an equivalent query that can be computed more efficiently. The process of finding a good strategy for processing a query is called *query optimization*.
- The evaluation of complex queries involves many accesses to disk. Since the transfer of data from disk is slow relative to the speed of main memory and the CPU of the computer system, it is worthwhile to allocate a considerable amount of processing to choose a method that minimizes disk accesses.
- There are a number of equivalence rules that we can use to transform an expression into an equivalent one. We use these rules to generate systematically all expressions equivalent to the given query.
- Each relational-algebra expression represents a particular sequence of operations. The first step in selecting a query-processing strategy is to find a relational-algebra expression that is equivalent to the given expression and is estimated to cost less to execute.
- The strategy that the database system chooses for evaluating an operation depends on the size of each relation and on the distribution of values within columns. So that they can base the strategy choice on reliable information, database systems may store statistics for each relation r. These statistics include:
  - The number of tuples in the relation r.
  - The size of a record (tuple) of relation r in bytes.
  - The number of distinct values that appear in the relation r for a particular attribute.
- Most database systems use histograms to store the number of values for an attribute within each of several ranges of values. Histograms are often computed using sampling.
- These statistics allow us to estimate the sizes of the results of various operations, as well as the cost of executing the operations. Statistical information about relations is particularly useful when several indices are available to assist in the processing of a query. The presence of these structures has a significant influence on the choice of a query-processing strategy.

- Alternative evaluation plans for each expression can be generated by equivalence rules, and the cheapest plan across all expressions can be chosen. Several optimization techniques are available to reduce the number of alternative expressions and plans that need to be generated.
- We use heuristics to reduce the number of plans considered, and thereby to reduce the cost of optimization. Heuristic rules for transforming relational-algebra queries include "Perform selection operations as early as possible," "Perform projections early," and "Avoid Cartesian products."
- Materialized views can be used to speed up query processing. Incremental view maintenance is needed to efficiently update materialized views when the underlying relations are modified. The differential of an operation can be computed by means of algebraic expressions involving differentials of the inputs of the operation. Other issues related to materialized views include how to optimize queries by making use of available materialized views, and how to select views to be materialized.
- A number of advanced optimization techniques have been proposed, such as top-K optimization, join minimization, optimization of updates, multiquery optimization, and parametric query optimization.

Review Terms

- Query optimization
- Transformation of expressions
- Equivalence of expressions
- Equivalence rules
  - Join commutativity
  - Join associativity
- Minimal set of equivalence rules
- Enumeration of equivalent expressions
- Statistics estimation
- Catalog information
- Size estimation
  - Selection
  - Selectivity
  - Join
- Histograms
- Distinct value estimation
- Random sample
- Choice of evaluation plans
- Interaction of evaluation techniques
- Cost-based optimization
- Join-order optimization
  - Dynamic-programming algorithm
  - Left-deep join order
  - Interesting sort order
- Heuristic optimization
- Plan caching
- Access-plan selection

- Correlated evaluation
- Decorrelation
- Semijoin
- Anti-semijoin
- Materialized views
- Materialized view maintenance
  - Recomputation
  - Incremental maintenance
  - Insertion
  - Deletion
  - Updates
- Query optimization with materialized views
- Index selection
- Materialized view selection
- Top-K optimization
- Join minimization
- Halloween problem
- Multiquery optimization

Practice Exercises

16.1 Download the university database schema and the large university dataset from dbbook.com. Create the university schema on your favorite database, and load the large university dataset. Use the **explain** feature described in Note 16.1 on page 746 to view the plan chosen by the database, in different cases as detailed below.

a. Write a query with an equality condition on *student.name* (which does not have an index), and view the plan chosen.

b. Create an index on the attribute *student.name*, and view the plan chosen for the above query.

c. Create simple queries joining two relations, or three relations, and view the plans chosen.

d. Create a query that computes an aggregate with grouping, and view the plan chosen.

e. Create an SQL query whose chosen plan uses a semijoin operation.

f. Create an SQL query that uses a **not in** clause, with a subquery using aggregation. Observe what plan is chosen.

g. Create a query for which the chosen plan uses correlated evaluation (the way correlated evaluation is represented varies by database, but most databases would show a filter or a project operator with a subplan or subquery).

h. Create an SQL update query that updates a single row in a relation. View the plan chosen for the update query.

i. Create an SQL update query that updates a large number of rows in a relation, using a subquery to compute the new value. View the plan chosen for the update query.

16.2 Show that the following equivalences hold. Explain how you can apply them to improve the efficiency of certain queries:

a. $E_1 \bowtie_\theta (E_2 - E_3) \equiv (E_1 \bowtie_\theta E_2 - E_1 \bowtie_\theta E_3)$.

b. $\sigma_\theta({}_A\gamma_F(E)) \equiv {}_A\gamma_F(\sigma_\theta(E))$, where θ uses only attributes from A.

c. $\sigma_\theta(E_1 \rtimes E_2) \equiv \sigma_\theta(E_1) \rtimes E_2$, where θ uses only attributes from E_1.

16.3 For each of the following pairs of expressions, give instances of relations that show the expressions are not equivalent.

a. $\Pi_A(r - s)$ and $\Pi_A(r) - \Pi_A(s)$.

b. $\sigma_{B<4}({}_A\gamma_{max(B)\ \textbf{as}\ B}(r))$ and ${}_A\gamma_{max(B)\ \textbf{as}\ B}(\sigma_{B<4}(r))$.

c. In the preceding expressions, if both occurrences of *max* were replaced by *min*, would the expressions be equivalent?

d. $(r \bowtie s) \bowtie t$ and $r \bowtie (s \bowtie t)$
In other words, the natural right outer join is not associative.

e. $\sigma_\theta(E_1 \rtimes E_2)$ and $E_1 \rtimes \sigma_\theta(E_2)$, where θ uses only attributes from E_2.

16.4 SQL allows relations with duplicates (Chapter 3), and the multiset version of the relational algebra is defined in Note 3.1 on page 80, Note 3.2 on page 97, and Note 3.3 on page 108. Check which of the equivalence rules 1 through 7.b hold for the multiset version of the relational algebra.

16.5 Consider the relations $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$, with primary keys A, C, and E, respectively. Assume that r_1 has 1000 tuples, r_2 has 1500 tuples, and r_3 has 750 tuples. Estimate the size of $r_1 \bowtie r_2 \bowtie r_3$, and give an efficient strategy for computing the join.

16.6 Consider the relations $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$ of Practice Exercise 16.5. Assume that there are no primary keys, except the entire schema. Let $V(C, r_1)$ be 900, $V(C, r_2)$ be 1100, $V(E, r_2)$ be 50, and $V(E, r_3)$ be 100. Assume that r_1 has 1000 tuples, r_2 has 1500 tuples, and r_3 has 750 tuples. Estimate the size of $r_1 \bowtie r_2 \bowtie r_3$ and give an efficient strategy for computing the join.

16.7 Suppose that a B^+-tree index on *building* is available on relation *department* and that no other index is available. What would be the best way to handle the following selections that involve negation?

a. $\sigma_{\neg(building < \text{“Watson”})}(department)$

b. $\sigma_{\neg(building = \text{“Watson”})}(department)$

c. $\sigma_{\neg(building < \text{“Watson”} \vee budget < 50000)}(department)$

16.8 Consider the query:

```
select *
from r, s
where upper(r.A) = upper(s.A);
```

where “upper” is a function that returns its input argument with all lowercase letters replaced by the corresponding uppercase letters.

a. Find out what plan is generated for this query on the database system you use.

b. Some database systems would use a (block) nested-loop join for this query, which can be very inefficient. Briefly explain how hash-join or merge-join can be used for this query.

16.9 Give conditions under which the following expressions are equivalent:

$$_{A,B}\gamma_{agg(C)}(E_1 \bowtie E_2) \quad \text{and} \quad (_{A}\gamma_{agg(C)}(E_1)) \bowtie E_2$$

where *agg* denotes any aggregation operation. How can the above conditions be relaxed if *agg* is one of **min** or **max**?

16.10 Consider the issue of interesting orders in optimization. Suppose you are given a query that computes the natural join of a set of relations S. Given a subset $S1$ of S, what are the interesting orders of $S1$?

16.11 Modify the FindBestPlan(S) function to create a function FindBestPlan(S, O), where O is a desired sort order for S, and which considers interesting sort orders. A *null* order indicates that the order is not relevant. *Hints*: An algorithm A may give the desired order O; if not a sort operation may need to be added to get the desired order. If A is a merge-join, FindBestPlan must be invoked on the two inputs with the desired orders for the inputs.

16.12 Show that, with n relations, there are $(2(n-1))!/(n-1)!$ different join orders. *Hint:* A **complete binary tree** is one where every internal node has exactly two children. Use the fact that the number of different complete binary trees with n leaf nodes is:

$$\frac{1}{n}\binom{2(n-1)}{(n-1)}$$

If you wish, you can derive the formula for the number of complete binary trees with n nodes from the formula for the number of binary trees with n nodes. The number of binary trees with n nodes is:

$$\frac{1}{n+1}\binom{2n}{n}$$

This number is known as the **Catalan number**, and its derivation can be found in any standard textbook on data structures or algorithms.

16.13 Show that the lowest-cost join order can be computed in time $O(3^n)$. Assume that you can store and look up information about a set of relations (such as the optimal join order for the set, and the cost of that join order) in constant time. (If you find this exercise difficult, at least show the looser time bound of $O(2^{2n})$.)

16.14 Show that, if only left-deep join trees are considered, as in the System R optimizer, the time taken to find the most efficient join order is around $n2^n$. Assume that there is only one interesting sort order.

16.15 Consider the bank database of Figure 16.9, where the primary keys are underlined. Construct the following SQL queries for this relational database.

a. Write a nested query on the relation *account* to find, for each branch with name starting with B, all accounts with the maximum balance at the branch.

b. Rewrite the preceding query without using a nested subquery; in other words, decorrelate the query, but in SQL.

c. Give a relational algebra expression using semijoin equivalent to the query.

d. Give a procedure (similar to that described in Section 16.4.4) for decorrelating such queries.

Exercises

16.16 Suppose that a B^+-tree index on (*dept_name*, *building*) is available on relation *department*. What would be the best way to handle the following selection?

$$\sigma_{(building < \text{“Watson”}) \wedge (budget < 55000) \wedge (dept\_name = \text{“Music”})}(department)$$

---

branch(*branch_name*, *branch_city, assets*)
customer (*customer_name*, *customer_street, customer_city*)
loan (*loan_number*, *branch_name, amount*)
borrower (*customer_name*, *loan_number*)
account (*account_number*, *branch_name, balance*)
depositor (*customer_name*, *account_number*)

Figure 16.9 Banking database.

16.17 Show how to derive the following equivalences by a sequence of transformations using the equivalence rules in Section 16.2.1.

a. $\sigma_{\theta_1 \wedge \theta_2 \wedge \theta_3}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(\sigma_{\theta_3}(E)))$

b. $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta_3} E_2) \equiv \sigma_{\theta_1}(E_1 \bowtie_{\theta_3} (\sigma_{\theta_2}(E_2)))$, where θ_2 involves only attributes from E_2

16.18 Consider the two expressions $\sigma_\theta(E_1 \rtimes E_2)$ and $\sigma_\theta(E_1 \bowtie E_2)$.

a. Show using an example that the two expressions are not equivalent in general.

b. Give a simple condition on the predicate θ, which if satisfied will ensure that the two expressions are equivalent.

16.19 A set of equivalence rules is said to be *complete* if, whenever two expressions are equivalent, one can be derived from the other by a sequence of uses of the equivalence rules. Is the set of equivalence rules that we considered in Section 16.2.1 complete? Hint: Consider the equivalence $\sigma_{3=5}(r) \equiv \{\ \}$.

16.20 Explain how to use a histogram to estimate the size of a selection of the form $\sigma_{A \le v}(r)$.

16.21 Suppose two relations r and s have histograms on attributes $r.A$ and $s.A$, respectively, but with different ranges. Suggest how to use the histograms to estimate the size of $r \bowtie s$. Hint: Split the ranges of each histogram further.

16.22 Consider the query

```
select A, B
from   r
where r.B < some (select B
                  from   s
                  where s.A = r.A)
```

Show how to decorrelate this query using the multiset version of the semi join operation.

16.23 Describe how to incrementally maintain the results of the following operations on both insertions and deletions:

a. Union and set difference.

b. Left outer join.

16.24 Give an example of an expression defining a materialized view and two situations (sets of statistics for the input relations and the differentials) such that incremental view maintenance is better than recomputation in one situation, and recomputation is better in the other situation.

16.25 Suppose you want to get answers to $r \bowtie s$ sorted on an attribute of r, and want only the top K answers for some relatively small K. Give a good way of evaluating the query:

a. When the join is on a foreign key of r referencing s, where the foreign key attribute is declared to be not null.

b. When the join is not on a foreign key.

16.26 Consider a relation $r(A, B, C)$, with an index on attribute A. Give an example of a query that can be answered by using the index only, without looking at the tuples in the relation. (Query plans that use only the index, without accessing the actual relation, are called *index-only* plans.)

16.27 Suppose you have an update query U. Give a simple sufficient condition on U that will ensure that the Halloween problem cannot occur, regardless of the execution plan chosen or the indices that exist.

Further Reading

The seminal work of [Selinger et al. (1979)] describes access-path selection in the System R optimizer, which was one of the earliest relational-query optimizers. Query processing in Starburst, described in [Haas et al. (1989)], forms the basis for query optimization in IBM DB2.

[Graefe and McKenna (1993)] describes Volcano, an equivalence-rule–based query optimizer that, along with its successor Cascades ([Graefe (1995)]), forms the basis of query optimization in Microsoft SQL Server. [Moerkotte (2014)] provides extensive textbook coverage of query optimization, including optimizations of the dynamic programming algorithm for join order optimization to avoid considering Cartesian products. Avoiding generation of plans with Cartesian products can result in substantial reduction in optimization cost for common queries.

The bibliographic notes for this chapter, available online, provides references to research on a variety of optimization techniques, including optimization of queries with aggregates, with outer joins, nested subqueries, top-K queries, join minimization, optimization of update queries, materialized view maintenance and view matching, index and materialized view selection, parametric query optimization, and multiquery optimization.

Bibliography

[Graefe (1995)] G. Graefe, "The Cascades Framework for Query Optimization", *Data Engineering Bulletin*, Volume 18, Number 3 (1995), pages 19–29.

[Graefe and McKenna (1993)] G. Graefe and W. McKenna, "The Volcano Optimizer Generator", In *Proc. of the International Conf. on Data Engineering* (1993), pages 209–218.

[Haas et al. (1989)] L. M. Haas, J. C. Freytag, G. M. Lohman, and H. Pirahesh, "Extensible Query Processing in Starburst", In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1989), pages 377–388.

[Moerkotte (2014)] G. Moerkotte, *Building Query Compilers*, available online at http://pi3.informatik.uni-mannheim.de/~moer/querycompiler.pdf, retrieved 13 Dec 2018 (2014).

[Selinger et al. (1979)] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access Path Selection in a Relational Database System", In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1979), pages 23–34.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.